

Microservices Knowledge Boost

Copyright © 2026 Titus. All rights reserved.

No part of this document may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without prior written permission of the author.

A brief about this eBook:

This book focuses on **how microservices work in the real world**, not just in theory. It follows a structured learning path that begins with basics and gradually introduces real scenarios faced in production systems.

Instead of abstract definitions, concepts are explained using practical situations that engineers actually deal with.

What's included:

- Microservices concepts explained through scenarios
- Common design and architecture questions
- Code examples to show how things are implemented
- Real-world use cases for every major microservices concept
- Reference articles for deeper learning

Category	Questions	Page Number
Microservices Conceptual Questions	134	12
Microservices Scenarios	60	167

Microservices and system design are not about writing code. They are about understanding how systems behave, scale, and fail. They need strong basics of Java and Spring.

Java & Spring Knowledge boost eBook: <https://topmate.io/codinglyf/1746709> . This book focuses deeply on **Java and Spring**, explaining concepts in a clear and practical way.

Contents

1. Microservice: Content Outline	8
2. Microservice 1: What is Microservice?	12
3. Microservice 2: What is the difference between Monolithic and microservice?	12
4. Microservice 3: Is Microservices an Architectural Pattern?	14
5. Microservice 4: Difference between Architectural Pattern and Design pattern?	14

6.	Microservice 5: What is architectural pattern?	15
7.	Microservice 6: What is a Distributed System?	15
8.	Microservice 7: What are the advantages of Microservice?	16
9.	Microservice 8: What are the disadvantages or challenges of Microservice?	17
10.	Microservice 9: What are the challenges you see while developing the microservices?	18
11.	Microservice 10: When to choose monolithic and when to use microservice?	19
12.	Microservice 11: What is load balancer and how it plays important role in microservices? 20	
13.	Microservice 12: What is are the types of load balancers?	21
14.	Microservice 13: How load balancing works internally or Algorithms of Load Balancer? ...	22
15.	Microservice 14: Explain Microservice architecture?	23
16.	Microservice 15: What are the design patterns of microservice.?	24
17.	Microservice 16: What is the Domain Driven Design Pattern?	27
18.	Microservice 17: What is Decomposition pattern or how you decompose a business into microservice?	28
19.	Microservice 18: What is Strangler Pattern?	28
20.	Microservice 19: What are benefits of using Strangler Pattern?	29
21.	Microservice 20: What are disadvantages or challenges of Strangler pattern?	30
22.	Microservice 21: What is Side Car Pattern?	30
23.	Microservice 22: What are the Database design patterns in microservices?	31
24.	Microservice 23: What is distributed transaction?	32
25.	Microservice 24: What is data consistency and how we achieve in Microservices?	33
26.	Microservice 25: What is Saga design pattern?	34
27.	Microservice 26: How do you implement saga pattern in Spring?	35
28.	Microservice 27: What are advantages and disadvantages of choreography pattern?	38
29.	Microservice 28: What are advantages and disadvantages of orchestration pattern?	38
30.	Microservice 29: Questions on Saga pattern	39
31.	Microservice 30: What is Two-Phase commit pattern?	40
32.	Microservice 31: What is difference between Saga and Two-Phase commit pattern?	42
33.	Microservice 32: When to use Saga and Two-Phase Commit?	43
34.	Microservice 33: What is CQRS Pattern?	43
35.	Microservice 34: When to use CQRS Pattern?	46

36.	Microservice 35: Questions on CQRS Pattern	46
37.	Microservice 36: What is Event-Driven database pattern?	47
38.	Microservice 37: What is Event Driven Architecture?	48
39.	Microservice 38: What is Eventual Consistency?	49
40.	Microservice 39: What is the recommended approach when two microservices using the same database?.....	50
41.	Microservice 40: What is communication pattern and its types?	51
42.	Microservice 41: How do you achieve Synchronous Communication in Microservices?	51
43.	Microservice 42: What do you know about Graph QL?	54
44.	Microservice 43: Questions on GraphQL?	57
45.	Microservice 44: When to use GraphQL?.....	60
46.	Microservice 45: What is gRPC and how it is useful?	61
47.	Microservice 46: What is asynchronous communication and how it is useful?	62
48.	Microservice 47: How can you implement Asynchronous communication in Microservice? 63	
49.	Microservice 48: When to use Asynchronous and when to use synchronous communication?.....	64
50.	Microservice 49: How does Apache Kafka works and why it is fast?	64
51.	Microservice 50: Explain the Kafka architecture?	67
52.	Microservice 51: Can you tell some real-world use cases for Kafka?	68
53.	Microservice 52: What are message queues?	69
54.	Microservice 53: What is message broker?.....	71
55.	Microservice 54: When to use message queues?	72
56.	Microservice 55: When to use Kafka, RabbitMQ and Pub/sub?	73
57.	Microservice 56: Implementations of Kafka and RabbitMQ in Spring boot?	74
58.	Microservice 57: Difference between Kafka and RabbitMQ?	74
59.	Microservice 58: What are the integration patterns used in Microservice?	75
60.	Microservice 59: What is API gateway pattern and why we use it?	76
61.	Microservice 60: What are the features of API gateway?	77
62.	Microservice 61: What are the ways to implement API gateway in Spring?	78
63.	Microservice 62: How API gateway works as load balancer?.....	81
64.	Microservice 63: What is difference between API gateway and load-balancer?.....	82

65.	Microservice 64: Can API Gateway replace load balancer completely?	82
66.	Microservice 65: Real time uses of API Gateway?	83
67.	Microservice 66: Some more Questions on API Gateway	83
68.	Microservice 67: If API gateway is single entry point, how does it handle millions of requests?	85
69.	Microservice 68: What is the Aggregator pattern?	86
70.	Microservice 69: What is the API Composition and how does it different from Aggregator?	88
71.	Microservice 70: How API Composition is different from GraphQL?	89
72.	Microservice 71: What is service discovery and service registry?	89
73.	Microservice 72: What is Server-side discovery and Client-side discovery.?	90
74.	Microservice 73: When to choose Server-side discovery and Client-side discovery?	91
75.	Microservice 74: How do you implement Service registry in Spring?	92
76.	Microservice 75: How Eureka server works internally?	96
77.	Microservice 76: If two or more instances of same service are running, how Eureka client decides which service to call?	97
78.	Microservice 77: What is the difference between API Gateway and Service Discovery?	98
79.	Microservice 78: How API Gateway and Eureka server works together?	99
80.	Microservice 79: Is Service discovery mandatory to have in distributed systems?	101
81.	Microservice 80: What is the deployments pattern we use Microservice?	102
82.	Microservice 81: What is End to End Deployment Steps we follow in Microservice?	102
83.	Microservice 82: What is containerization and why it useful?	106
84.	Microservice 83: How are Docker and Kubernetes related to containers??	109
85.	Microservice 84: What is Docker, how it useful and how it works?	110
86.	Microservice 85: Difference of Virtual machine, container and Docker?	112
87.	Microservice 86: List out some docker commands?	113
88.	Microservice 87: What is Kubernetes and how it is useful in Microservices?	114
89.	Microservice 88: Explain the architecture of Kubernetes?	115
90.	Microservice 89: What are the types of deployments?	117
91.	Microservice 90: What is the role of continuous integration (CI) and continuous deployment (CD) in Microservices??	118
92.	Microservice 91: Can we use embedded server in microservice deployment for production?	119

93.	Microservice 92: What observability design pattern?.....	120
94.	Microservice 93: What is monitoring in the microservices?	122
95.	Microservice 94: What is centralized logging in the microservices?	123
96.	Microservice 95: What is distributed tracing in the microservices?.....	123
97.	Microservice 96: What are the best practices we use for monitoring?	124
98.	Microservice 97: What are tools and techniques we use for monitoring and logging in the microservices?.....	124
99.	Microservice 98: What are the cross-cutting pattern in Microservices?	125
100.	Microservice 99: What is Resilience or Resilient System?.....	127
101.	Microservice 100: How do you achieve resilience?.....	128
102.	Microservice 101: What is Fault Tolerance?.....	128
103.	Microservice 102: What is the difference between fault tolerance and Resilience?	129
104.	Microservice 103: What is circuit-breaker pattern?	129
105.	Microservice 104: How do you implement Circuit breaker in Spring?	131
106.	Microservice 105: When to use Circuit breaker in Spring?.....	133
107.	Microservice 106: What is retry mechanism and how you achieve it?	134
108.	Microservice 107: What is Rate limiting and how do you implement in Microservices? ..	136
109.	Microservice 108: How can you implement Rate Limiter in Spring?	137
110.	Microservice 109: What are the Rate limiting algorithms?.....	139
111.	Microservice 110: Explain about Token bucket algorithm	140
112.	Microservice 111: What is the Bulkhead pattern, and how does it prevent system-wide failures?.....	143
113.	Microservice 112: How to implement bulkhead pattern using Spring?	144
114.	Microservice 113: What is difference between Rate-limiting and bulkhead pattern?	145
115.	Microservice 114: How do you handle failures in Microservices?	146
116.	Microservice 115: Explain about security in microservices?	147
117.	Microservice 116: How do you manage the configurations in distributed systems?	148
118.	Microservice 117: What is Local-Cache and Distributed-Cache?.....	150
119.	Microservice 118: What is In-memory caching and give any example?	151
120.	Microservice 119: How do you achieve caching in distributed systems?	152
121.	Microservice 120: What are pros and cons of caching?	153
122.	Microservice 121: What is Cache Invalidation?.....	153

123.	Microservice 122: Explain Cashe Aside, Write Through and Write-Behind?	155
124.	Microservice 123: Where would you place caching and which is best (DB level, Service Level or API Gateway)?	156
125.	Microservice 124: What is Spring cloud and how it is useful in Microservices?	157
126.	Microservice 125: What do you know about feign-client?	158
127.	Microservice 126: How Feign-client work internally?	160
128.	Microservice 127: How does Feign Client know which service URL to call in a microservice system?	160
129.	Microservice 128: How Feign client works if Service registry is not used in the project? .	161
130.	Microservice 129: How feign-client is different from Rest -Template and Web Client?	161
131.	Microservice 130: When to use feign-client, Rest -Template and Web Client?	162
132.	Microservice 131: What are the advantages of Feign Client?	163
133.	Microservice 132: What is service mesh and how it is useful in microservices??	163
134.	Microservice 133: How service mesh is different from API Gateway?	165
135.	Microservice 134: Explain service mesh architecture?	166
136.	Scenario 1: Monolith to Microservice?	167
137.	Scenario 2: Monolith to Microservice: Billing system	169
138.	Scenario 3: Authentication Token Refresh - Streaming App.....	170
139.	Scenario 4: Shared data between multiple services of Database.....	171
140.	Scenario 5: Selection of Database.....	171
141.	Scenario 6: Book a Flight - Travel App.....	172
142.	Scenario 7: Bank Account Transfer Between Two Different Databases.....	172
143.	Scenario 8: Multiple instances of a Service	173
144.	Scenario 9: Communication between two services - 1	174
145.	Scenario 10: Communication between two services - 2.....	174
146.	Scenario 11: Invoke API from Frontend	174
147.	Scenario 12: Real-Time Data Requirements	175
148.	Scenario 13: API Gateway	176
149.	Scenario 14: Rate Limiting	176
150.	Scenario 15: API Aggregation or Composition	178
151.	Scenario 16: Traffic routing	180
152.	Scenario 17: Secure the microservices.....	180

153.	Scenario 18: Routing	181
154.	Scenario 19: Handling errors	181
155.	Scenario 20: Event Driven System	182
156.	Scenario 21: Synchronous API Processing	183
157.	Scenario 22: Real-time Event Streaming	184
158.	Scenario 23: Real-time Analytics vs Task Queue	185
159.	Scenario 24: Order Processing in E-Commerce	186
160.	Scenario 25: Microservices Communication	186
161.	Scenario 26: Payment Retry System	186
162.	Scenario 27: Real-Time Fraud Detection	187
163.	Scenario 28: Deploying 50 Microservices - Docker vs Kubernetes	187
164.	Scenario 29: Zero Downtime Deployment	187
165.	Scenario 30: Managing Dev, QA, Stage, and Production Environments	188
166.	Scenario 31: Handling Traffic Spikes	188
167.	Scenario 32: Ensuring Backward Compatibility when a breaking changes in a Microservice: 188	
168.	Scenario 33: Managing Dev, QA, Stage, and Production Environments	189
169.	Scenario 34: Choosing deployment strategy	189
170.	Scenario 35: Service Failure	189
171.	Scenario 36: Retry without causing overload	190
172.	Scenario 37: Fallback on failure	190
173.	Scenario 38: Limit the resources	191
174.	Scenario 39: Dynamic IPs in microservices	191
175.	Scenario 40: Load Balancing without an external Load Balancer	191
176.	Scenario 41: Handling service failures using service registry	192
177.	Scenario 42: Handling usage spikes	192
178.	Scenario 43: Load balancing inside microservices	192
179.	Scenario 44: Some servers are overloaded while others are idle	193
180.	Scenario 45: Rate-limiting	193
181.	Scenario 46: Debugging Failures in a Microservice Chain	193
182.	Scenario 47: Handling High Volume Logs	194
183.	Scenario 48: Detecting Problems Before Users Report Them	194

184.	Scenario 49: Debugging Slow or failing Requests	194
185.	Scenario 50: Partial Failure	194
186.	Scenario 51: Data consistency -1.....	196
187.	Scenario 52: Data consistency -2.....	196
188.	Scenario 53: Sending notifications / Broadcast an event.....	197
189.	Scenario 54: Scaling the microservice	198
190.	Scenario 55: Polling.....	198
191.	Scenario 56: Scheduler + Multiple Instances	199
192.	Scenario 57: API is failing.....	200
193.	Scenario 58: Security	200
194.	Scenario 59: Read heavy Service.....	201

1. **Microservice:** Content Outline

1. Microservice Basics

- What is Microservice?
- Difference between Monolithic and Microservices
- Is Microservices an Architectural Pattern?
- Difference between Architectural Pattern and Design Pattern
- What is Architectural Pattern?
- What is a Distributed System?
- Advantages of Microservices
- Disadvantages of Microservices
- Challenges in Microservices
- When to Choose Monolithic and When to Use Microservices

2. Load Balancing

- What is Load Balancer
- Types of Load Balancers
- Load Balancing Algorithms

3. Microservice Architecture

- Explain Microservice Architecture
- Microservice Design Patterns Overview

4. Decomposition Design Patterns

- Domain Driven Design Pattern
- What is Decomposition Pattern
- What is Strangler Pattern

- Benefits of Strangler Pattern
- Challenges of Strangler Pattern
- What is Sidecar Pattern

5. Database Design Patterns

- What are Database Patterns in Microservices
- What is Distributed Transaction
- What is Data Consistency
- What is Saga Pattern
- Saga Design Pattern: Choreography and Orchestration
- Advantages of Choreography Pattern
- Disadvantages of Choreography Pattern
- Advantages of Orchestration Pattern
- Disadvantages of Orchestration Pattern
- Saga Pattern Interview Questions
- Saga Implementation using Spring.
- What is Two-Phase Commit
- Difference Between Saga and Two-Phase Commit
- When to Use Saga and When to Use Two-Phase Commit
- What is CQRS Pattern
- When to Use CQRS Pattern
- CQRS Pattern Interview Questions
- What is Event-Driven Database Pattern
- What is Eventual Consistency

6. Communication Patterns

- What is Communication Pattern
- Types of Communication in Microservices
- Synchronous Communication
- What is GraphQL
- GraphQL Technical Questions
- When to Use GraphQL
- What is gRPC
- Asynchronous Communication
- Implementing Asynchronous Communication
- When to Use Synchronous and Asynchronous Communication

7. Message Brokers & Streaming

- What is Kafka
- How Kafka Works
- Kafka Architecture
- Kafka Real-World Use Cases
- What are Message Queues

- What is Message Broker
- When to Use Message Queues
- When to Use Kafka, RabbitMQ, or Pub/Sub

8. API Gateway

- What is API Gateway
- Why Use API Gateway
- Features of API Gateway
- How API Gateway Works
- Implementation of API Gateway in Spring.
- API Gateway vs Load Balancer
- Zuul API Gateway
- Spring Cloud Gateway
- Rate Limiting in API Gateway
- Logging in API Gateway

9. Service Discovery

- What is Service Discovery
- Client-Side vs Server-Side Discovery
- Eureka Server
- Eureka Client
- How Eureka Works
- Heartbeats in Eureka
- Service Discovery Tools Comparison

10. Deployment patterns

- End to End deployment steps.
- Docket and its uses
- Virtual Machine vs Docker
- Kubernetes and its architecture
- Types of deployments

11. Logging and Monitoring

- What is Monitoring in Microservices
- Centralized Logging
- Log Correlation ID
- Distributed Tracing
- ELK Stack
- Zipkin
- Jaeger
- Sleuth
- Prometheus
- Grafana

12. Fault Tolerance and Resilience

- Why Fault Tolerance is Needed
- Retry Pattern
- Timeout Pattern
- Bulkhead Pattern
- Circuit Breaker Pattern
- Circuit Breaker States
- Rate Limiting Pattern
- Resilience4j Overview

13. Security in Microservices

- Security Challenges in Microservices
- OAuth2 in Microservices
- JWT Authentication
- Role-Based Access Control
- API Gateway Security
- Token Validation
- SSL/TLS Usage

13. Configuration Management

- Centralized Configuration
- Spring Cloud Config Server
- Config Repo with Git
- @RefreshScope Usage

14. Caching

- What is Caching
- Cache Strategies
- Local Cache vs Distributed Cache
- Redis in Microservices
- Cache Invalidation problems
- Write-Through, Write-Behind, Cache-Aside patterns

15. Spring cloud

- Use of Spring cloud
- library and its usage
- Feign client and its purpose
- Service Mesh

Coding Lyf

2. Microservice 1: What is Microservice?

Think of a big application like a single large box. Everything is kept inside that one box. If something inside breaks, the whole box has to be opened and fixed. Even a small change means touching the entire thing. This is how a **monolithic** system works.

A microservice is a different way of building applications. Instead of putting everything in one big box, the system is divided into many small boxes. Each box does one clear job. These small boxes are called microservices. They work together but can run on their own.

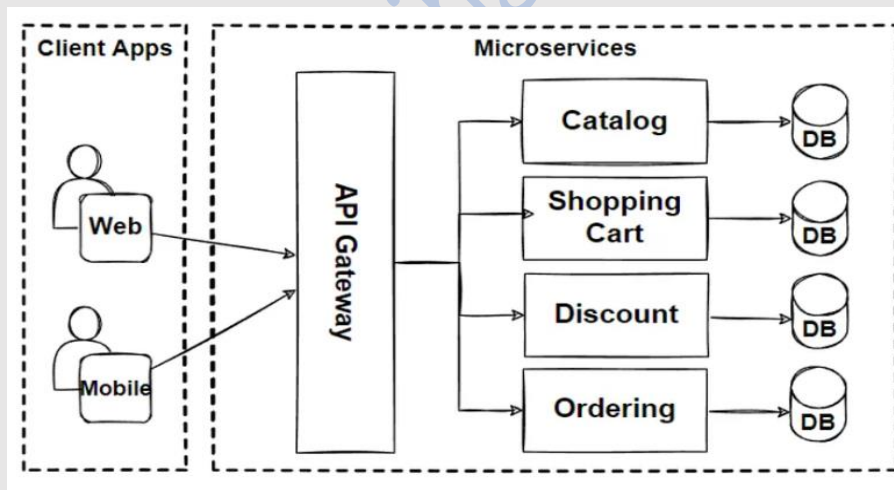
Imagine a simple food delivery app.

- One service takes care of Login and creating accounts.
- One service shows restaurants and menus.
- One service handles order.
- One service manages payments.

If the payment part has a problem, the rest of the app still works. Menus can be checked and orders can be placed. Once the payment service is ready, it joins back without disturbing others.

Definition:

A Microservice is a small, independent, and self-contained piece of software that does one specific job well. Instead of one big application, we break it into many smaller services. Each service runs on its own, can be developed and deployed separately, and communicates with others through lightweight APIs (usually HTTP or messaging queues).



3. Microservice 2: What is the difference between Monolithic and microservice?

Software can be built in two common ways: as one big system or as many small independent parts. These two styles are called **Monolithic architecture** and **Microservice architecture**.

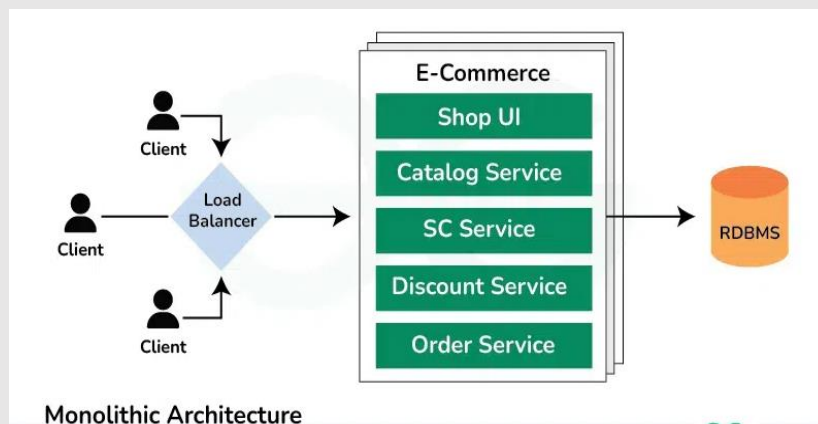
What is Monolithic?

In a monolithic application, everything lives in a single codebase. All features are developed, built, and deployed together.

For example, imagine a simple e-commerce application. It has user login, product listing, cart, payment, and order history. In a monolithic system, all these parts sit together in one project and use one database.

This is easy to build in the beginning. But as the application grows, problems start.

- A small change needs a full project build
- Deployment becomes slow and risky
- One bug can bring down the whole system
- It is hard for large teams to work together in the same codebase

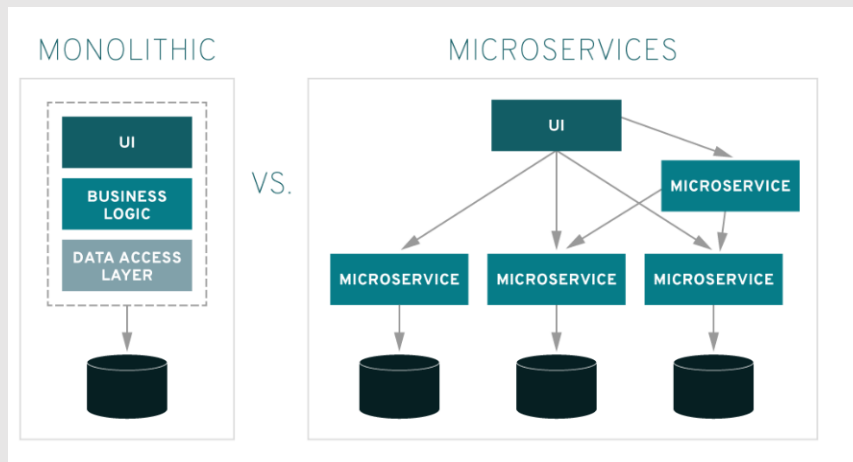


What is Microservice?

Microservice architecture divides a large system into small services. Each service handles only one business function and runs separately. For example, in the same e-commerce system:

Feature	Microservice
User login	User Service
Product listing	Product Service
Cart operations	Cart Service
Payments	Payment Service
Orders	Order Service

Each service has its own code, can use its own database, and is deployed on its own. If the payment service fails, the rest of the system still works.



Reference: [Microservices vs. monolithic architecture | Atlassian](#)

4. **Microservice 3:** Is Microservices an Architectural Pattern?

Yes, **microservices is an architectural style**. It is a way to design software by breaking a large system into small services that can run independently. Each service handles one business function, such as login, payment, product search, or order management.

Every service in microservices:

- Can be developed and deployed separately
- Can use its own database
- Can communicate with other services through APIs
- Can scale on its own based on load

5. **Microservice 4:** Difference between Architectural Pattern and Design pattern?

Architectural Pattern?

An architectural pattern defines the overall structure of a software system. It decides how different parts of the application are organized and how they communicate. It gives a big picture view.

Examples of architectural patterns:

- Layered Architecture
- Microservices Architecture
- Event-driven Architecture
- Client-Server Architecture

These patterns focus on high-level decisions. They guide how modules are arranged, how data flows, and how the system is deployed

Design Pattern?

A design pattern solves common coding problems inside a single module or class. It does not talk about the whole system. Instead, it focuses on how to write clean and reusable code.

Examples of design patterns:

- Singleton
- Factory Method
- Builder
- Strategy
- Observer

These patterns focus on class-level design and object interaction.

Simple Example

Think of constructing a building.

- The architectural pattern is like the building layout. It decides where the kitchen goes, how many floors there are, and where rooms connect.
- The design pattern is like the interior design. It decides how to arrange furniture inside a room and how switches and lights are placed.

6. **Microservice 5: What is architectural pattern?**

An architectural pattern is a way of designing the structure of a software system. It gives a clear guideline on how to organize system, how different parts of the system should interact, and how the application should grow in the future. It is not a programming language or a framework. It is simply a design approach that solves common problems in software structure.

Some common architectural patterns are:

Pattern	Idea
Layered Architecture	Code is divided into layers like controller, service, and repository
Event-Driven Architecture	System reacts to events
Microservices Architecture	System is split into small independent services
Client-Server Architecture	Frontend and backend are separated
Pipe and Filter Architecture	Data passes through processing steps

7. **Microservice 6: What is a Distributed System?**

A distributed system is a group of many computers that work together like a single system. These computers are often placed in different locations, but they talk to each other through a network to finish one common task.

Instead of putting all the work on one machine, we divide the work into smaller parts and give it to many machines. Each machine does its own job and then shares the result with others. This makes the system faster and more reliable.

Let's take a simple example. Think about an online shopping website. When we place an order, different things happen at the same time:

- One service checks if the product is available.
- Another service takes care of the payment.
- Another service handles the delivery.

Each of these services can run on a different machine. But to us, it looks like one single smooth process. All these machines talk to each other behind the scenes to make it happen.

The main goal of a distributed system is to share the load, work faster, and keep the system running even if one part fails. This is how large systems like online stores, banking apps, and streaming platforms work every day



Reference: [Distributed Systems: Basics. A distributed system is a system of... | by Naveenjain | Medium](#)

8. **Microservice 7:** What are the advantages of Microservice?

When an application is made of many small services instead of one big system, it gives us many benefits. Each service has its own job, and that makes work easier to build and manage.

1. Easy to build and change

Since every service is small, it is easier to understand what it does. If something needs to be updated, only that small part is touched. There is no need to touch the full system. This saves time and reduces mistakes.

Example: In a food delivery app, if the restaurant menu service needs a small change, only that part is updated. The order and payment parts stay safe and continue to work.

2. Faster development

Different teams can work on different services at the same time. One team can improve the login part while another team adds a new payment method. This makes development move faster.

Example: One team adds Google Pay to the payment service while another team improves the search feature in the restaurant service. Both changes can happen without waiting for each other.

3. Better fault control

If one-part stops working, the full application does not stop. Other services can still run. This helps reduce downtime and keeps the system stable.

Example: If the payment service has a problem, users can still browse restaurants and add food to their cart. Once the payment is fixed, everything connects back smoothly.

4. Easy to scale

Some parts of an application may get more traffic than others. Those parts can be scaled up without touching the rest of the system.

Example: During lunch time, the order service may get more traffic. Only that part can be scaled up to handle more orders, while the other services stay as they are.

5. Simple maintenance

Since each service is smaller and separate, finding and fixing bugs is easier. Testing becomes simple and updates become faster.

Example: If a bug is found in the restaurant listing service, it can be fixed and deployed quickly without affecting login, orders, or payment.

6. Polyglot

This is one of the significant aspects of microservices architecture. For *example*, if a team is working on one of the microservice using Java, Spring Boot, and MySQL, another team can work on another microservice using Python, Node JS, and NoSQL.

9. **Microservice 8: What are the disadvantages or challenges of Microservice?**

Microservices give many benefits, but they also bring a few challenges. When an application is broken into many small parts, managing everything can become more complex. Understanding these drawbacks helps us plan better.

1. Hard to manage many services

A single application can have dozens of microservices. Each one needs to be monitored, updated, and deployed carefully. Keeping track of all services can take extra time and effort.

Example: In a food delivery app, if there are ten services, like login, restaurant menu, order, payment, delivery, and more, each one needs to be checked regularly. If one service fails, finding the problem can take longer.

2. More communication between services

Since microservices talk to each other to get work done, the number of calls between services increases. If one service is slow or fails, it can affect others.

Example: When an order is placed, the order service talks to both the restaurant service and the payment service. If the restaurant service takes too long to respond, the whole order process slows down.

3. Difficult testing

Testing one single system is simple, but testing many services together can be tricky. All services need to work properly with each other, not just alone.

Example: The payment service may work fine alone, and the order service may work fine alone. But when they are connected, new issues may appear that were not seen before.

4. More deployment steps

In a monolithic system, there is only one thing to deploy. In a microservices system, each service can have its own deployment process. Handling many deployments can be more time-consuming.

Example: If the food delivery app has 12 services, some may be updated weekly and some daily. All deployments must be tracked and managed carefully.

10. **Microservice 9: What are the challenges you see while developing the microservices?**

Building microservices gives us flexibility, but it also brings some real challenges in complexity, data consistency, and inter-service communication.

1. Clear service boundaries

The first challenge is deciding what each service should do. If the boundaries are not clear, services can start mixing responsibilities. This can create confusion and extra work later.

Example: In a food delivery app, if both the order service and the payment service handle discounts, it becomes unclear which one should control it. A better plan would be to keep discounts inside one clear service.

2. Communication between services

Each service needs to talk to others to get things done. If this communication is not handled well, the system can become slow or break in unexpected ways.

Example: When an order is placed, the order service talks to the payment service and the restaurant service. If these connections are not planned properly, the whole order flow can fail.

3. Handling failures

Since there are many services, there are more chances for something to fail. A single failed service should not stop the full system. Handling failures carefully is one of the biggest challenges.

Example: If the payment service is down, the order service should not crash. It should handle the error and give a clear message.

4. Data consistency

In a monolithic system, all data sits in one place. In microservices, each service may have its own data. Keeping data in sync across many services is not easy.

Example: If an order is placed and the payment succeeds but the delivery service does not update the status, the system can show wrong information to the user.

5. Testing the whole system

Testing each service alone is simple. But testing how all services work together is harder. Small changes in one service can affect other services.

Example: If a small change is made in the login service, it can affect how the order service receives user information. This needs careful testing.

6. Deployment and version control

Each service has its own version and its own deployment cycle. Making sure all versions work together is a challenge.

Example: If the payment service is updated to a new version, but the order service is still using an old version, they might not talk properly.

11. **Microservice 10:** When to choose monolithic and when to use microservice?

Deciding between a monolithic application and microservices depends on the size of the project, the team, and how the system is expected to grow.

When to Choose Monolithic

A monolithic system is simpler and easier to start with. It is best when the application is small, has fewer features, and the team is small. Since everything is in one place, building, testing, and deploying is easier.

Example: If we are building a small online bookstore with only a few features like browsing books, adding them to a cart, and making payments, a monolithic system is enough. All the code sits together, making it easier to manage at the start.

When to Choose Microservices

Microservices are better when the application is large, has many features, or is expected to grow. They are also useful when different teams work on different parts of the system. Microservices allow independent development, easy scaling, and better fault control.

Example: In a large food delivery app, there are many features like login, restaurant listing, order tracking, payments, ratings, and delivery tracking. Splitting these into microservices allows each feature to grow independently. Teams can work on payments while another team improves the restaurant search without interfering with each other.

Key Points to Decide

- If the project is small, simple, and unlikely to grow quickly, monolithic is faster and easier.
- If the project is large, complex, and expected to scale with multiple teams, microservices are better.

- Microservices are more complex to manage, so the team should be ready for monitoring, testing, and deployment challenges.

12. Microservice 11: What is load balancer and how it plays important role in microservices?

In a microservice system, we never rely on just one instance of a service. If only one instance is running and it receives too many requests, it may slow down or even crash. To avoid this, we run **multiple instances** of the same service.

Example:

```
order-service running at port 8081
order-service running at port 8082
order-service running at port 8083
```

Now the challenge is:

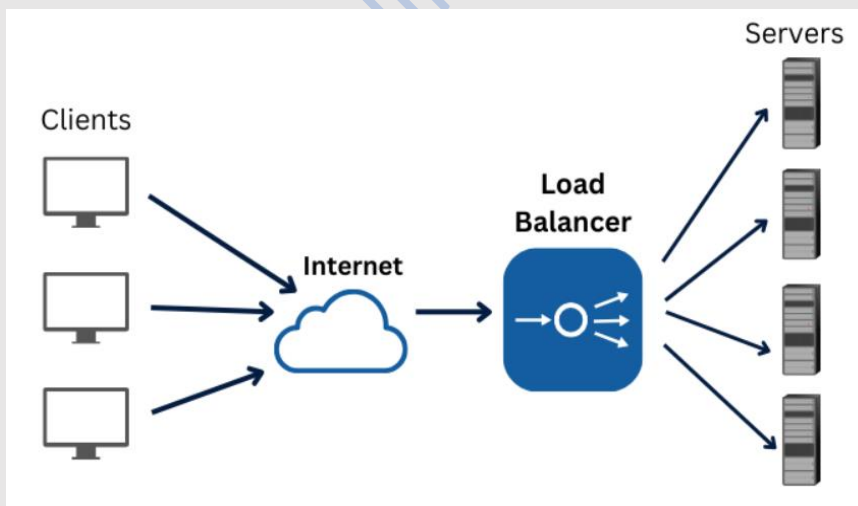
When a request comes, which instance should handle it?

If all requests go to only one instance, that instance gets overloaded while others sit idle. This is where a **Load Balancer** becomes important.

What Is a Load Balancer

A load balancer is a component that distributes incoming requests across multiple service instances. It makes sure no single instance gets overloaded and traffic is shared fairly.

Load balancer sits between the client and the microservices and plays the role of a traffic manager.



Real-Life Example

Think about a bank with three counters open. If all customers go to only one counter, it gets crowded and slow. So, a security guard at the entrance directs customers evenly to all counters.

Here,

Customers = Requests

Bank counters = Service instance

Security guard = Load Balancer

Load Balancer Tools Used in Spring Microservices

Tool	Type	Purpose
Spring Cloud LoadBalancer	Client-side	Built-in load balancer in Spring
Eureka + LoadBalancer	Client-side	Service discovery + load balancing
API Gateway (Spring Cloud Gateway)	Server-side	Routes and load balances
Nginx	Server-side	Works as reverse proxy and load balancer
Kubernetes LoadBalancer	Cluster level	Manages load balancing in containers
Ribbon (Deprecated)	Client-side	Old Netflix load balancer

Reference: [Load Balancing: System Design Fundamentals | by Abhirup Acharya | Medium](#)

Video link: [What is a LOAD BALANCER really about?](#)

13. **Microservice 12: What is are the types of load balancers?**

1. Client-Side Load Balancer

In this approach, the calling service decides which service instance to invoke. The client maintains a list of available instances, usually provided by a service registry, and distributes requests using a rule like Round Robin.

How it works

Calling Service > chooses instance ? calls target service

Correct example scenario

An Order Service calls Product Service. Product Service has multiple instances running:

- product-service:8081
- product-service:8082
- product-service:8083

The Order Service uses a client-side load balancer to select one instance for each request.

Tools used

- Spring Cloud LoadBalancer
- Feign Client
- RestTemplate with @LoadBalanced

2. Server-Side Load Balancer

Here, the client does not decide where to send the request. Instead, it sends the request to a load balancer or API Gateway, and that component forwards the request to one of the service instances.

How it works:

Client > API Gateway > service instance

Example scenario:

All requests go through: `http://api.company.com/products`

The gateway then sends them to one of the product-service instances.

Tools used: Spring Cloud Gateway, Nginx, AWS Elastic Load Balancer

14. Microservice 13: How load balancing works internally or Algorithms of Load Balancer?

Imagine a bank with three cash counters. If all customers go to counter 1 and ignore the other two, the work will be slow and messy. Instead, there must be a rule to guide each customer to a free counter. That rule is what we call a **load balancing algorithm** in software.

The same logic is applied in microservices to forward requests to different service instances.

Common Load Balancing Algorithms

Below are the most used load balancing methods in real systems.

1. Round Robin

This is the simplest and most common method.

The load balancer sends each incoming request to the service instances one by one in a circular order.

Example:

We have three instances:

```
payment-service:8081
```

```
payment-service:8082
```

```
payment-service:8083
```

Requests go like this:

```
1st request > 8081
```

```
2nd request > 8082
```

```
3rd request > 8083
```

```
4th request > 8081
```

```
5th request > 8082
```

2. Random Selection

As the name says, requests are sent to any instance randomly.

Example:

Requests may go like: `8082 > 8081 > 8083 > 8082 > 8081`

3. Least Connections

In this method, the request is assigned to the instance that has **the fewest active connections**. It is useful when each request takes a different amount of time to complete.

Example:

```
Current active requests: 8081 > 5 requests active
8082 > 2 requests active
8083 > 1 request active
Next request goes to -> 8083
```

4. Weighted Round Robin

This is a smarter version of Round Robin. If one service instance has **more power (CPU/RAM)**, we give it a **higher weight** so it can handle more traffic.

Example:

Weights:

```
8081 > weight 3
8082 > weight 1
8083 > weight 1
```

Order:

```
8081 > 8081 > 8081 > 8082 > 8083 > repeat: 8081 handles more traffic because it has higher capacity.
```

5. IP Hash

In this method, the request always goes to an instance based on **client IP address**. So, the same user always connects to the same backend instance.

Reference: [Load Balancing Algorithms](#)

[Load Balancing Algorithms Explained with Code \(and Visuals\)](#)

15. **Microservice 14:** Explain Microservice architecture?

Microservice architecture is the way we organize an application as a set of small, independent services. Each service has a clear job, manages its own data, and talks to other services when needed. This makes the system flexible, easier to maintain, and easier to scale.

Let's take a simple example to understand it better.

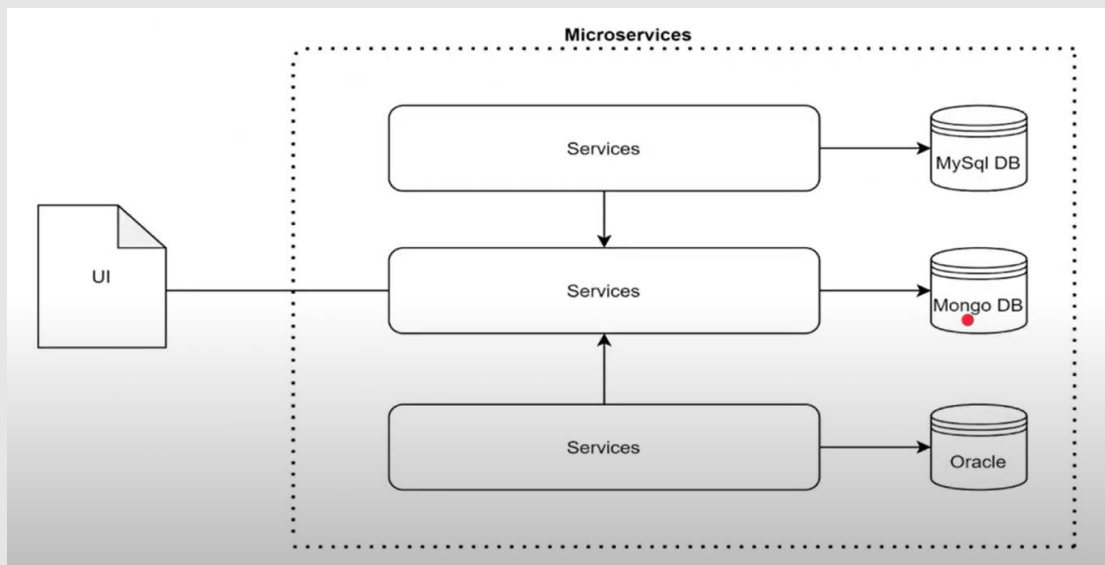
Example Scenario

Imagine an online shopping app. The **UI** is what the user sees. It could be a website or a mobile app. Behind the scenes; there are **three services**:

1. **User Service:** handles login, signup, and profile information
2. **Product Service:** manages product details, inventory, and categories
3. **Order Service:** handles placing orders, tracking them, and managing payments

Each service has its **own database**. For example:

- User Service uses **SQL** to store user details
- Product Service uses **Oracle** to store product information
- Order Service uses **MongoDB** to store orders and payment history



How the Services Work Together

When a user opens the app and places an order:

1. The **UI** sends a request to the **User Service** to verify the user.
2. Then the **Product Service** is asked to check if the product is available.
3. Once confirmed, the **Order Service** creates the order and processes the payment.

Even though the services talk to different databases and run independently, they communicate with each other through simple APIs.

Key Points

- Each service is **independent**. Changes in one service do not directly affect others.
- Each service **manages its own database**, which avoids conflicts and allows different types of storage.
- Services **talk only through defined APIs**, which keeps communication clear and simple.
- The UI can interact with multiple services at the same time to complete a single action.

16. Microservice 15: What are the design patterns of microservice.?

We can divide patterns to 7 types. Decomposition pattern, Database pattern, Communication pattern, Integration patterns, Deployment patterns, observability and cross cutting patterns.

Note: Each pattern contains sub patterns. We will see one by one from next question.

Microservices Patterns

1. Decomposition Patterns

- By Business Capability
- By Subdomain

2. Database Patterns

- Database per Service
- Shared Database

3. Communication Patterns

- Synchronous
- Asynchronous

4. Integration Patterns

- API Gateway
- Backend for Frontend (BFF)

5. Deployment Patterns

- Single Service Deployment
- Multiple Service Deployment

6. Observability Patterns

- Logging
- Monitoring
- Tracing

7. Cross-Cutting Patterns

- Security
- Configuration Management
- Error Handling

1. Decomposition Patterns

This pattern is about deciding how to split a large application into smaller services. There are two main ways:

- **By Business Capability:** Each service handles a specific business area.
Example: In a shopping app, we can have a User Service, Product Service, and Order Service. Each does one clear job.
- **By Subdomain:** Each service is built around a specific part of the system.
Example: In a movie booking system, there can be a Booking Service, Payment Service, and Review Service. Each focus on a subdomain.

2. Database Patterns

Every microservice usually has its own database. This avoids conflicts and allows each service to choose the best database type.

- **Database per Service:** Each service has its own database.
Example: User Service uses SQL, Product Service uses Oracle, and Order Service uses MongoDB.
- **Shared Database:** Sometimes services share a database, but this can create tight coupling and is less preferred.

3. Communication Patterns

Microservices need to talk to each other to get work done. There are two main ways:

- **Synchronous Communication:** Services call each other directly and wait for a response.
Example: Order Service calls Payment Service to confirm payment before completing an order.

- **Asynchronous Communication:** Services communicate using messages and do not wait for a response.
Example: When a user places an order, Order Service sends a message to Notification Service to send an email. The order does not wait for the email to be sent.

4. Integration Patterns

This pattern deals with how microservices work with other systems or external applications.

- **API Gateway:** A single-entry point for clients to access multiple services.
Example: The mobile app calls the API Gateway, and it forwards requests to User, Product, and Order Services.
- **Backend for Frontend (BFF):** A service designed specifically to serve a particular UI.
Example: The mobile app and website have separate BFF services to optimize responses for each.

5. Deployment Patterns

Deployment patterns explain how services are deployed and managed.

- **Single Service Deployment:** Each service is deployed independently.
Example: Payment Service can be updated without touching Order Service.
- **Multiple Service Deployment:** Several services are deployed together as a group.
Example: Minor updates to Product Service and Order Service are deployed at the same time.

6. Observability Patterns

Observability patterns help us monitor and understand how services are performing.

- **Logging:** Keep records of important events in each service.
- **Monitoring:** Track health and performance of services.
- **Tracing:** Track requests as they move between services.

Example: If an order fails, tracing shows whether the problem was in the Order Service or Payment Service.

7. Cross-Cutting Patterns

These patterns handle tasks that affect many services, like security, configuration, and error handling.

- **Security:** Services check authentication and authorization.
Example: User Service validates login before allowing access to Order Service.
- **Configuration Management:** Centralized settings for all services.
Example: Database URLs or API keys are managed in a single place for all services.
- **Error Handling:** Standard way to handle failures and retries.
Example: If Payment Service fails temporarily, Order Service retries the request a few times.

Reference: [Microservice Architecture and Design Patterns for Microservices | by madhuka udantha | Medium](#)

17. **Microservice 16: What is the Domain Driven Design Pattern?**

Domain Driven Design, or DDD, is a way of building systems where the focus stays on the business problem first and the code second. Instead of jumping into classes and tables, we start by understanding what the business actually does. Once the business area is clear, the code follows the same shape.

The goal is simple. Make the system easy to change and easy to understand.

Core Idea

Every business has multiple areas. For Example, In a billing system, we have invoicing, payment, pricing, tax and customer accounts. Each area has its own rules. When one area mixes with another, the code becomes confusing. DDD helps avoid this by giving each area a clean boundary.

This clean boundary is called a **bounded context**.

What is Bounded Context?

A bounded context is nothing but a clear boundary around a part of our application where one specific meaning of data and rules is followed. Inside this boundary, everyone uses the same words with the same meaning, and the logic is the same. Outside this boundary, the meaning can change.

In simple words, each bounded context is like a separate world with its own rules.

Think of a college

In a college, different departments exist:

- **Computer Science Department**
- **Mechanical Department**
- **Civil Department**

Each department uses some common words, but the meaning changes.

For example, the word "**Project**":

- In **Computer Science**, a project might be a software application.
- In **Mechanical**, a project might be a machine model.
- In **Civil**, a project might be a building design.

The same word has different meanings depending on which department we are talking about. Each department follows its own rules, subjects, teachers, and marks calculation method. That is a **bounded context**.

In Microservices

In microservices, each service usually follows one bounded context. That means each service handles only one part of the business. It has its own data, logic, and meaning for the things it deals with. It does not depend on the internal details of other services.

Example: E-commerce application

Think of an online shopping system. It has many areas. Each area can be one bounded context.

1. Order Management

This service handles orders placed by customers.

- It has its own idea of what an **Order** is.
- It knows the **Order Status**, **Payment details**, and **Shipping details**.
- These rules do not mix with other services.

2. Inventory Management

This service deals with product stock.

- It has its own meaning for **Inventory**, **Stock**, and **Warehouse**.
- It knows when items are available or when stock is low.
- It does not care how orders are created. Its focus is stock.

3. Customer Management

This service handles customer data.

- It defines what a **Customer** is.
- It stores customer profile, address, and loyalty points.
- It does not worry about stock or order rules.

Each of these services works inside its own boundary. They may talk to each other when needed, but they never mix their internal rules.

Reference: [Understanding the Bounded Context in Microservices | Bits and Pieces](#)

18. **Microservice 17: What is Decomposition pattern or how you decompose a business into microservice?**

Decomposition pattern is about **how to break a big application into small and clear microservices**. It is the first and one of the most important steps when we design a microservice architecture.

If the system is not broken in a smart way, it can create confusion, repeated work, and make the system hard to manage later. Decomposition helps us keep each service simple, focused, and easy to scale.

In Decomposition pattern, as part from business capability and subdomain, we have two another patterns

1. Strangler pattern.
2. Sidecar pattern.

19. **Microservice 18: What is Strangler Pattern?**

The **Strangler pattern** is like slowly replacing an old system with a new one. Instead of breaking the old system all at once, we move one part at a time into microservices.

Example:

Imagine we have a big monolithic food delivery app. It has login, restaurant listing, orders, payments, and delivery tracking in one single block of code.

Instead of rewriting everything, we start with one part, say Payments.

1. We build a new Payment microservice.
2. We redirect only the payment requests from the old system to the new Payment service.
3. The rest of the features keep using the old system.
4. Over time, we move other parts like Orders and Delivery the same way.

This pattern lets us modernize the system step by step without shutting it down. It reduces risk and makes the transition smooth.

It's called "Strangler" because the new microservices slowly **replace and surround** the old system until the old system is no longer needed.

Reference: [Strangler Design Pattern Tutorial with Examples for Programmer & Beginners](#)

20. **Microservice 19:** What are benefits of using Strangler Pattern?

The Strangler Pattern gives a safe way to **move from an old system to a new system step by step**. Instead of replacing everything at once, we slowly build the new system around the old one and move one feature at a time. Over time, the old system gets smaller and eventually disappears.

Benefits of the Strangler Pattern

1. No big shutdown

The biggest advantage is that we do not need to stop the existing system. The business can continue normally while we build the new system piece by piece. This avoids long downtime and angry users.

2. Lower risk

Replacing a full system at once is risky. If something goes wrong, the whole system fails. With the Strangler Pattern, we move one feature at a time. If a new feature fails, only that part is affected, not the entire system. This lowers the risk of failure.

3. Works well with modernisation

It allows us to slowly modernise. For example, we can move one module from a monolithic Java application to microservices. Or we can move from REST to GraphQL, or from a SQL database to NoSQL, one step at a time.

4. Easy rollback

If something goes wrong in the new version of a feature, we can simply route traffic back to the old feature. Rollback becomes easy and safe.

5. Keeps business value running

The team can deliver improvements continuously instead of waiting for a big project release after 6–12 months. This keeps users happy and helps the business see value early.

6. Better code quality over time

Because we move feature by feature, we get time to clean code, improve design, write tests, and follow good engineering practices. This results in a healthier system overall.

Let us imagine we have an old online shopping website built as a big monolithic Java application. It has many features:

- User login
- Product search
- Cart
- Orders
- Payments

The team wants to move to microservices because the monolith is slow and hard to maintain. Instead of rewriting everything, they apply the Strangler Pattern like this:

1. First, they create a new Product Service as a separate microservice.
2. They add an API Gateway in front of the old system and route only /products calls to the new service.
3. Everything else still goes to the old monolith.
4. After product search is moved successfully, they migrate /cart next.
5. Then /orders, then /payments, and so on.
6. In the end, the monolith has no features left and can be removed.

21. Microservice 20: What are disadvantages or challenges of Strangler pattern?

- It takes time. Since we move feature by feature, full migration can take months or even years.
- Extra complexity. During migration, we maintain both old and new systems, which increases effort.
- Data syncing issues. Old and new services may use different databases, so keeping data in sync becomes difficult.

22. Microservice 21: What is Side Car Pattern?

The **Sidecar pattern** is about **attaching a helper service** next to the main service. The helper service adds extra features without changing the main service.

Example:

Let's say we have an Order Service. We want to add logging and monitoring but don't want to change its code.

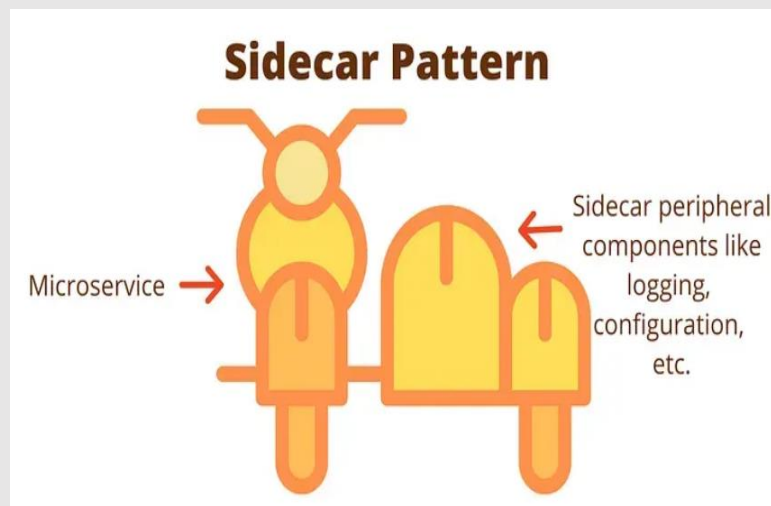
We can create a Sidecar service that sits next to the Order Service.

- The Order Service continues doing its job.
- The Sidecar service collects logs, metrics, and other useful information.
- Both run together in the same environment.

The sidecar is like a trusted assistant. It supports the main service but doesn't interfere with its logic. This makes it easy to add security, monitoring, or configuration without touching the main application.

A **sidecar service** can handle concerns such as:

- **Observability:** Centralized logging, distributed tracing (e.g., OpenTelemetry, Prometheus, Datadog).
- **Security:** Identity management, API security, authentication proxies (e.g., Istio, Envoy).
- **Resilience:** Circuit breaking, fault tolerance (e.g., Hystrix, Resilience4j).



Reference: [The Sidecar Pattern Explained: Decoupling Operational Features](#)

Let's see 2nd Design pattern, that is Database design pattern

23. **Microservice 22:** What are the Database design patterns in microservices?

In a microservice architecture, every service needs to manage its own data in a clear and reliable way. **Database pattern** is the method we follow to decide how services will use and store their data. It helps us keep the data organized, avoid unwanted mixing between services, and make the system easy to grow in the future.

Think of it like a group of shops in a mall. Each shop has its own cash counter, store room, and records. One shop doesn't depend on another shop's records to run its business. In the same way, each microservice should handle its data without depending too much on other services.

Here are the common **types of database patterns** in microservices:

1. Database per service pattern

2. Shared database pattern
3. Saga pattern
4. CQRS (Command Query Responsibility Segregation) pattern
5. Event sourcing pattern
6. Two-Phase commit patterns

Let's see each pattern from next question.

24. **Microservice 23:** What is distributed transaction?

A distributed transaction is a single business operation that touches more than one system or database at the same time. All parts of the operation must succeed together. If any part fails, everything must be rolled back to keep data correct and safe.

This type of transaction is called *distributed* because it is spread across different services or databases, may be running on different servers or networks.

Why do we need it?

In small systems with one database, a normal database transaction is enough. But in modern applications, data is stored in multiple services. When one task involves data from two or more services, we need a distributed transaction to keep everything consistent.

Working of Distributed Transactions

The working of Distributed Transactions is the same as that of simple transactions but the challenge is to implement them upon multiple databases. Due to the use of multiple nodes or database systems, there arises certain problems such as network failure, to maintain the availability of extra hardware servers and database servers.

Real Life Example of Distributed transaction

- **E-commerce Checkout:** Payment, inventory update, and order creation must all succeed or rollback together to ensure data consistency.
- **Banking Fund Transfer:** Transferring money between accounts in different banks involves debiting one account and crediting another atomically across multiple databases.
- **Flight Booking System:** Booking a flight may involve reserving a seat, processing payment, and updating frequent flyer miles across separate systems; all must succeed or fail together.

Example

Let us take an example of a bank money transfer:

A customer transfers 5,000 rupees from Account A to Account B.

Steps:

1. Debit 5,000 from Account A (Database 1)
2. Credit 5,000 to Account B (Database 2)

Both steps must succeed together. If money is deducted from Account A but not added to Account B, we lose money. That is a **data inconsistency** and must not happen.

Here, the transfer is a distributed transaction because it touches two databases.

To manage distributed transactions safely, systems use special methods like:

- **Two-Phase Commit (2PC):** ensures both databases commit or rollback together (Refer question - [Microservice 30](#))
- **Saga Pattern:** uses a series of steps with compensation logic to maintain data (Refer question - [Microservice 25](#))

Reference: [What is a Distributed Transaction? - GeeksforGeeks](#)

25. **Microservice 24:** What is data consistency and how we achieve in Microservices?

Data consistency means keeping data **correct and reliable** across the system. When one operation updates data, every part of the system must reflect the same the date. There should be no mismatch.

Example

Let us take a simple wallet system:

- Balance: 1,000 rupees
- A user sends 200 rupees to a friend

After the transfer:

- Wallet balance should become 800
- Transaction history should record the payment
- The friend should receive 200

If any one of these results is missing or wrong, the system is inconsistent. This can cause trust issues and serious business problems.

In a monolithic system, all data stays in one database, so transactions are simple. In microservices, every service has its own database. Services communicate through APIs or messages. Because of this, keeping data in sync becomes harder.

For example, in an e-commerce system:

- Order Service stores order data
- Payment Service stores payment data
- Inventory Service stores stock data

When a customer places an order, all three services must stay in sync. If payment fails, the order should be cancelled and stock must be restored. This must happen safely to keep data consistent.

How We Achieve Data Consistency in Microservices

We use patterns and strategies to maintain data correctness across services.

Method	How it helps
Saga Pattern	Keeps multiple services in sync using step-wise transactions and rollback logic
Event-Driven Communication	Services publish and listen to events like OrderCreated or PaymentFailed
Two-Phase Commit (2PC)	Ensures strong consistency across databases, often used in banking
Idempotency	Prevents duplicate updates when a request is repeated

Example Using Saga Pattern

When placing an order:

1. Order Service creates order
2. Payment Service takes payment
3. Inventory Service reduces stock

If payment fails:

- Inventory Service restores stock
- Order Service cancels order

This step-by-step handling keeps data consistent across services.

26. **Microservice 25:** What is Saga design pattern?

Saga pattern is a way to handle transactions that span across many services. In a microservice system, one business action may need changes in multiple services. Each service has its own database, so a single big transaction is not possible. Saga pattern helps us break this big action into many small local transactions.

Each local transaction updates its own data and then passes a message or event to the next service. If one of the steps fails, Saga pattern rolls back the changes by calling compensation steps. This keeps the system in a proper state even when something goes wrong.

Let's take a small example. Think about an online food delivery app. When we place an order, three services are involved:

- Order service
- Payment service
- Delivery service

First, the Order service creates the order in its database. Then it calls the Payment service to process the payment. After payment is successful, it calls the Delivery service to arrange delivery.

If payment fails, Saga pattern calls a compensation step in the Order service to cancel the order. This way, we do not keep half-done work in the system.

Saga pattern gives us a simple way to keep data correct across services without depending on one big transaction. It works step by step, and if any step fails, it knows how to undo what is already done.

Reference Links:

Video: [SAGA Design Pattern for Microservices Tutorial with Examples for Software Programmers](#)

27. Microservice 26: How do you implement saga pattern in Spring?

Saga pattern can be built in two main ways in Spring. One is using **events**, and the other is using a **central controller**. Both help us manage a business action that runs across many services.

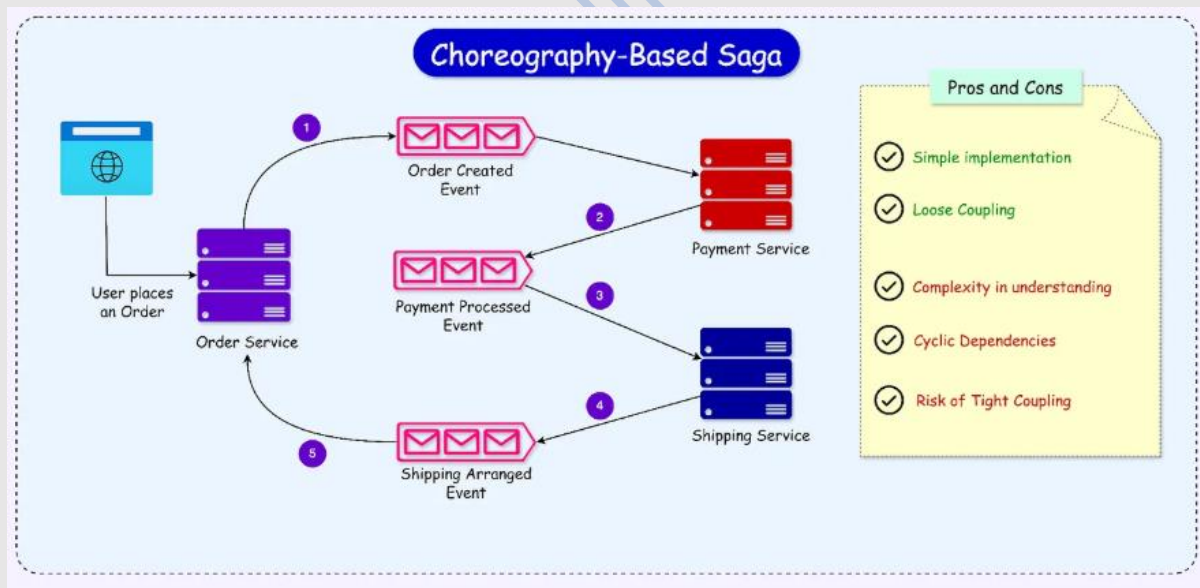
1. **Choreography Saga** - event based
2. **Orchestration Saga** - central coordinator

1. Choreography Saga

In this style, each service listens to events and reacts. There is no single service controlling everything. Let us take a small food delivery example.

1. Order service creates an order in its database and sends an OrderCreated event.
2. Payment service listens to this event. It processes the payment and sends a PaymentSuccess event.
3. Delivery service listens to this event and arranges the delivery.

If payment fails, Payment service sends a PaymentFailed event. The Order service listens to this and cancels the order.



Choreography-Based Saga Flow (Number-wise)

1. Order Created Event

The user places an order.

The Order Service saves the order in its database and publishes an **Order Created** event. It does not call any service directly. It only sends an event.

2. Payment Service reacts

The Payment Service is listening for the **Order Created** event. When it receives the event, it processes the payment and updates its own database.

3. Payment Processed Event

After payment is successful, the Payment Service publishes a **Payment Processed** event. Again, it does not call the next service directly. It only emits an event.

4. Shipping Service reacts

The Shipping Service listens for the **Payment Processed** event. Once it receives it, shipping is arranged and shipping details are saved

5. Shipping Arranged Event

The Shipping Service publishes a **Shipping Arranged** event. The Order Service listens to this event and updates the order status to shipped or completed.

Summary: In choreography-based saga, each service listens to events, performs its action, and publishes the next event, creating a chain without a central coordinator.

Reference: [Microservice Patterns: Saga Choreography | by Salitha Chathuranga | Medium](#)

Steps in Spring:

- Create events like OrderCreated, PaymentSuccess, and PaymentFailed.
- Use Spring Boot to build each service.
- Use a message broker like Kafka or RabbitMQ to send and receive events.
- Use @EventListener or message listener methods to react to events.
- Write simple compensation steps in case something fails.

This method works well when the process is small and easy to follow.

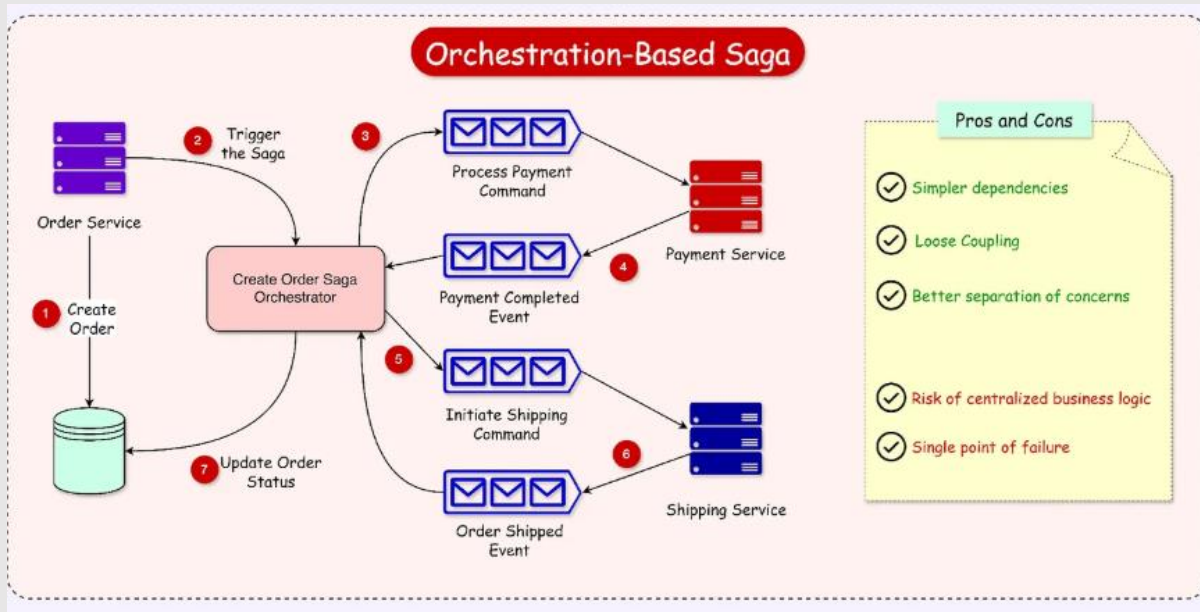
2. Orchestration Saga

In this style, one service acts like a controller. It tells each service what to do next.

Using the same food delivery example:

1. The Saga Orchestrator calls Order service to create the order.
2. Then it calls Payment service to complete the payment.
3. Then it calls Delivery service to arrange the delivery.

If payment fails, the Orchestrator calls Order service to cancel the order.



1. Create Order

The user places an order. The Order Service creates the order and saves it in the database.

2. Trigger the Saga

After creating the order, the Order Service triggers the **Order Saga Orchestrator**. From this point, the orchestrator controls the entire flow.

3. Process Payment Command

The orchestrator sends a **Process Payment** command to the Payment Service. This is a direct instruction, not just an event.

4. Payment Completed Event

The Payment Service processes the payment and sends back a **Payment Completed** event to the orchestrator.

5. Initiate Shipping Command

Once payment is confirmed, the orchestrator sends an **Initiate Shipping** command to the Shipping Service.

6. Order Shipped Event

The Shipping Service arranges delivery and sends an **Order Shipped** event back to the orchestrator.

7. Update Order Status

After receiving the shipping confirmation, the orchestrator updates the order status in the Order Service as shipped or completed.

Summary: In orchestration-based saga, a central orchestrator tells each service what to do and in what order, and updates the order status based on their responses.

Reference: [Implementing the Saga Pattern with Spring Boot and ActiveMQ in Microservice - DEV Community](#)

Steps in Spring:

- Create a separate Orchestrator service in Spring Boot.
- Call other services using REST or messaging.
- Keep track of the saga state in the Orchestrator.
- Define simple methods in each service to undo work if something fails.

This method gives more control and works better for bigger processes.

When to use each.

- Choreography fits well when the process is simple and the services are not tightly linked.
- Orchestration works better when the process has many steps and we need more control.

In real projects, we often use Spring Boot with Kafka or RabbitMQ to send messages between services.

Saga pattern helps us make sure that all steps in a big process are either fully completed or safely rolled back. This keeps the system stable and clean.

Sample for Orchestration: [Spring-Samples/SagaOrchestration at main · CodingLyf-Fullstack/Spring-Samples](#)

If we send orderId as “fail” in Request param, it will revoke back the order.

28. Microservice 27: What are advantages and disadvantages of choreography pattern?**Advantages of Choreography Pattern**

1. **Loose communication between services**
Each service talks through events, not direct calls. This removes tight connections and makes the system easier to grow and change.
2. **Good for scalability**
Since services do not wait for each other, they can handle their work in parallel. This allows the system to handle more load without slowing down.
3. **Faster development**
Teams can build and deploy services independently. As long as they follow the same event format, they can work without waiting for other teams.

Disadvantages of Choreography Pattern

1. **Difficult to track flow**
There is no central controller. When many services listen and react to events, it becomes hard to see where a business process starts and ends.
2. **Complex debugging**
When something fails in a workflow, finding the root cause is difficult because the steps are spread across many services and event messages.

29. Microservice 28: What are advantages and disadvantages of orchestration pattern?**Advantages of Orchestration Pattern**

1. Clear control and visibility

One central service control the workflow and calls other services in order. This makes it easy to understand the business flow and track what is happening step by step.

2. Simpler debugging and error handling

Since one service controls everything, it becomes easier to handle failures, retries, and logging in one place.

Disadvantages of Orchestration Pattern**1. Tighter coupling between services**

Services depend on the orchestrator to tell them what to do. If the orchestrator changes, other services may also need changes.

2. Single point of failure

If the orchestrator goes down, the whole workflow stops. We need to add high availability to reduce this risk.

30. Microservice 29: Questions on Saga pattern**1. How does the Saga pattern ensure data consistency across multiple services?**

Saga ensures consistency by breaking a long transaction into smaller steps, where each step updates its own service data. If any step fails, a compensating action is triggered to undo the previous step. This keeps data consistent without locking databases across services.

2. What happens if one step in a Saga fails?

If one step fails, the Saga pattern runs compensating transactions for all previously successful steps. For example, if payment succeeds but shipment fails, the system automatically triggers a refund. This rollback approach ensures all services return to a consistent state.

3. How do we choose between choreography-based and orchestration-based Saga?

We use choreography when services can react to events on their own, and there's no central coordinator. We use orchestration when one service (the orchestrator) controls the sequence of steps. For small, simple flows, choreography works well; for complex flows, orchestration gives better control.

4. Can Saga handle long-running transactions?

Yes, Saga is well-suited for long-running business processes like booking a hotel, flight, and car together. Each service completes its part independently. Even if one part is delayed, the overall process continues smoothly without blocking other services, making Saga ideal for such scenarios.

5. How does Saga differ from a distributed transaction?

A distributed transaction locks multiple databases until all services finish, which affects performance. Saga avoids locking by using local transactions and compensating actions. It focuses on eventual consistency rather than instant consistency, allowing services to stay available and scalable during transactions.

6. What kind of systems are not suitable for Saga?

Systems that need strict, immediate consistency are not ideal for Saga. For example, banking systems that require instant balance updates across multiple accounts may prefer a two-phase commit. Saga works best when slight delays in consistency are acceptable for better scalability.

31. Microservice 30: What is Two-Phase commit pattern?

Two-Phase Commit (2PC) is a distributed transaction protocol used to ensure that a transaction either commits (i.e., completes successfully) or rolls back (i.e., is undone) in a consistent manner across multiple systems or databases. It is commonly used in distributed systems to ensure **atomicity** and **consistency** in situations where multiple databases or services are involved in a transaction.

Phases of 2PC

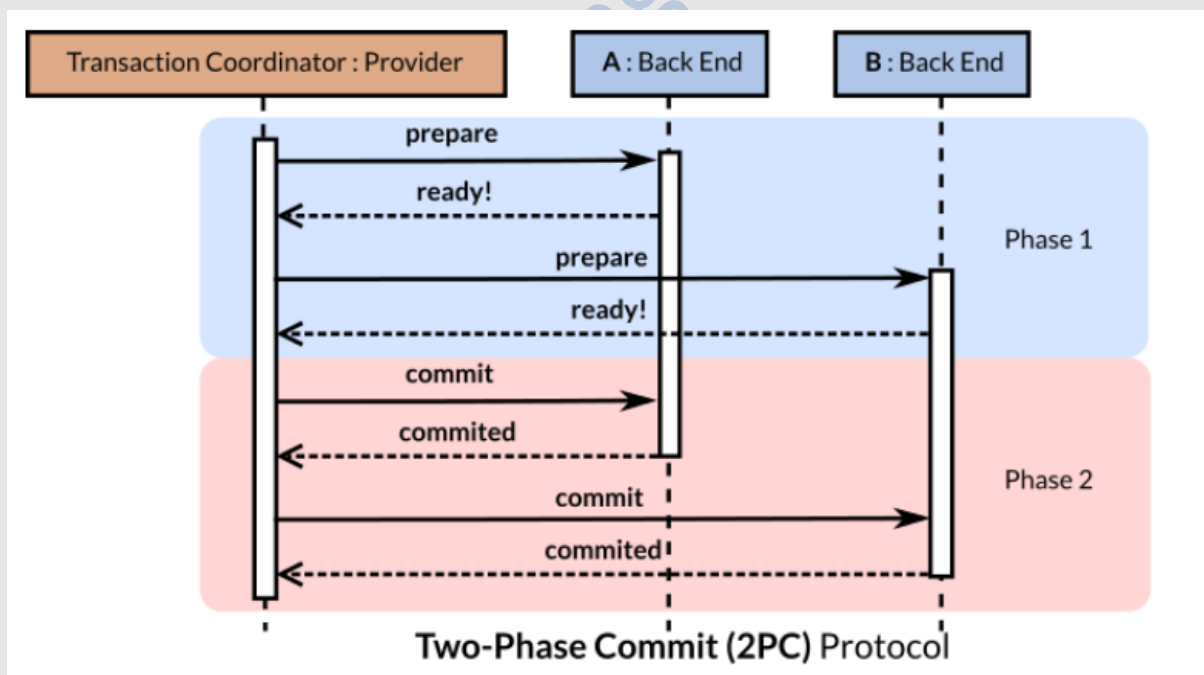
Phase 1: Prepare (Voting Phase)

The coordinator (the system initiating the transaction) sends a prepare message to all participants (the systems involved in the transaction).

Each participant checks if it can commit the transaction. If the participant is ready to commit, it responds with a YES vote. If there is any issue (e.g., resource constraints, validation failure), it responds with a NO vote.

Phase 2: Commit or Rollback (Decision Phase)

- If all participants vote YES, the coordinator sends a commit message to all participants, indicating that they should commit the transaction and finalize it.
- If any participant votes NO, the coordinator sends a rollback message to all participants, instructing them to undo any changes and revert to the previous state.



Phase 1: Prepare phase

1. A Transaction Coordinator starts the transaction.
2. It sends a prepare request to Service A.
3. Service A checks if it can commit and replies ready.

4. The coordinator sends a prepare request to Service B.
5. Service B also checks and replies ready.

At this point, no data is saved permanently. All services are only saying yes, I am ready.

Phase 2: Commit phase

6. Since all services replied ready, the coordinator sends commit to Service A.
7. Service A commits the data and replies committed.
8. The coordinator sends commit to Service B.
9. Service B commits the data and replies committed.

Now the transaction is complete.

Important points to remember

- If any service says no in Phase 1, the coordinator sends rollback to all.
- Services lock resources during the prepare phase.
- This guarantees strong consistency, but makes the system slow.
- If the coordinator fails, services can stay locked and wait.

Real-Time Use Case

Scenario: E-commerce Order Placement

In a real-world scenario like an e-commerce platform, when a user places an order, multiple systems or services need to work together:

- **Inventory System:** Decreases the stock of the ordered items.
- **Payment System:** Charges the customer's payment method.
- **Shipping System:** Creates a shipment for the order.

All these systems need to be involved in the transaction, and if one of them fails (e.g., payment fails), the entire transaction must be rolled back to maintain consistency.

Steps Using Two-Phase Commit (2PC):

Prepare Phase:

- The order service (the coordinator) sends a "prepare" message to the inventory, payment, and shipping systems.
- Each of these systems checks if they can commit:
 - **Inventory** checks if there is enough stock.
 - **Payment** checks if the transaction can be processed.
 - **Shipping** checks if the address and delivery options are valid.

Decision Phase:

- If all systems respond with a YES, the order service sends a commit message to all systems to finalize the order (inventory is reduced, payment is processed, shipping is created).
- If any system responds with a NO (e.g., if payment fails), the order service sends a rollback message to all systems to undo their operations (stock is returned, payment is not charged, no shipment is created).

Reference: [Understanding Two-Phase and Three-Phase Commit Protocols: Key Differences, Use Cases, and Practical Examples | by Damini Bansal | Medium](#)

32. Microservice 31: What is difference between Saga and Two-Phase commit pattern?

Both Two-Phase Commit (2PC) and Saga are used to manage transactions in systems where multiple services or databases are involved. The goal is to keep data consistent. However, they work in very different ways.

1. How They Work**2PC (Two-Phase Commit):**

- Works like a strict referee.
- All services must agree to commit before any changes are saved.
- If one service fails, everything is rolled back immediately.
- It is **synchronous**, services wait for each other.

Saga:

- Works like a step-by-step story.
- Each service completes its own work and then informs the next service.
- If a service fails, previous steps are undone using **compensation actions**.
- It is usually **asynchronous**; services don't block each other.

Example

Imagine an online shopping system with **Order Service** and **Payment Service**.

2PC:

1. Coordinator asks both services if they can commit.
2. Payment Service and Order Service say yes.
3. Coordinator commits both.
4. If Payment Service cannot commit, Order Service also rolls back.

Saga:

1. Order Service creates an order.
2. Payment Service processes the payment.
3. If payment fails, Order Service cancels the order using a compensation action.
4. Services do not wait for a central coordinator all the time.

33. **Microservice 32: When to use Saga and Two-Phase Commit?**

Choosing between Saga and Two-Phase Commit (2PC) depends on the size of the system, how services interact, and how fast we want the system to work. Both solve the problem of keeping data consistent across multiple services, but in different ways.

When to Use Two-Phase Commit (2PC)

2PC is best when strong consistency is very important and the system is small or medium-sized. It works well when the number of services involved in a transaction is limited.

Tiny example:

In a banking system, when transferring money from one account to another:

- Account A must decrease the balance.
 - Account B must increase the balance.
- Both steps must happen together. If one fails, the other must not happen.
2PC ensures that either both succeed or both fail, keeping money safe.

Key point: Use 2PC when we cannot allow partial changes and can tolerate a bit slower performance.

When to Use Saga

Saga is better when the system is large, distributed, and built with many microservices. It allows services to work independently, improving speed and scalability.

Example:

In an online shopping app:

- Order Service creates the order.
 - Payment Service processes the payment.
 - Delivery Service schedules delivery.
- If payment fails, Saga triggers a compensation step to cancel the order.

Key point: Use Saga when we need the system to be fast, resilient, and scalable, even if the steps are handled asynchronously.

34. **Microservice 33: What is CQRS Pattern?**

Command Query Responsibility Segregation (CQRS) is a design pattern that separates the operations that read data (queries) from those that write or update data (commands)

Command Model:

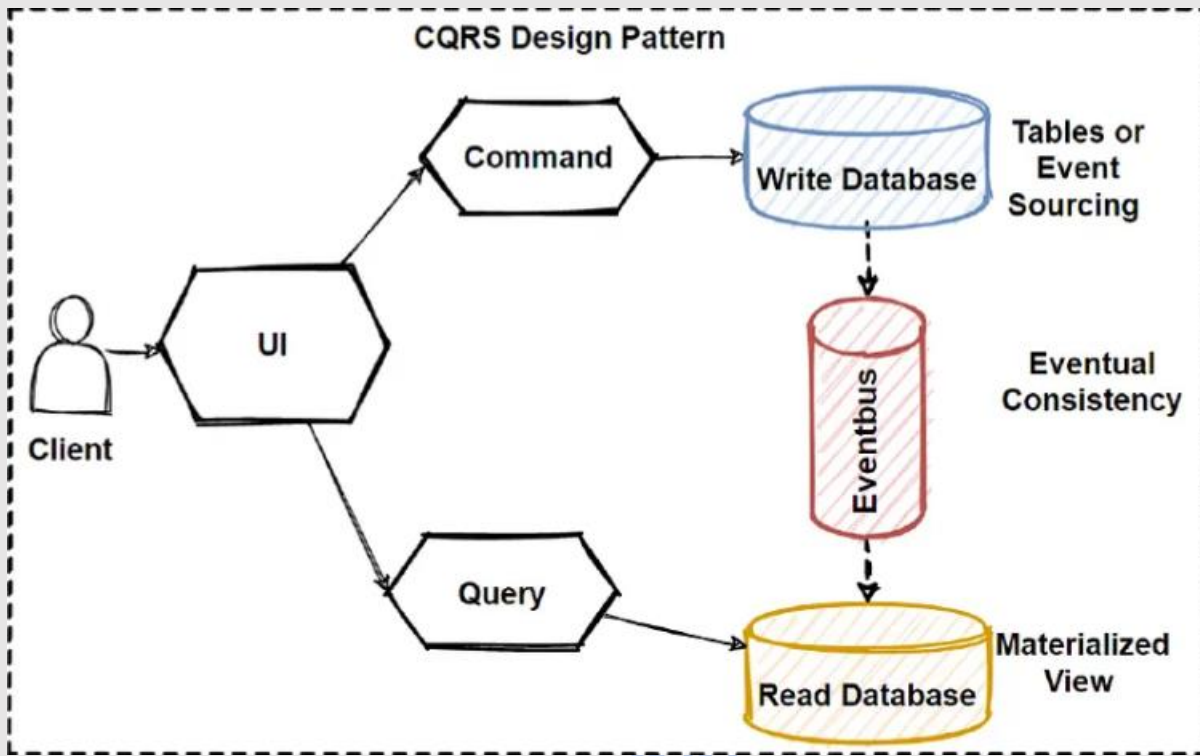
- Handles operations that write or update data (e.g., Create, Update, Delete).
- Uses normalized, transactional, relational databases like PostgreSQL or MySQL to ensure consistency.

Query Model:

- Handles operations that read data (e.g., Fetch, Retrieve).

- Uses denormalized, read-optimized stores like Elasticsearch, MongoDB, or even in-memory caches (e.g., Redis).

Typically, the Command Model and Query Model use different databases optimized for their respective tasks:



Let us understand the above diagram:

We start with the client. The client talks to the system through the UI. From the UI, two different paths are possible. One path is for changing data. The other path is for reading data. CQRS separates these two paths clearly.

When we want to change something in the system, we send a command. A command represents an intention to change the state of the application. For example, placing an order, updating a profile, or cancelling a booking are all commands. These commands go to the write side of the system.

On the write side, the command is processed with full business rules and validations. The data is then saved in the write database. This database is designed to keep data correct and consistent.

Once the data is saved, an event is published. This event is sent to an event bus. The event simply says that something has changed in the system. For example, order created or payment completed. The write side does not care who listens to this event.

The read side listens to these events. When an event arrives, the read side updates its own database. This database is built only for reading. The data is already prepared in a simple format so screens can load fast. This prepared data is often called a materialized view.

Now let us look at the reading flow. When we want to view data, such as order history or dashboard details, the UI sends a query. Queries never change data. They only ask for information. The query directly reads from the read database and returns the result quickly.

There is an important point here. The write database and read database are not updated at the same exact time. There can be a small delay between a command and when the data appears in the read side. This is called **eventual consistency**. After a short time, both sides show the same information.

To make this more real, let us take a simple example. In a food delivery app, placing an order is a command. It goes to the write database and creates the order. An order created event is published. The read side listens and updates the order history view. When we open the order history screen, the query reads from the read database and shows the order.

Example: Food Delivery Application

Command Model

Use case: A customer places a food order.

Steps:

- The customer taps "**Order Now**" after selecting food items from a restaurant.
- The application checks if the restaurant is open, items are available, and delivery partners are active in that area.
- Once validated, an **Order** is created with items, price, taxes and delivery location.
- The order is stored in the **Command Database** (often a relational DB with normalized tables like *Orders*, *OrderItems*, *Restaurants*).
- An **OrderCreatedEvent** is published to notify other services like payment, restaurant display, and delivery assignment.
- **Important point:**

A separate service (called the Query Updater or Projection Service) listens to **OrderCreatedEvent**.

It transforms this data into a read-friendly format. The transformed data is stored in MongoDB. Instead of complex normalized tables, MongoDB stores a single document:

Query Model

Use case: The customer wants to track the order status.

Steps:

- The customer opens the "**Track Order**" screen in the app.
- The system fetches details such as delivery partner name, restaurant, live order status (Preparing, Picked Up, Out for Delivery), ETA and map location.
- These details are pulled from the **Query Database** (often a document-based or search engine store like MongoDB or Elasticsearch), which holds **denormalized** order views optimized for fast reads.
- The response is returned within milliseconds since everything needed for tracking is stored in one read-friendly document.

Technologies:

- **Command Side:**
 - Relational DB: PostgreSQL or MySQL for ACID compliance.
 - Event Bus: Apache Kafka, RabbitMQ, or AWS SNS.
- **Query Side:**

- NoSQL DB: MongoDB for flexibility.
- Search Engine: Elasticsearch for full-text search capabilities.

Disadvantages of CQRS

1. Increased complexity

CQRS introduces separate write and read paths, multiple databases and event handling. This makes the system harder to design, understand and maintain, especially for small applications.

2. Eventual consistency issues

The read data is not updated immediately after a write. Users may see old data for a short time, which can be confusing if the system expects instant updates.

3. More operational effort

We need to manage events, message brokers and multiple data stores. This increases deployment, monitoring and troubleshooting effort compared to a simple CRUD system.

Reference: [CQRS Design Pattern in Microservices Architectures | by Mehmet Ozkaya | Design Microservices Architecture with Patterns & Principles | Medium](#)

35. **Microservice 34:** When to use CQRS Pattern?

CQRS stands for Command Query Responsibility Segregation. It is a design pattern where we separate the part of the system that changes data (commands) from the part that reads data (queries). This separation helps us scale, organize, and optimize the system for both reading and writing.

CQRS is useful in situations where:

Read and write workloads are different

Some applications have many reads but fewer writes, or vice versa.

Separating read and write operations helps optimize performance.

We want clear separation of responsibilities

Commands only focus on updating data.

Queries only focus on fetching data.

This makes the system easier to maintain and extend.

Example

Imagine a **social media app**:

- When a user posts a photo, the **command side** saves it in the database.
- When users browse the feed, the **query side** fetches posts quickly using a separate optimized structure.

Here, writes are handled carefully to ensure data integrity, while reads are fast and can be scaled separately.

36. **Microservice 35:** Questions on CQRS Pattern

1. Why do we separate read and write operations in CQRS?

In CQRS, we separate read and write operations because they have different needs. Writes focus on updating data safely, while reads focus on fetching it quickly. By handling them separately, we can design each side for its specific goal, improving performance and flexibility.

2. What are the main benefits of CQRS in a microservice system?

CQRS improves scalability, speed, and maintainability. It allows us to scale read and write sides independently. For example, if most traffic is for reading data, we can scale only the query side. It also simplifies complex business logic on the write side, making systems cleaner and faster.

3. How does CQRS improve performance in large-scale systems?

In large systems, most requests are read operations. CQRS improves performance by allowing the read side to use faster, optimized databases like Elasticsearch or Redis. Meanwhile, the write side can focus on accuracy and business rules. This separation reduces load and speeds up the overall system.

4. How does CQRS handle real-time updates between write and read models?

When a change happens in the write model, an event is published to notify the read model. Tools like Kafka or RabbitMQ carry these updates. The read model listens, updates its data, and stays current. This way, the system remains almost real-time and responsive.

7. What types of applications benefit most from CQRS?

CQRS fits best for systems with heavy reads, complex business rules, or large-scale data. Examples include e-commerce, banking dashboards, or social media feeds. These applications often have more reads than writes, so separating them improves speed, scalability, and overall performance.

8. How do we keep write data in sync with the read side?

We keep them in sync using event-driven updates. Every time a write happens, an event is created and sent to update the read model. Message brokers or event queues ensure delivery, so the read model eventually matches the latest data.

9. What's the difference between CQRS and a simple CRUD system?

In CRUD, the same model handles create, read, update, and delete operations. In CQRS, we separate reads and writes into different models. CRUD is simpler for small apps, while CQRS suits larger systems where performance and scalability are more important.

10. How can we test and monitor systems built using CQRS?

We test each side separately. The write side is tested for correctness and validation, while the read side is tested for accuracy and performance.

Reference Video: [CQRS Design Pattern in Microservices Interview Questions and Answers | Code Decode](#)

37. **Microservice 36:** What is Event-Driven database pattern?

The **Event-Driven Database Pattern** is a way to keep different microservices in sync using **events** instead of direct database calls. Instead of one service updating another service's database directly, a service publishes an event whenever its data changes. Other services listen to these events and update their own databases accordingly.

This helps each service stay independent, reduces tight coupling, and keeps data consistent across the system.

This approach is commonly used in **event-driven architectures** to decouple services, improve scalability, and achieve **eventual consistency**.

Example: Food Delivery System

Consider two services:

- **Restaurant Service:** manages restaurant menus and availability
- **Delivery Service:** assigns delivery partners to active orders

When a restaurant temporarily stops taking orders (maybe due to overload or closing time):

1. Restaurant Service updates its status in its own database.
2. It publishes a **RestaurantUnavailable** event.
3. Delivery Service listens to this event and removes the restaurant from its list of assignable options. It may cancel pending delivery assignments or stop assigning new ones to that restaurant.

Notice that the Delivery Service never queries the Restaurant Service database. It reacts only to events and updates its own data.

38. **Microservice 37: What is Event Driven Architecture?**

Event-Driven Architecture (EDA) is a way to design a system where services communicate by sending and reacting to events. Instead of calling each other directly, services produce events when something happens, and other services listen for those events and act accordingly.

This approach makes the system flexible, scalable, and easier to maintain because services are loosely connected.

Imagine a riding app like a taxi service:

- **Booking Service:** When a customer requests a ride, it creates a RideRequested event.
- **Driver Service:** Listens to RideRequested events and finds a nearby driver.
- **Notification Service:** Listens to the same event to send a message to the customer: "Driver is on the way."
- **Billing Service:** Listens to RideRequested and calculates the estimated fare.

Each service reacts to the event independently. The Booking Service doesn't need to know the details of drivers, notifications, or billing.

We use **Kafka**, **RabbitMQ** or **Amazon SQS** for publishing and consuming the events.

39. Microservice 38: What is Eventual Consistency?

Consistency means all databases in a distributed system see the same data at the same time. If one service changes a value, every other service should show that same new value. No one should see old or different data.

For example, if we have ₹1000 in a bank account and we spend ₹200, every place that checks the balance should now show ₹800. If one server shows ₹800 and another shows ₹1000, the data is not consistent. So, consistency makes sure the data is always correct and the same everywhere.

Data consistency is crucial for ensuring that all users and systems have access to the same, up-to-date information. It helps prevent errors, confusion, and conflicts that can arise from inconsistent data.

Strong Consistency means every service in the system always sees the latest data. When one service updates some data, that update is saved first and only after that other services are allowed to read it. So, if the Order service changes a bank balance from ₹1000 to ₹800, the Payment service, Wallet service, and any other service will all see ₹800. No service will ever see the old ₹1000. There is no delay or mismatch between services. Everyone reads the same latest value.

Eventual Consistency is a way to handle data in distributed systems where all copies of the data may not be updated at the same time, but they will become consistent eventually. Instead of forcing every service to update immediately, changes are allowed to propagate slowly. This helps systems scale and remain fast, even when many services or databases are involved.

Real-World Example

Imagine a social media app:

- A user updates their profile picture.
- The change is saved in the main database and an event is sent to other services, like the feed service and search service.
- These services might not update instantly. A few seconds later, the new picture appears in feeds and search results.

At first, some parts of the system may show the old picture, but eventually, all parts will show the same updated picture.

Eventual Consistency example

We send money from Account A to B

Transfer completes

But balance in mobile app is not updated instantly

After few seconds, app refreshes and shows correct balance

That is **eventual consistency** (system becomes correct after delay)

Key Points

- Eventual consistency is not immediate, but it guarantees that all systems will match over time.
- It is common in large distributed systems where strong consistency (immediate updates everywhere) is too slow or expensive.

- Systems that use event-driven architecture, microservices, or cloud databases often rely on eventual consistency.

Reference: [Strong Consistency vs Eventual Consistency | by Abhirup Acharya | Medium](#)

40. **Microservice 39:** What is the recommended approach when two microservices using the same database?

When two microservices need to update the same database, the safest approach is to **avoid direct shared writes**. Sharing a database sounds simple, but it causes problems like race conditions, data corruption, tight coupling, and difficulties in scaling. Instead, we follow a pattern that keeps each service independent and the data consistent.

1. Each microservice owns its own data

The cleanest design is when each service updates only the tables it owns.

If another service needs to change something, it should **call the owning service**, not touch the database directly.

Example:

- Order Service should call Inventory Service to reduce stock.
- Inventory Service should not write directly into Order tables.

This keeps the boundaries clear and avoids conflicts.

2. Use service-to-service communication

If two microservices need to change related data, one service talks to the other through an API or message queue.

Example:

When a payment is successful:

- Payment Service publishes an event: *"PaymentCompleted"*.
- Order Service listens to that event and updates its own tables.

Both services update only their own data, but the system stays in sync.

3. Use Events (best for modern systems)

Instead of sharing a database, services share **events**.

- Service A finishes work and publishes an event.
- Service B listens to the event and updates its own data.

This avoids direct database sharing and keeps everything loosely connected.

4. Implement optimistic locking:

Optimistic locking is a technique in which a version number is attached to a record in the database. When a service updates the record, it increments the version number. If another service

attempts to update the same record concurrently, it checks the version number and rejects the update if the version number has changed. This technique can prevent conflicts and ensure consistency.

In Spring, we achieve this using JPA's `@Version` annotation. We add a version field in the entity class, usually a number. Every time the record is updated, Spring increments this version automatically. When two updates happen at the same time, Spring compares the version in the database with the version in the request. If they don't match, it means another update already happened, so Spring throws an **OptimisticLockException**.

3.Communication Patterns

41. **Microservice 40:** What is communication pattern and its types?

Communication pattern is the way microservices **talk to each other** to share information or perform a task. Since each service is independent, they need clear ways to communicate. Choosing the right pattern helps the system be **fast, reliable, and easy to maintain**.

Types of Communication Patterns

1. Synchronous Communication

- Services talk directly and wait for a response.
- One service call another, waits for it to finish, and then continues.
- This is simple but can slow down the system if a service is busy or down.

Example:

In a hotel booking system, the Booking Service calls the Room Service to check availability. It waits for Room Service to respond before confirming the booking.

2. Asynchronous Communication

- Services send messages and do not wait for an immediate response.
- The receiving service processes the message when it can.
- This makes the system more resilient and scalable.

Example: In a banking system, when a customer transfers money:

- The Transfer Service publishes a `MoneyTransferRequested` event.
- Notification Service listens to this event and sends an SMS.
- Transfer Service does not wait for the SMS to be sent.

42. **Microservice 41:** How do you achieve Synchronous Communication in Microservices?

Synchronous communication happens when one service calls another service directly and waits for a response before continuing. It is simple to understand and works well when services need immediate answers.

How can we achieve It

HTTP/REST Calls

- Services use REST APIs over HTTP to talk to each other.
- One service sends a request, waits for the response, and continues.

GraphQL

- GraphQL allows a service to provide a single endpoint where other services or clients can ask for only the data they need.
- When a service makes a GraphQL request, it waits for the response before moving forward, just like a normal synchronous call.
- This is especially useful when we need to **get information from multiple services** at once, because GraphQL can combine the data in a single request instead of making many separate REST calls.

(Refer Questions – Microservices 42-44)

gRPC (Remote Procedure Call)

- A faster alternative to REST, especially for internal service-to-service communication.
- Services call methods on other services like normal functions.

SOAP or other RPC protocols

- Used in legacy systems for strict contracts between services.

Follow-up Question:

What is REST?

REST is not a protocol or standard but rather a set of guiding principles that leverage the existing HTTP protocol to enable communication between clients and servers.

At its core, REST is built around resources. Each resource (such as a user, order, or product) is uniquely identified by a URL (Uniform Resource Locator), and clients interact with these resources using a fixed set of HTTP methods.

- **GET** → Retrieve a resource (e.g., GET /api/users/123 to fetch user data).
- **POST** → Create a new resource (e.g., POST /api/users to add a new user).
- **PUT/PATCH** → Update an existing resource (e.g., PUT /api/users/123 to update user details).
- **DELETE** → Remove a resource (e.g., DELETE /api/users/123 to delete a user).

For example, let's say a client needs information about a specific user with **ID 123**.

- The client makes a request
- The server responds with a JSON representation of the user

REST APIs typically **return data in JSON** and use **HTTP status codes** to communicate the outcome of the request:

- **200 OK** → Success

- **201 Created** → Resource successfully created
- **400 Bad Request** → Client error (e.g., missing required fields)
- **404 Not Found** → Requested resource does not exist
- **500 Internal Server Error** → Unexpected server issue

Benefits of REST

- **Simplicity and Intuitive Design:** The resource-based model aligns well with most business domains, making REST intuitive for developers.
- **Statelessness:** Each request contains all the information needed to complete it, making REST scalable across distributed systems.
- **Cacheability:** HTTP's caching mechanisms can be leveraged to improve performance.
- **Scalability:** REST APIs can be easily scaled using load balancers and CDNs.
- **Mature Ecosystem:** With nearly two decades of widespread use, REST enjoys robust tooling, documentation, and developer familiarity.

Drawbacks of REST

- **Over-fetching:** REST endpoints often return **more data than needed**, leading to inefficient network usage. For example, if a mobile app only needs a user's name and email, but the API response includes additional fields like address, phone number, and metadata, it results in **wasted bandwidth**.
- **Under-fetching:** If an API doesn't return related data, the client may need to **make multiple requests** to retrieve all required information. For example, to get user details and their posts, a client might have to make:
 1. GET /api/users/123 (fetch user)
 2. GET /api/users/123/posts (fetch user's posts)
- **Versioning issues:** When APIs evolve, maintaining backward compatibility becomes difficult. REST APIs often require **versioned URLs** (/v1/users, /v2/users), adding maintenance overhead.

For years, **REST** was the standard for building APIs. However, as applications grew more complex, REST began to show limitations—especially in scenarios where clients needed fine-grained control over the data they fetched.

To address these challenges, **Facebook introduced GraphQL**, offering a more flexible and efficient approach to data retrieval.

43. **Microservice 42: What do you know about GraphQL?**

GraphQL is a query language for APIs, developed by Facebook, and released to the public in 2015. It helps developers interact with a server in a way that's more flexible and efficient compared to traditional REST APIs.

A **single GraphQL endpoint** (/graphql) replaces multiple REST endpoints, allowing clients to structure their own queries instead of relying on predefined responses.

It was introduced because REST APIs have some limitations:

A REST API is an architectural concept for application communication. On the other hand, GraphQL is a specification, an API query language, and a set of tools. GraphQL operates over a single endpoint using HTTP.

In addition, REST development has been more focused on making new APIs. Meanwhile, GraphQL's focus has been on API performance and flexibility.

Over-fetching: Sometimes, a REST API returns more data than you need.

Example: You fetch a list of users, but each user object includes unnecessary details like their address, age, etc.

Under-fetching: REST endpoints often don't provide all the data you need in one call.

Example: You need both a user's profile and their posts, so you must make two separate requests.

Rigid structure: REST endpoints are predefined and can be inflexible. If you need different data, the API often needs to be updated.

GraphQL solves these issues by allowing clients to specify exactly what data they need and get everything in a single request.

How GraphQL fixes this

- The client asks only for required fields. Nothing extra.
- One request can bring related data.
- The backend doesn't need to create multiple endpoints for different data needs.

Example query: One request, both user name and posts.

```
query {  
  user {  
    name  
    posts {  
      title  
    }  
  }  
}
```

Use Cases for GraphQL

GraphQL shines in scenarios where flexibility, efficiency, and dynamic data fetching are required. Here are some of the most common use cases:

1. Dynamic UI Applications

- **Example:** Dashboards, content-driven websites, and admin panels.
- **Why GraphQL?** UI components can fetch only the necessary data, improving performance and reducing unnecessary API calls.

2. Mobile Applications

- **Example:** Social media apps, e-commerce platforms.
- **Why GraphQL?** Mobile devices have limited bandwidth, so fetching only required data optimizes performance.

3. Integrating Data from Multiple Sources

- **Example:** Aggregators, business intelligence tools.
- **Why GraphQL?** Can unify data from multiple REST APIs, databases, or third-party services into a single query.

4. Microservices Architecture

- **Example:** Large-scale enterprise applications.
- **Why GraphQL?** Acts as an API gateway that consolidates multiple microservices into a single schema.

Reference: [GraphQL in 2025: Pros & Cons, Public APIs, and Use Cases — Part 1](#) | by Frontend Highlights | Medium

Reference links for sample:

[Getting Started](#) | [Building a GraphQL service](#)

[Tutorial with Spring for GraphQL](#) | [GraphQL Java](#)

Code Sample: [Spring-Samples/graphqldemo/graphqldemo at main · CodingLyf-Fullstack/Spring-Samples](#)

Steps to run:

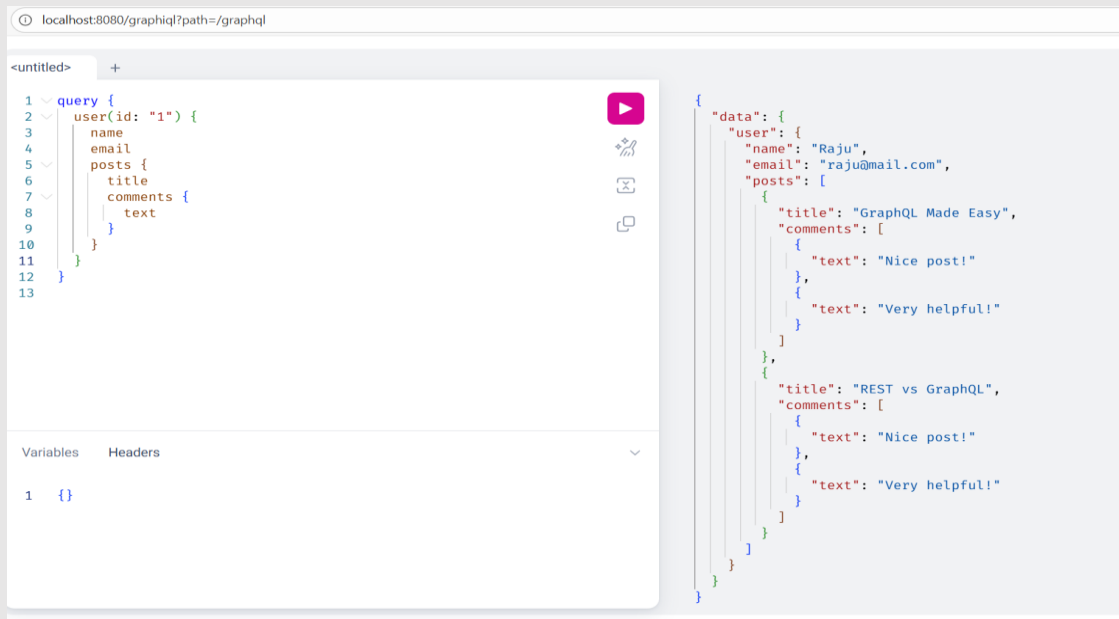
1. Run mvn clean compile
2. Run GraphqldemoApplication.java
3. Open <http://localhost:8080/graphiql?path=/graphql> in Browser
4. Copy below code and click play button
5. GraphQL returns only requested data like name, mail, title not others like ids

```
query {  
  user(id: "1") {  
    name  
    email
```

```

    posts {
      title
      comments {
        text
      }
    }
  }
}

```

Output:**Follow-up Question:****Difference between REST and GraphQL?****REST**

We create many URLs like `/users`, `/orders`, `/products` to get different data.

The backend decides what data is sent in the response.

If we need user and order data, we call two APIs.

It may send extra data that we do not need.

It is simple and easy to build for most projects.

GraphQL

We use one single URL to get all types of data.

We decide what data we want to get in the request.

We can get user and order data in one single call.

It sends only the exact data we ask for.

It is better when serving multiple clients (mobile, web, IoT) with different data needs

Reference: [GraphQL vs REST API | Similarities & Differences | by David Mosyan | Medium](#)

[REST vs GraphQL - by Ashish Pratap Singh](#)**44. Microservice 43: Questions on GraphQL?****1. What is dependency used for GraphQL in Spring?**

We add the dependency spring-boot-starter-graphql in our project to enable GraphQL support. This brings everything needed to handle GraphQL requests, queries, schema files and mapping annotations. Once added, we can expose GraphQL endpoints and write queries without creating multiple REST controllers or endpoints.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
```

2. What is schema.graphqls?

The schema.graphqls file defines the structure of our GraphQL API. It tells what queries, inputs and data types exist. The client asks data only from what is defined in this schema. It works like a contract between the client and server. The file is usually placed under src/main/resources/graphql.

Example:

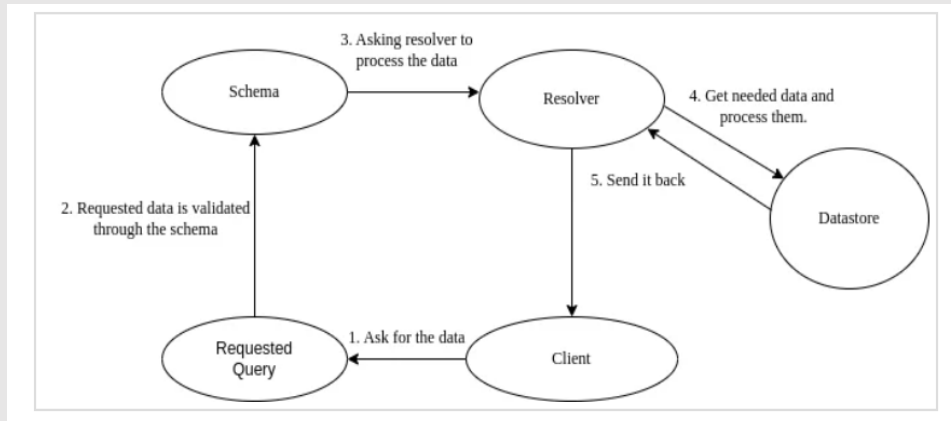
```
type Query {
  getBook(id: ID): Book
}
```

3. If a greater number of queries are there, will all be saved in same single file?

We can put all queries in one schema file, but it becomes hard to read when the application grows. Spring allows splitting schema into multiple .graphqls files. The framework automatically merges them. This helps us keep schemas based on modules like user, order and product for better organization.

4. How does GraphQL work?

- GraphQL provides a different set of types that query on that particular data by using schema and then, queries and mutations are come, and validated by using schema, and processed by using resolver.



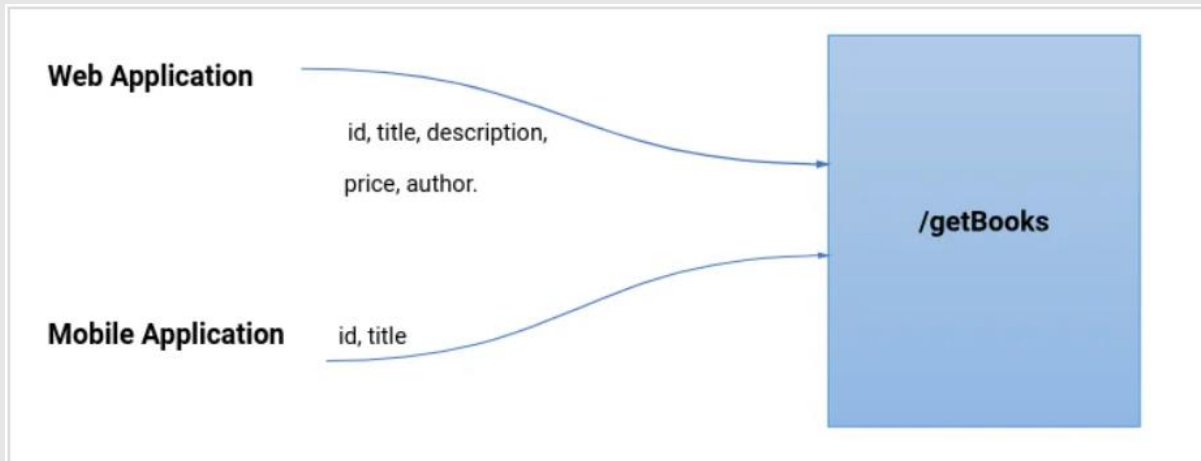
- **Schema**- Schema is how you will define your data or how you request your data and how the API will respond to that data. The schema is the concept of server and client.
- **Resolver**- Resolver is used for fetching the data, processing the data, then transferring the fetched data and processing the data by using GraphQL array. Resolver gives the result by using the callable function.

5. Why use GraphQL instead of REST?

- It gives us a single endpoint, we do not need to create the different APIs for the same data. GraphQL is strongly typed, you can also check or validate the GraphQL query before the execution.
- GraphQL helps to build powerful APIs and clients also know what they exactly want and what different types of operations they can perform.

Example-

- Suppose, we have two applications one is Web Application and the second is Mobile Application, and /getBooks is an API for fetching the data.
- Web Application can send the request for getting the book details by using the field like id, title, description, price, and author, also Mobile application wants the same data but uses only two fields id and title only.
- But we made the normal APIs, so Mobile Application wants only two fields of id and title, but by using normal APIs Mobile Application gets all the data like id, title, description, price, and author when the Mobile application hits the same endpoint /getBooks.
- Even though we use different data we made the different REST APIs for Web Application and also for Mobile Application, so this is a very costly and inefficient way. This problem is solved by GraphQL. By using GraphQL the data is passed into the form of the query by using a single endpoint.



6. Types of operations in GraphQL

1. Query

Used to **fetch data** from the server. It does not change anything. Think of it like a SELECT in SQL.

Example

```
query {  
  getBook(id: 1) {  
    title  
    author  
  }  
}
```

2. Mutation

Used to **create, update, or delete** data. It changes the state of the server. This matches INSERT, UPDATE, DELETE in SQL.

Example

```
mutation {  
  addBook(title: "Spring GraphQL") {  
    id  
    title  
  }  
}
```

3. Subscription

Used to **receive real-time updates** from the server. When data changes, the server pushes updates automatically. Works like a live feed or notification system.

Example

```
subscription {  
  bookAdded {  
    id  
    title  
  }  
}
```

45. **Microservice 44: When to use GraphQL?**

Some clients or services need only certain pieces of data. GraphQL lets them ask for exactly what they want.

Instead of calling many REST endpoints, GraphQL can fetch all needed data in a single request.

Real-World Example

Imagine a hospital management system with these services:

- Patient Service: stores patient details.
- Appointment Service: stores doctor appointments.
- Lab Service: stores test results.

A doctor's dashboard needs:

- Patient name and age
- Upcoming appointments
- Latest lab results

Without GraphQL, the dashboard might make three separate REST calls.

With GraphQL:

1. The dashboard sends a single query requesting patient info, appointments, and lab results.
2. GraphQL collects data from all three services and combines it into one response.

What this really means is GraphQL itself does not store data. It acts like a middle layer. It calls Service A, Service B, and Service C, gets data from each one, then puts everything together and sends one clean response back.

3. The response contains exactly what the dashboard needs in one call.

Use REST if:

- Your API is simple and doesn't require flexible queries.
- You need caching benefits from HTTP.
- You need a standardized, well-established API approach.
- You're integrating with third-party services.
- Your team is already familiar with REST and need faster implementation.

Use GraphQL if:

- You need flexible and efficient data fetching.
- Your API serves multiple clients (mobile, web, IoT) with different data needs.
- Real-time updates are required (GraphQL subscriptions).
- You want to avoid API versioning issues.
- Your application requires deeply nested data

Reference: [What Is GraphQL and How It Works - GraphQL Academy | Hygraph](#)

46. **Microservice 45: What is gRPC and how it is useful?**

gRPC stands for **g**oogle **R**emote **P**rocedure **C**all.

It is a communication framework developed by Google that allows services to talk to each other efficiently. Instead of sending text-based messages like REST, gRPC uses binary data with Protocol Buffers, which makes communication faster, lighter and suitable for microservices and real-time systems.

gRPC is a way for services to communicate with each other quickly and efficiently. It allows one service to call a method on another service as if it were a local function, even if the services are on different servers.

gRPC is faster than traditional REST because it uses a binary protocol instead of text, and it supports streaming data in both directions. It is especially useful for internal communication between microservices in large systems.

gRPC supports different ways for services to communicate. These patterns make it flexible for different types of tasks.

1. Unary RPC

- **What it is:** One service sends a single request and gets a single response.
- **Example:** In a smart city system, Emergency Service asks Traffic Service for the current traffic status at a specific location. Traffic Service replies with one response.

2. Server Streaming RPC

- **What it is:** One request is sent, but the server sends multiple responses over time.
- **Example:** Traffic Service sends live traffic updates to Public Display Service. The display gets updates continuously without sending new requests.

3. Client Streaming RPC

- **What it is:** The client sends multiple requests and waits for a single response from the server.
- **Example:** Sensors in a building send temperature readings every few seconds to Building Management Service. Once all readings are sent, the service calculates and responds with the average temperature.

4. Bidirectional Streaming RPC

- **What it is:** Both client and server send messages continuously.
- **Example:** In a hospital monitoring system, Patient Monitoring Service streams live vital signs to Doctor Service. Doctor Service can send alerts or instructions back in real time.

Real-World Example:

Imagine a smart city system with multiple services:

- Traffic Service: monitors traffic cameras and sensors.
- Emergency Service: responds to accidents or emergencies.
- Public Display Service: updates electronic signboards with traffic info.

Flow using gRPC:

1. Traffic Service detects a traffic jam and calls Emergency Service using gRPC to alert them.
2. Emergency Service responds immediately with the estimated response time.
3. Traffic Service streams live updates to Public Display Service to show current traffic on signboards.

Here, gRPC allows fast, real-time communication between services, even with large amounts of data like live sensor feeds.

Reference video link: [gRPC for Beginners 📺 | What, Why & How gRPC ? | Must-Know Basics! @Java Techie](#)

47. **Microservice 46: What is asynchronous communication and how it is useful?**

Asynchronous communication is a way for services to talk without waiting for a response immediately. One service sends a message or event, and the other processes it when it can. This makes systems more scalable, reliable, and efficient.

How It Works

- Service A sends a message to Service B.
- Service A continues its work without waiting for Service B to respond.
- Service B processes the message later and may trigger another event.

Real-World Example

Imagine a university notification system:

When a student finishes the admission process in the **Admission Service**, a message is sent out saying "New Student Admitted."

This message is not sent to one single service. Instead, it goes to a **message broker** (like Kafka or RabbitMQ, [Refer below questions Kafka and RabbitMQ](#)).

- The **Library Service** listens and creates a library account for the student.
- The **Hostel Service** listens and checks for room availability.
- The **ID Card Service** listens and starts preparing the ID card.

The Admission Service does not wait for any of them to finish. It just sends the message and continues admitting other students. Each service does its work when it receives the event.

This pattern makes the system more flexible. If tomorrow a new service is added, like a **Sports Club Service**, it can also listen to the same event without touching the Admission Service.

Commonly used tools for asynchronous communication in microservices:

1. Apache Kafka
2. RabbitMQ
3. Amazon SQS (Simple Queue Service)
4. Google Pub/Sub

48. **Microservice 47:** How can you implement Asynchronous communication in Microservice?

We can use a message broker to handle asynchronous messages. **Kafka** and **RabbitMQ** are two common choices.

Example using Spring with RabbitMQ.

1. Add message sender in Admission Service

```
@Service
public class AdmissionService {
    private final RabbitTemplate rabbitTemplate;

    public AdmissionService(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    public void admitStudent(String studentId) {
        // business logic
        rabbitTemplate.convertAndSend("studentExchange", "student.admitted", studentId);
    }
}
```

2. Add message listener in Library Service

```
@Service
public class LibraryListener {

    @RabbitListener(queues = "studentQueue")
    public void handleNewAdmission(String studentId) {
        // create library account
        System.out.println("Library account created for: " + studentId);
    }
}
```

3. Add simple configuration

```
@Configuration
public class RabbitConfig {

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange("studentExchange");
    }

    @Bean
    public Queue queue() {
        return new Queue("studentQueue");
    }

    @Bean
    public Binding binding(Queue queue, TopicExchange exchange) {
        return BindingBuilder.bind(queue).to(exchange).with("student.admitted");
    }
}
```

How this works:

Admission Service sends an event.

Library Service listens to the event and works on it when ready.

Both services are independent. If the Library Service is down, Admission Service still works.

When the Library Service comes back, it will receive the pending messages.

49. Microservice 48: When to use Asynchronous and when to use synchronous communication?**Synchronous Communication**

Synchronous means, one service sends a request to another and waits until it gets the response. Only after getting the response does it continue its work.

When to Use It

- When we need an instant response.
- When the request and response are part of one workflow.
- When real-time confirmation matters.

Example: In an online shopping portal, when we place an order, the Order Service needs an immediate confirmation from the Payment Service before marking the order as complete.

This is a synchronous call because the Order Service must wait for the Payment Service to confirm the payment before moving ahead.

Asynchronous Communication

Asynchronous means, a service sends a message and continues its work without waiting for the other service to respond immediately.

The other service processes the message later and sends the result when ready.

When to Use It

- When the task can happen later without blocking the main flow.
- When we want services to be independent and loosely connected.
- When performance and scalability are more important than real-time replies.

Example:

In the same shopping portal, after an order is confirmed, a Notification Service sends an email or SMS to the user.

The Order Service need not to wait for the Notification Service to finish sending messages. It just sends a message to a queue like Kafka or RabbitMQ and continues processing.

50. Microservice 49: How does Apache Kafka works and why it is fast?

Think of Kafka as a large, really fast post office that receives messages (data) from various sources and sends them to their respective destinations.

Let's imagine a system like Uber, but we'll focus only on how Kafka fits in, not the full product.

Real world example: Ride booking system

Think about how many events happen when a ride is booked.

- A rider books a cab.
- A driver gets the ride request.
- The price is calculated.
- A live map is updated.
- A bill is generated after the trip ends.
- A notification is sent to the rider.

If all these services talked directly to each other, it would get messy. If one service fails, the others might break too. This is where Kafka becomes powerful.

Here's how it works step by step.

Step 1: Producer sends message

When a rider books a cab, the Booking Service acts as a producer. It sends a message like *"Ride Requested"* to a Kafka topic called ride-events.

Step 2: Kafka stores the event

Kafka receives the event and keeps it in the ride-events topic. It doesn't care which service will use it. It just stores the message safely and in order.

Step 3: Multiple services read the same event

- Driver Matching Service reads the event and finds a nearby driver.
- Pricing Service reads the same event and calculates the fare.
- Notification Service reads it to send a push message to the rider.
- Tracking Service reads it to update the live location.
- Billing Service reads it after the ride ends to create the invoice.

All these services are consumers of the same Kafka topic.

Step 4: System runs smoothly even if one service is slow

Let's say the Billing Service is down for a few minutes. No problem. Kafka still keeps the messages. When Billing comes back, it reads the stored events and continues processing. The rest of the system keeps running.

Why Kafka works well here

- One event can trigger many actions.
- Services are not tightly connected.
- If one part fails, the rest of the system keeps working.
- The system can handle millions of ride events per day without crashing.

This is exactly why companies like Uber use Kafka at the center of their system. It lets the platform move fast, stay stable, and scale without losing messages.

Why is Apache Kafka so fast?

In the data streaming world, speed is often of paramount importance. Whether it's tracking user activities on a website, processing financial transactions, or monitoring IoT devices, organizations require a system that can handle a continuous flow of data with minimal latency.

Kafka excels in this regard, and its speed can be attributed to several key factors.

1. Distributed Architecture

At the heart of Kafka's speed lies its distributed architecture. Unlike traditional message queuing systems, Kafka doesn't rely on a single centralized server to handle all the data.

Instead, it distributes data across multiple nodes or brokers.

This **parallel processing capability allows Kafka to scale horizontally**, meaning it can handle an increasing volume of data by adding more machines to the cluster.

2. Partitioning

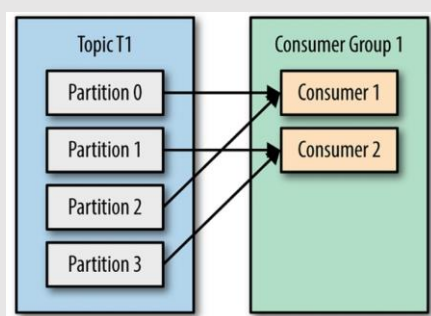
Partitioning is a core concept in Kafka that contributes significantly to its speed. **Data is divided into partitions, and each partition is assigned to a specific broker.**

This division of data allows Kafka to parallelize the processing of messages.

As messages are produced and consumed, Kafka ensures that each partition is handled by a single consumer at any given time.

The use of partitions allows Kafka to achieve both parallelism and order. Within a partition, messages are processed sequentially, ensuring that the order of events is maintained.

However, since different partitions can be processed independently by different brokers or consumers, Kafka can achieve high levels of parallelism.



Reference: [How does Apache Kafka work? Why is Kafka So fast? - DEV Community](#)

51. **Microservice 50:** Explain the Kafka architecture?

Kafka works like a smart post office that can handle thousands of messages every second. It has four main parts: Producer, Topic, Broker, and Consumer. Along with these, Zookeeper or Kafka Controller helps to keep everything in sync.

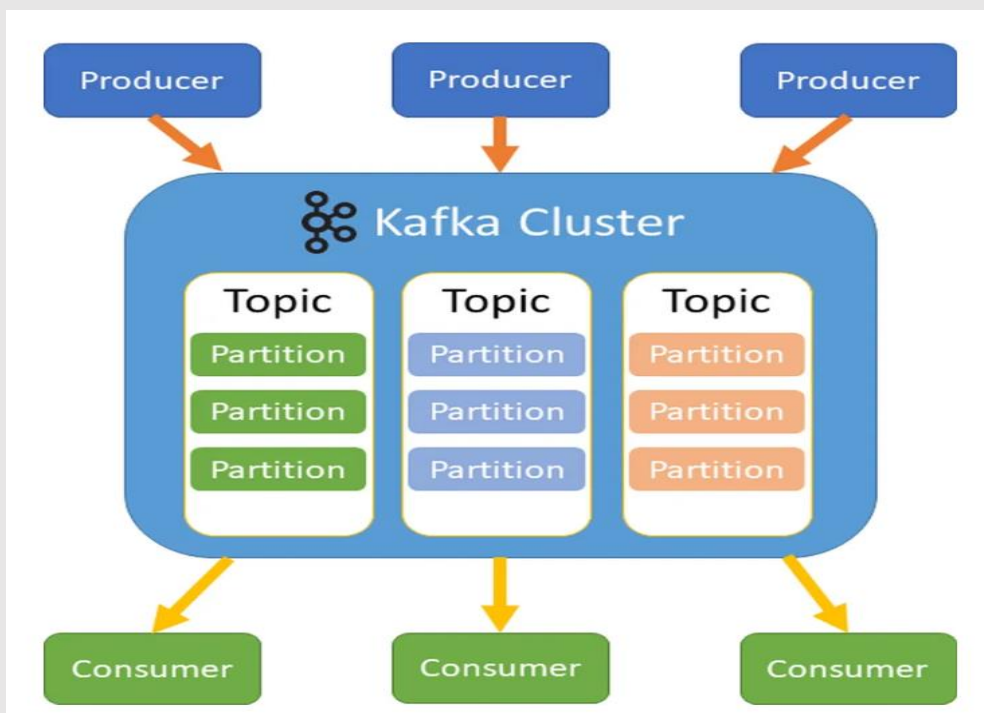
Real world example: Weather alert system

Imagine a big city with many weather sensors. Each sensor tracks temperature, humidity, air quality, and sends data every few seconds.

Many departments use this data:

- One team updates the city weather dashboard
- Another team sends alerts to mobile apps
- A third team stores the data for reports and analysis

We want to handle all this data smoothly. Kafka helps us here.



1. Producer: who sends the message

Each weather sensor acts like a **producer**. It sends messages to Kafka like “Temperature in Zone A is 33°C” or “Air Quality Alert in Zone B”.

2. Topic: mailbox to store messages

Kafka has a **topic** for each type of data. For example:

- temperature-topic for temperature updates

- air-quality-topic for pollution alerts
- humidity-topic for humidity changes

Topics help to group messages so they are easy to read later.

3. Broker: the Kafka server

Kafka runs on servers called **brokers**. These brokers keep all messages safely and distribute them across multiple machines.

If one broker goes down, another broker still has the data. This keeps the system safe and reliable.

4. Partition: splitting the load

Each topic is divided into **partitions**.

Think of partitions as multiple boxes in the same mailbox. This helps Kafka process many messages at the same time without slowing down.

For example, the temperature-topic may have 3 partitions, so 3 different sensors can send data at the same time.

5. Consumer: who reads the message

Different services act as **consumers**.

- The Dashboard Service reads temperature updates to show them live on the screen.
- The Alert Service reads air quality alerts to warn citizens.
- The Analytics Service reads everything and stores it in a data warehouse.

Consumers can read from the same topic without affecting each other.

6. Zookeeper or Controller: keeps things in order

Kafka uses **Zookeeper** or its built-in controller to manage the cluster, track brokers, and balance the load.

It's like the manager who ensures everything runs smoothly.

Final flow

1. Sensors send weather data to Kafka topics (Producer).
2. Kafka brokers receive and store the data in partitions.
3. Different services read the data as per their need (Consumer).
4. If something goes wrong, Kafka still keeps the data safe.

52. **Microservice 51:** Can you tell some real-world use cases for Kafka?

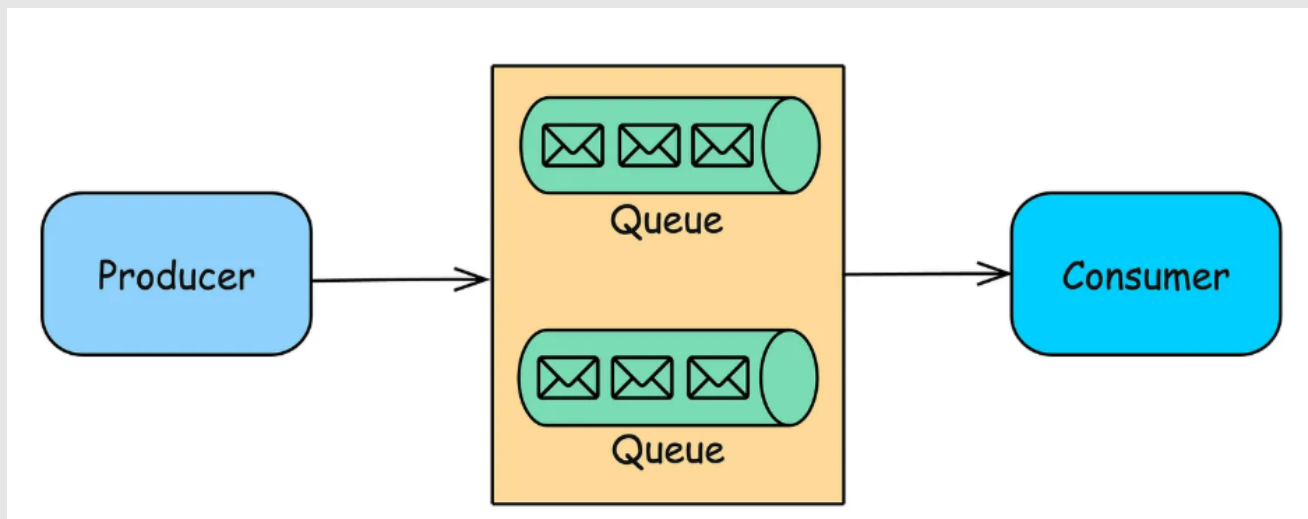
Real-World Usage of Apache Kafka

- **Netflix** uses Kafka to apply a recommendation in real-time while you're watching TV shows.
- **Uber** uses Kafka to gather taxi user and trip data in real time and they will use it to compute and forecast demand and then they can compute the infamous search pricing in real-time to know how much to charge you for a ride in case there is a lot of high demand.
- **LinkedIn** uses Kafka to prevent spam and collect user interactions to make better connection recommendations in real time.

For more questions on Kafka Refer Question No. 370

53. Microservice 52: What are message queues?

A message queue is a system that allows services to send messages to each other without waiting. Messages are stored in a queue until the receiving service is ready to process them. This makes systems more reliable and flexible, because services do not depend on each other to work at the same time.



Real world example: Call Center Support System

Imagine a call center with multiple support agents:

- Customers submit support tickets online or via phone.
- The **ticket system** receives all tickets and puts them into a queue.
- Agent 1 picks a ticket and starts resolving it.
- Agent 2 picks another ticket and works on it.
- If a new agent joins the team, they can also start picking tickets from the same queue without disrupting the system.

Each ticket waits in the queue until an agent is ready to process it. If an agent is busy or temporarily unavailable, the ticket stays safely in the queue. Nothing is lost, and all tickets are handled efficiently.

How Message Queues Work:

1. **Producer:** The service that sends the message.
 - Customers submitting tickets act as the **producer**.

2. **Queue:** The storage place where messages wait.
 - The **ticket system** is the queue. Tickets wait here until an agent is available.
3. **Consumer:** The service that reads and processes the message.
 - The **support agents** are consumers. Each agent picks a ticket from the queue and resolves it.

4. Broker/Queue Manager

The software or service that manages the message queue, handles the delivery of messages, and ensures that messages are routed correctly between producers and consumers.

4. **Acknowledgment:** The consumer tells the queue the message is processed.
 - Once an agent finishes resolving a ticket, the message is removed from the queue.

Examples

- **AWS SQS:** A managed service for message queuing.
- **RabbitMQ & Kafka:** Popular message brokers for various architectures.
- **IBM MQ:** Enterprise-grade message queuing middleware.

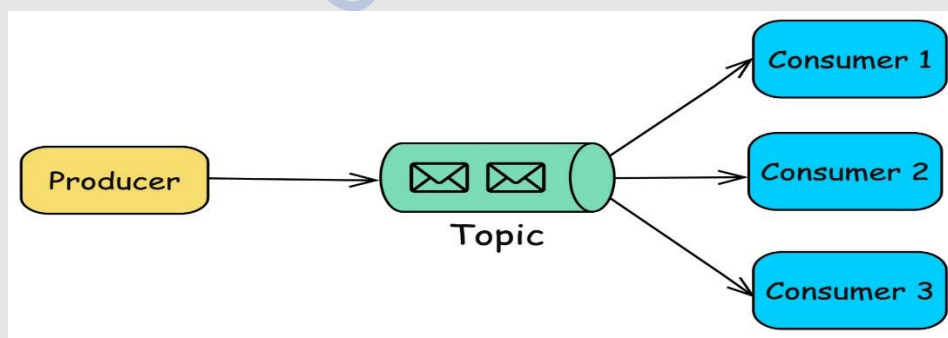
Types of Message Queues:

There are several types of message queues, each designed to solve specific problems:

1. Point-to-Point (P2P) Queue

In this model, messages are sent from one producer to one consumer. Used when a message needs to be processed by a single consumer, such as in task processing systems.

2. Publish/Subscribe (Pub/Sub) Queue



In this model, messages are published to a topic, and multiple consumers can subscribe to that topic to receive messages.

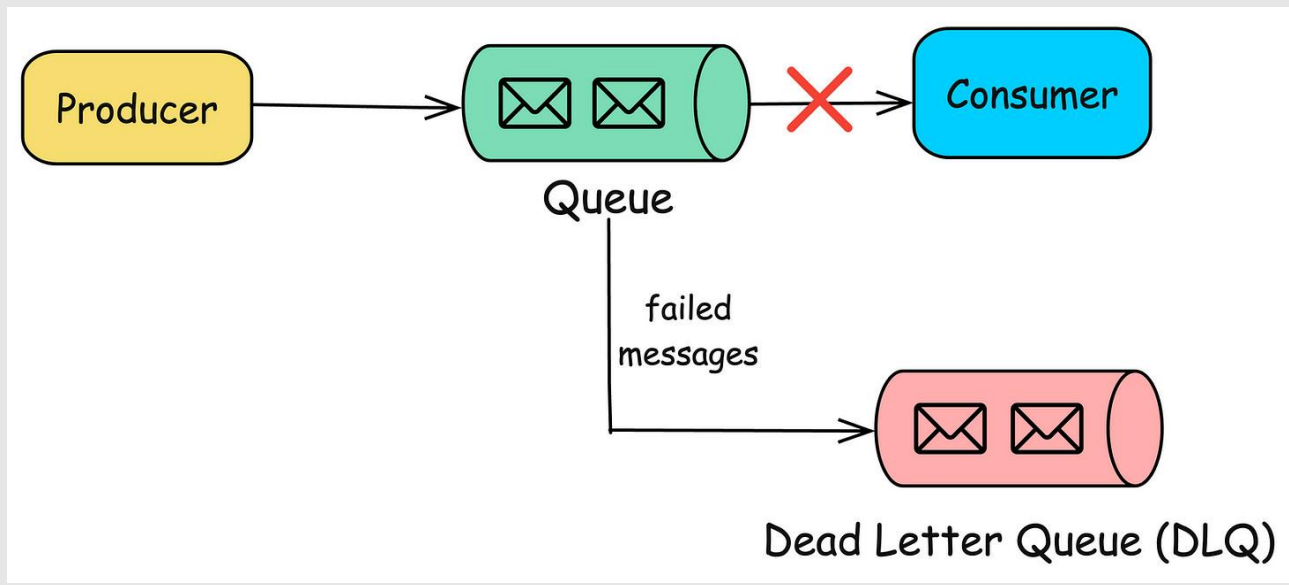
Used for broadcasting messages to multiple consumers, such as in notification systems.

3. Priority Queue

Messages in the queue are assigned priorities, and higher-priority messages are processed before lower-priority ones.

Used when certain tasks need to be handled more urgently than others.

4. Dead Letter Queue (DLQ)



A special type of queue where messages that cannot be processed (due to errors or retries) are sent. Useful for troubleshooting and handling failed messages.

54. **Microservice 53:** What is message broker?

A message broker is a software that enables applications, systems, and services to communicate with each other and exchange information.

Think of it like post office where it receives messages from one service and delivers them to other services. The broker ensures messages are delivered safely, in order, and without losing any, even if some services are busy or temporarily down.

Message brokers are very useful in microservices because they decouple services. Producers (services that send messages) don't need to know who the consumers are or when they will process the message.

Popular Message Brokers in the Market

1. **RabbitMQ:** Easy to use, reliable, works well for most queue-based tasks. *RabbitMQ is an open-source message broker that implements the Advanced Message Queuing Protocol (AMQP). It helps in decoupling services and ensures reliable message delivery.*
2. **Apache Kafka:** Handles very high volumes of messages, good for streaming and event-driven systems.
3. **ActiveMQ:** Another general-purpose messaging system, supports multiple protocols.
4. **Amazon SQS:** Cloud-based, fully managed queue service, no server setup required.
5. **Google Pub/Sub:** Cloud messaging service, scales automatically, good for event-driven systems.

Reference: [Messaging Made Easy: A List of Message Brokers and Event Streaming Platforms](#) | by Carlos Armando Marcano Vargas | Medium

55. **Microservice 54: When to use message queues?**

Message queues are useful whenever services need to communicate without waiting for each other. They help systems work smoothly even if some parts are slow or temporarily down, and make scaling easier.

1. Microservices Architecture

- **Problem:** Microservices need to communicate with each other, but direct communication can lead to tight coupling and cascading failures.
- **Solution:** Use message queues to enable asynchronous communication between microservices, allowing each service to operate independently and resiliently.

2. Task Scheduling and Background Processing

- **Problem:** Certain tasks, such as image processing or sending emails, are time-consuming and should not block the main application flow.
- **Solution:** Offload these tasks to a message queue and have background workers (consumers) process them asynchronously.

3. Event-Driven Architectures

- **Problem:** Events need to be propagated to multiple services or components, but direct communication would be inefficient.
- **Solution:** Use a Pub/Sub message queue to broadcast events to all interested consumers, ensuring that all parts of the system receive the necessary updates.

4. Load Leveling

- **Problem:** Sudden spikes in requests can overwhelm a system, leading to degraded performance or failures.
- **Solution:** Queue incoming requests using a message queue and process them at a steady rate, ensuring that the system remains stable under load.

5. Reliable Communication

- **Problem:** Communication between components needs to be reliable, even in the face of network or service failures.
- **Solution:** Use persistent message queues to ensure that messages are not lost and can be retried if delivery fails.

Real time Examples:

- **Instagram (Meta):** Used RabbitMQ in early systems to queue tasks like photo processing before scaling up to Kafka.
- **Reddit:** Uses RabbitMQ for background jobs like email delivery and notification processing.
- **Spotify:** Uses Kafka to handle streaming events, like logging song plays and updating recommendations.
- **Uber:** Uses Kafka to manage events from drivers, riders, and trip updates in real-time.

- **Airbnb:** Uses RabbitMQ to queue background tasks such as sending booking confirmations and notifications.
- **LinkedIn:** Uses Kafka to process activity feeds, notifications, and data pipelines efficiently.

56. **Microservice 55:** When to use Kafka, RabbitMQ and Pub/sub?

When to use Kafka?

Best for: Large-scale event streaming, High-throughput real-time data processing, Multiple consumers reading the same data

Real-Time Example: User Activity Tracking (E-commerce App) or Driver location updates in Uber

- Imagine Amazon wants to track user actions (searches, clicks, purchases).
- This data needs to be analysed in real-time for recommendations, fraud detection, and analytics.
- Kafka is perfect here because:
 - It acts like a log, where events are stored and processed later.
 - Multiple services (analytics, recommendation engine, etc.) can consume the same data.
 - It can handle millions of events per second efficiently.

When to use RabbitMQ?

Best for: Short-lived messages (tasks that should be processed once and deleted), Asynchronous communication (background jobs).

Real-Time Example: Food Delivery App (Swiggy/Zomato)

- A customer places an order.
- The system needs to:
 - 1 Send order details to the restaurant.
 - 2 Notify the delivery agent.
 - 3 Send an SMS confirmation to the user.
- RabbitMQ is ideal here because:
 - It queues messages and ensures they are processed once.
 - Retries failed messages (e.g., if restaurant API is down, it retries later).
 - It pushes messages to consumers instead of waiting for them to fetch data.

When to use Pub/Sub?

Best for: Cloud-based applications (Google Cloud, AWS, Azure), Real-time notifications across multiple services.

Real-Time Example: Live Sports Score Updates

- ESPN or Cricbuzz wants to send live cricket scores to millions of users.
- Each score update should be sent to multiple services like:
 - The mobile app
 - The website
 - A TV ticker service

- Pub/Sub is ideal here because:
 - It can push real-time updates to millions of subscribers.
 - It scales automatically based on load.
 - No need to manage infrastructure (Google/AWS handles it).

Reference: [Day 8: Messaging Queues: RabbitMQ vs Kafka — Understanding the Right Tool for the Job | by Dev Cookies | Medium](#)

57. **Microservice 56:** Implementations of Kafka and RabbitMQ in Spring boot?

if you are interested to do samples follow the below links.

Kafka: [Integrating Kafka in a Spring Boot Application: A Detailed Guide | Medium](#)

RabbitMQ: [Spring Boot Microservices with RabbitMQ Example | by Ramesh Fadatara | Medium](#)

[Spring Boot and RabbitMQ: Messaging with AMQP | by Igor Venturelli | CodeX | Medium](#)

Code Link: [codedecode25/Kafka](#)

58. **Microservice 57:** Difference between Kafka and RabbitMQ?

Good to know these differences to answer scenario-based questions on Kafka and RabbitMQ

Feature	Kafka	RabbitMQ
Type	Distributed event streaming platform	Traditional message broker / message queue
Use Cases	Real-time analytics, log aggregation, streaming pipelines, event-driven microservices	Task queues, background jobs, async workflows, reliable message delivery
Message Model	Publish–Subscribe with consumer groups	Queue-based (Pub/Sub + Work queues + Routing)
Message Ordering	Guaranteed per partition	Ordering not guaranteed across consumers
Delivery Guarantees	At-least-once (default), at-most-once, exactly-once (with Kafka Streams)	At-most-once, at-least-once via ack, manual retry
Replay Capability	Yes - replay old events anytime	No - messages are deleted once consumed unless manually stored
Retention	Time-based or size-based retention	Messages removed once acknowledged

Feature	Kafka	RabbitMQ
High Throughput	Designed for huge throughput (millions/sec)	Lower throughput compared to Kafka
Best For	Streaming large volumes of ordered events	Reliable task processing and message delivery
When to Use	For data pipelines, logs, streaming	For job queues, notifications, workflows

4.Integration Patterns

59. **Microservice 58:** What are the integration patterns used in Microservice?

When we build a system with multiple microservices, clients want to communicate with our system.

For example:

Imagine we are building a university management system with multiple microservices:

- Student Service: stores student details
- Course Service: manages course and subject information
- Library Service: tracks book records and issued books
- Hostel Service: manages hostel room allocations

The client here is the university web portal used by staff and students.

If the client talks to each service directly:

1. The portal calls Student Service to get student personal details.
2. The portal calls Course Service to get the list of subjects.
3. The portal calls Library Service to get which books are issued.
4. The portal calls Hostel Service to get room details.

Every time someone logs in, the portal has to make four different calls to collect all the data.

This setup works but becomes messy when the number of services grows.

The client needs to know every service's location, handle errors from each, and combine data on its own.

Note: Big companies like **Netflix and Uber** also started this way when their systems were small, then later moved to more centralized ways to reduce client complexity.

This is exactly why many teams introduce **API Gateway or Aggregator patterns** later.

60. Microservice 59: What is API gateway pattern and why we use it?

When we build a system with many microservices, it's hard for the client to talk to each one. The client needs to know all service locations, handle multiple calls, and combine the data. This is where the API Gateway pattern helps.

An **API Gateway** works like the single main entrance to a building. Instead of letting everyone enter through different doors, we create one main door. Inside the building, people can go to different rooms. Similarly, the gateway takes the client's request and sends it to the right microservice.

Real world example: Online Shopping Portal

Let's say we build an online shopping portal with these microservices:

- Product Service – gives product details.
- Order Service – handles orders.
- Payment Service – manages payments.
- Delivery Service – tracks deliveries.

Without an API Gateway, the shopping website would need to talk to each of these services directly.

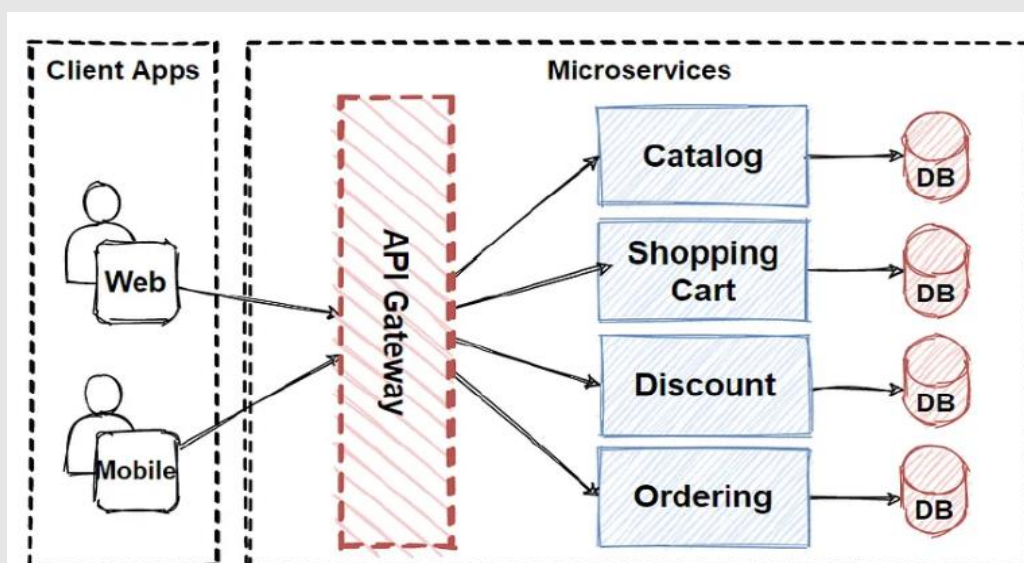
1. Product page calls Product Service.
2. Checkout page calls Order Service and Payment Service.
3. Delivery tracking page calls Delivery Service.

That means multiple calls, more complexity, and more chances of things breaking.

With an API Gateway, the website talks to just one gateway.

- The gateway receives the product page request and sends it to Product Service.
- It receives checkout requests and passes them to Order and Payment Services.
- It receives tracking requests and passes them to Delivery Service.

The client doesn't need to know where the services live or how they work. It only knows the gateway.



Reference: [API Gateway Pattern. In this article, we are going to talk... | by Mehmet Ozkaya | Design Microservices Architecture with Patterns & Principles | Medium](#)

61. **Microservice 60: What are the features of API gateway?**

An **API Gateway** is more than just a single-entry point for microservices. It provides several features that make a system easier to manage, faster, and more secure. Let's explain these features with an **online shopping portal** example.

1. Request Routing

The gateway decides which microservice should handle a client request.

Example:

- When a user opens the product page, the gateway sends the request to Product Service.
- When the user checks out, the request goes to Order Service and Payment Service.

The client doesn't need to know where the services are or how to call them.

2. Load Balancing

The gateway can distribute requests evenly among multiple instances of a microservice.

Example:

- If thousands of users open the product page at the same time, the gateway sends requests to different Product Service instances so that no single instance is overloaded.

3. Authentication and Security

The gateway can check if a request is valid and secure before letting it reach the services.

Example:

- Only logged-in users can place orders.
- The gateway verifies the user's token before forwarding the request to Order Service.

4. Response Aggregation

Sometimes the client needs data from multiple services at once. The gateway can combine responses into one.

Example:

- On the checkout page, the client wants product details, delivery options, and user balance.
- Instead of calling three services separately, the gateway collects all the data and sends one response.

5. Protocol Translation

The gateway can translate requests and responses between different formats.

Example:

- The client may call the gateway using HTTP, but internally, the gateway can talk to a service using gRPC.

6. Caching

The gateway can store frequently requested data to reduce repeated calls to services.

Example:

- Product catalog data doesn't change often. The gateway can cache it and serve many users quickly without hitting the Product Service each time.

7. Logging and Monitoring

The gateway can track all requests and help monitor system performance.

Example:

- If checkout requests fail often, the gateway logs it.
- Developers can use these logs to quickly find and fix issues.

62. Microservice 61: What are the ways to implement API gateway in Spring?

In a system with many microservices, an API Gateway acts as the single-entry point for the client. In Spring, we have simple ways to implement it. Let's see how, using an online shopping portal example.

1. Spring Cloud Gateway

This is the official Spring solution to create an API Gateway. It works with Spring Boot applications and is easy to configure.

Example:

- The online shopping portal has Product Service, Order Service, Payment Service, and Delivery Service.
- We set up Spring Cloud Gateway to route requests:
 - /products/** -> Product Service
 - /orders/** -> Order Service
 - /payments/** -> Payment Service
 - /delivery/** -> Delivery Service

Features we get:

- Routing requests to the correct service
- Load balancing using Spring Cloud LoadBalancer
- Security and authentication filters
- Logging and monitoring

This makes the client only talk to one URL instead of four separate services.

2. Zuul Gateway

Zuul is another popular API Gateway from Netflix, now integrated with Spring Cloud. It works similarly to Spring Cloud Gateway but was more widely used before Spring Cloud Gateway became standard.

Example:

- The shopping portal client calls /api/** on the Zuul Gateway.
- Zuul forwards requests to the correct microservice and can handle authentication, rate limiting, and logging.

3. Custom Spring Boot Gateway

We can also create a custom gateway using Spring Boot by writing a small service that acts as a router.

Example:

- The gateway service receives all requests at `/api/**`.
- It checks the path and forwards the request to the correct service using `RestTemplate` or `WebClient`.
- We can add security, caching, or response aggregation manually.

This approach gives full control, but it requires more effort to implement features like load balancing or monitoring.

Code Sample using `spring-cloud-gateway`

[Spring-Samples/APIGateway at main · CodingLyf-Fullstack/Spring-Samples](#)

Note: If you find any issues while running this sample, you can ping me or mail me.

This project contains 3 services.

- Product service
- Order service
- API gateway: we have used `spring-cloud-starter-gateway-server-webmvc` dependency for gateway

spring-cloud-starter-gateway-server-webmvc is a Spring Boot starter that enables the use of Spring Cloud Gateway with Spring WebMvc, providing a servlet-based implementation of the API Gateway. we can also use spring-cloud-starter-gateway-server-webflux for reactive programming model.

1. Start Product Service which runs on port 8081.
2. Start Order Service which runs on port 8082.
3. Start API Gateway which runs on port 8080.

Client calls through the gateway:

- GET `http://localhost:8080/products` -> returns the list of products from Product Service
- POST `http://localhost:8080/orders` with body "Laptop" -> returns "Order placed for: Laptop" from Order Service

How it works (You can test it from Postman):

Step 1: Client Sends Request

The client calls the gateway: `http://localhost:8080/products`

The client does not know about the internal services. It only talks to the Gateway.

Step 2: Gateway Receives Request

- Gateway is configured with routes in `application.yml`:

```
server:
  port: 8080 # Port where the API Gateway will run

spring:
  application:
    name: api-gateway # Name of the application
```

```

cloud:
  gateway:
    server:
      webmvc: # Using Spring MVC stack for the gateway
      routes: # List of routes the gateway will handle
        - id: product-service # Unique identifier for this route
          uri: http://localhost:8081 # The actual service URL where the request
will be forwarded
          predicates:
            - Path=/products/** # This route will handle any request that starts
with /products

        - id: order-service # Unique identifier for the second route
          uri: http://localhost:8082 # Forward requests to the Order Service
          predicates:
            - Path=/orders/** # This route handles requests starting with /orders

```

Gateway sees the path `/products/**` and it should forward the request to Product Service.

Step 3: Gateway Forwards Request to Service

- The Gateway makes an internal HTTP call to the Product Service at `http://localhost:8081/products`.
- If the path was `/orders/**`, it would forward to `http://localhost:8082/orders`.

Step 4: Service Processes Request

- **Product Service** receives the request.
- It executes the controller method:
- Service prepares the **response** (a list of products).

```

@RestController
public class ProductController {

    @GetMapping("/products")
    public List<String> getProducts() {
        return List.of("Laptop", "Phone", "Tablet");
    }
}

```

Step 5: Gateway Receives Response

- Gateway gets the response from Product Service.
- Any filters configured at the Gateway (like adding headers, rewriting paths, logging) are applied here.

Step 6: Gateway Sends Response to Client

- Gateway sends the final response back to the client.
- The client sees the response as if it came directly from the gateway, unaware of the internal microservices.

Example Response: `["Laptop", "Phone", "Tablet"]`

63. **Microservice 62: How API gateway works as load balancer?**

In a microservice system, we often run **multiple instances** of the same service to handle more traffic and avoid downtime. But then a question comes up:

When a request comes in, how does it get distributed across these service instances?

This is where the API Gateway acts like a load balancer. It does not just forward requests but also distributes them smartly to keep the system fast and stable.

Let us say we have three instances of **product-service** running in three instances:

Instance	Address
product-service-1	localhost:8081
product-service-2	localhost:8082
product-service-3	localhost:8083

When a client sends a request to /products, we do not want all traffic to go to just one instance. Instead, we want to share the load. That is exactly what the API Gateway does automatically.

How It Works Internally

1. API Gateway receives a request.
2. It checks which service to forward the request to.
3. It gets a list of service instances from Eureka (or static config).
4. It applies load balancing strategy.
5. It selects one instance and sends the request to it.

By default, it uses Round Robin, one request goes to instance 1, next to instance 2, and so on.

Tools for Load Balancing with API Gateway

Tool	Type	Purpose
Spring Cloud LoadBalancer	Client-side	Built-in load balancer in Spring
Eureka + LoadBalancer	Client-side	Service discovery + load balancing
API Gateway (Spring Cloud Gateway)	Server-side	Routes and load balances
Nginx	Server-side	Works as reverse proxy and load balancer
Kubernetes LoadBalancer	Cluster level	Manages load balancing in containers
Ribbon (Deprecated)	Client-side	Old Netflix load balancer

Let us see how load balancing works in a simple microservice setup using Spring Cloud Gateway and Eureka.

Step 1: pom.xml dependencies

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Step 2: application.yml configuration

```
spring:
  application:
    name: api-gateway

  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service //
          predicates:
            - Path=/products/**
```

Key Point

lb://product-service -> here **lb** stands for **load balancer**.

This tells Spring Cloud Gateway: "Send /products requests to product-service using load balancing."

So, if three instances of product-service are running, the Gateway will forward traffic to all of them in rotation using **Spring Cloud LoadBalancer**.

64. **Microservice 63:** What is difference between API gateway and load-balancer?

In a microservice system, both API Gateway and Load Balancer are used to handle incoming traffic, but they solve different problems.

A load balancer only distributes traffic across multiple servers or instances to improve performance and availability. It does not understand business logic or API routes. It simply takes incoming requests and shares them across instances to avoid overload.

API Gateway is more powerful. It sits at the entry point of the system and controls how requests move to different microservices. It can route requests to the correct service based on the URL path. It can also handle security, logging, authentication, request filtering, response transformation, and API composition.

Load balancer only works at the network level, but API Gateway works at the application level.

Reference: [API Gateway vs Load Balancer](#) | [Microservices Interviews](#)

65. **Microservice 64:** Can API Gateway replace load balancer completely?

No, API Gateway cannot fully replace a load balancer. A load balancer manages traffic distribution, fault tolerance, and high availability at the network level, while an API Gateway handles routing, authentication, rate limiting, and cross-cutting concerns. They solve different problems and are often used together in production.

66. **Microservice 65: Real time uses of API Gateway?**

To understand the real power of an API Gateway, let's see a few real-world examples from platforms we use every day.

API Gateway acts like the main entry point for all client requests in a system that has many microservices. It handles routing, authentication, and response aggregation. Let's see how it plays out in real life.

1. **Instagram - Managing Different Services Behind One Door**

When we open Instagram, so many things happen at once.

Our profile loads, new posts appear, stories show up, and notifications appear instantly.

Each of these comes from a different microservice:

- Feed Service (for posts)
- Story Service (for stories)
- Notification Service (for likes and comments)
- Chat Service (for messages)
- User Service (for profile data)

Instead of the mobile app calling each of these services separately, all requests go through one single API Gateway.

The Gateway receives the request, checks security, and routes it to the right service. It then combines the responses and sends them back to the app.

This makes the app faster, lighter, and easier to maintain.

67. **Microservice 66: Some more Questions on API Gateway**

1. What if multiple instances of same service are running, then how API gateway routes?

When multiple instances of the same service are running, the API Gateway does not choose randomly. It asks the Service Registry like Eureka (**Refer Question – Microservice 71**) for all available instances. Then it uses load balancing, usually round robin, to send each request to a different instance. This helps share load and improves performance.

2. What if Eureka is not included?

If Eureka is not used, the API Gateway cannot discover service instances automatically. In that case, we must configure fixed service URLs manually in the gateway. Load balancing will not be dynamic. It can only work if we provide static addresses or use another service registry like Consul or Kubernetes.

3. How service registry works with Kubernetes?

In Kubernetes, we do not use a separate service registry like Eureka because Kubernetes itself becomes the registry.

Let us see how it works .

When a microservice is deployed, Kubernetes runs it as many pods. These pods keep changing. Some die, some get created, some move to new machines. Their IP addresses also keep changing.

Now we create a Kubernetes Service for that microservice. This Service gets a fixed name like *order-service*

Kubernetes stores a live list of all pods that belong to this service. It knows which pods are healthy and running.

Inside the cluster, every service name is added to DNS. So when another service or an API Gateway calls *http://order-service*

Kubernetes DNS finds the service. Kubernetes then picks one healthy pod behind it. The request is sent to that pod.

If a pod crashes, Kubernetes removes it from the list. If a new pod starts, Kubernetes adds it.

So, the service name always points to the right running pods.

4. What happens if the API Gateway goes down?

If the API Gateway goes down, all client requests fail because it's the main entry point.

To avoid this, we usually run multiple gateway instances with load balancing and health checks. If one fails, another takes over, keeping the system available without interruptions.

5. How does an API Gateway handle traffic during peak hours or sudden load spikes?

During high traffic, the gateway scales horizontally by adding more instances automatically. Load balancers distribute requests across these instances. It may also use caching and rate limiting to reduce backend load, ensuring all services continue running smoothly without delays or failures.

6. What strategies can we use to make an API Gateway highly available?

We can make an API Gateway highly available by running it on multiple servers or containers across different zones. Load balancers manage traffic, while health checks detect failures and reroute requests. Auto-scaling and backups ensure smooth operation even during heavy traffic or downtime.

7. Can API Gateway perform authentication and authorization? How?

Yes, the API Gateway can handle authentication and authorization before sending requests to services. It checks tokens, API keys, or credentials, ensuring only valid users access protected routes. This centralizes security, reduces duplication across services, and simplifies managing permissions for all incoming requests.

8. What role does API Gateway play in request and response aggregation?

The API Gateway collects data from multiple microservices, combines it, and sends a single response to the client. For example, on a shopping app, it can merge product, price, and review data. This reduces the number of client requests and improves overall performance.

9. Can API Gateway be used to connect both internal and external clients?

Yes, an API Gateway can handle requests from external users like mobile apps and internal systems like dashboards. It can separate routes or apply different rules for each, ensuring security and controlling how external or internal users interact with microservices.

10. How do microservices communicate internally if the Gateway is for external calls?

Microservices often communicate internally through direct service-to-service calls, message queues, or event-driven systems. The API Gateway is mainly for handling external client requests, while internal communication usually happens through a private network or service mesh designed for secure and efficient interactions.

11. Why do large companies like Netflix and Amazon rely heavily on API Gateways?

Large companies like Netflix and Amazon use API Gateways to manage millions of requests efficiently. The gateway handles routing, security, caching, and load balancing in one place. It simplifies

system management, reduces network complexity, and keeps user experiences fast and consistent across platforms.

If need more questions on Spring cloud gateway, check this link:

[Spring Cloud Gateway — Crash Course | by A cup of JAVA coffee with NeeSri | Medium](#)

68. Microservice 67: If API gateway is single entry point, how does it handle millions of requests?

When we say an API Gateway is a single-entry point, we don't mean there's only one machine sitting somewhere doing all the work.

It means, it's one **logical entry point** for clients means a single URL or domain that clients talk to. Behind that, there are many **instances** (or copies) of the gateway running in parallel.

Think of it like a big airport with one main gate for passengers.

From the outside, it looks like one entry point. But once we go in, there are dozens of counters and staff members handling passengers at the same time. That's how the system scales.

How API Gateway Handles Millions of Requests

1. Load Balancing

There are multiple gateway instances running behind the same URL. A load balancer sits in front of them and distributes incoming traffic evenly.

Example:

If one million requests come in, the load balancer splits them across hundreds of gateway instances. Each instance handles a small portion of the total traffic.

This prevents any single server from overloading.

2. Auto Scaling

Modern gateways are deployed in containerized environments like **Kubernetes** or **AWS ECS**.

These systems automatically create more gateway instances when traffic increases and remove extra ones when traffic drops.

Example:

If traffic doubles during a sale or an event, new gateway containers are automatically added within seconds.

3. Caching

The gateway can store frequently accessed data in memory for quick access.

For example, static responses like product categories or user profiles can be cached.

This reduces the number of calls to backend services.

Example:

If thousands of people request the same product details, the gateway returns the cached response instead of asking the Product Service every time.

4. Rate Limiting and Throttling (Refer Question – Microservice 106,107)

The gateway controls how many requests a client can make in a given time.

This prevents misuse and protects backend services from sudden traffic spikes.

Example:

If one user tries to send 500 requests per second, the gateway allows only a few and blocks the rest.

5. Fault Tolerance and Circuit Breakers (Refer Question – Microservice 102-104)

If a backend service fails or becomes slow, the gateway detects it and temporarily stops sending requests to that service.

It can return a fallback response or an error message to the client without affecting other parts of the system.

Example:

If the Review Service goes down, the gateway can still serve product and price details while skipping reviews.

6. Horizontal Scaling

The gateway doesn't need a single powerful machine. It's better to have many smaller machines or containers running in parallel.

Together, they can handle millions of requests per second efficiently.

Real World Example - Netflix

Netflix is one of the best examples of this.

It has a single API Gateway that manages requests from millions of users every minute.

Behind the scenes, hundreds of gateway instances run across different regions, automatically scaling up or down depending on traffic.

Load balancers distribute requests, caching reduces load, and monitoring tools watch performance in real time.

69. Microservice 68: What is the Aggregator pattern?

When a system has many microservices, there are times when we need to combine data from more than one service to build a single response. This is where the Aggregator pattern helps.

Instead of the client calling each service one by one and waiting for all responses, one single service takes that job. It collects data from different services, combines them, and returns one final result.

Let's imagine an online shopping portal. When we open a product page, we don't just see just the product name. We will also see the description, price, reviews, ratings, delivery details, and stock availability. All this information comes from different microservices:

- Product Service gives the name and details of the product.
- Price Service gives the current price.
- Review Service gives customer reviews and ratings.
- Inventory Service gives stock status.

If the client tries to call each service directly, the client must make multiple calls, handle different responses, and merge everything. That adds complexity and increases load time.

Instead, we use an Aggregator Service.

The Aggregator Service receives one request for product details. It then calls all required services in the background, collects their responses, merges them, and returns one clean and complete response.

So, the client gets everything in one shot.

This also makes the system easy to change later. If one of the services changes, only the aggregator logic needs to be updated. The client stays the same.

Simple flow for product details page:

1. Client sends request to Aggregator Service -> /product-details/123
2. Aggregator calls Product Service, Price Service, Review Service, and Inventory Service.
3. All the data is combined into one single structured response.
4. Client receives a single response with everything needed for the page.

This pattern improves performance and keeps the client simple. It also gives better control over how different service data is stitched together.

In Spring, this is often done by creating a dedicated aggregator controller or using the API Gateway with routing and fan-out logic. Tools like WebClient, RestTemplate, or Feign clients can be used to call other services and merge the results.

Sample Code using Aggregator Controller + Feigh Client.

```
//Aggregator Controller
@RestController
public class UserProfileAggregatorController {

    @Autowired
    private UserServiceClient userServiceClient;

    @Autowired
    private AddressServiceClient addressServiceClient;

    @Autowired
    private ContactServiceClient contactServiceClient;

    @GetMapping("/user-profile/{userId}")
    public UserProfile getUserProfile(@PathVariable String userId) {

        // Call multiple microservices
        User user = userServiceClient.getUser(userId);
        Address address = addressServiceClient.getAddress(userId);
        Contact contact = contactServiceClient.getContact(userId);

        // Build combined response
        UserProfile profile = new UserProfile();
        profile.setUserId(userId);
        profile.setName(user.getName());
        profile.setEmail(contact.getEmail());
        profile.setPhone(contact.getPhone());
        profile.setCity(address.getCity());
        profile.setCountry(address.getCountry());

        return profile;
    }
}
```

Feign Client:

```

@FeignClient(name = "user-service")
public interface UserServiceClient {
    @GetMapping("/users/{id}")
    User getUser(@PathVariable String id);
}

@FeignClient(name = "address-service")
public interface AddressServiceClient {
    @GetMapping("/address/{userId}")
    Address getAddress(@PathVariable String userId);
}

@FeignClient(name = "contact-service")
public interface ContactServiceClient {
    @GetMapping("/contact/{userId}")
    Contact getContact(@PathVariable String userId);
}

```

This code acts as an **API Aggregator** in Spring Boot. It receives one request from the client and internally calls multiple services like User Service, Address Service, and Contact Service. It then combines all responses and sends one final merged response back to the client.

70. Microservice 69: What is the API Composition and how does it differ from Aggregator?

In a microservice system, a single feature sometimes needs data from many services. API Composition is a pattern used to combine data from multiple microservices and build a meaningful response. It not only merges data but may also apply small business logic before sending the result back.

What is API Aggregation then?

A service (Aggregator) calls multiple micro-services, retrieves their data, and combines them into a single, unified response. An Aggregator collects data from multiple services and combines them into a single, unified response. It is often implemented as a separate micro-service or gateway layer.

Examples:

1. Client wants to get user profile details. Aggregator calls:

- UserService → user data
 - OrderService → user orders
 - LoyaltyService → points balance
- Then combines the results into one DTO.

2. Fetching user details, order history, and loyalty points from three different micro-services and returning a single response to the frontend.

What is API Composition

API Composition is not a business service. It is a read layer for screens. It is used only to show data on UI. It calls many services and joins their data. But it does not apply rules.

For example, a Dashboard API.

It calls

- InventoryService for stock
- SalesService for sales
- NotificationService for alerts

It simply puts them together and sends them to the screen. It is just fetch and join.

API Composition is often used in query side of CQRS, Often used in GraphQL, BFF (Backend for Frontend), or API Gateway designs.

Reference:

71. **Microservice 70:** How API Composition is different from GraphQL?

API Composition and GraphQL are both used to collect data from multiple services, but they are not the same. They solve different problems and follow different approaches.

API Composition is a backend design pattern. It lives inside the server and combines data from different services to build a meaningful response. It can also apply business rules while merging data. The shape of the response is always fixed by the backend team. Clients cannot change it. API Composition is useful when the response needs some backend decisions before sending it to the client, like calculating a final status or merging business information.

GraphQL is a query language for APIs on Frontend. It allows the client to decide what data it needs. This gives full control to the frontend. Instead of receiving a fixed response, the client selects only the required fields. GraphQL works best when the main goal is to reduce multiple API calls and avoid sending unwanted data.

72. **Microservice 71:** What is service discovery and service registry?

In traditional systems, everything used to run on one server. We could simply say:

“Connect to localhost on port 8080.”

But in a microservice setup, things are different. Services can move between machines, restart automatically, or even scale up and down depending on demand.

One day, the Payment Service might run on 10.0.1.12, and the next day it might move to 10.0.1.15. If we hardcode these addresses, we'll break communication every time something changes.

We need a system that always knows where each service is running, no matter how many times it moves.

That's where **Service Registry** and **Service Discovery** come in.

What Is a Service Registry

A Service Registry is like a dynamic address book for the microservices. The Service Registry stores that information. If a service shuts down or crashes, it removes that entry automatically.

This means at any moment, the Service Registry always knows the current list of healthy, running services and where they are located.

What Is Service Discovery

Service Discovery is how other services find and connect to each other using the information inside the Service Registry.

When one service wants to talk to another, it doesn't guess the address. Instead, it asks the Service Registry: The registry responds with the current location. Then the calling service connects directly.

This discovery process happens automatically, and the services don't need to know each other's exact locations anymore.

How It Works in Microservices

Let's take an example of three services:

- Order Service
 - Payment Service
 - Notification Service
1. When the **Payment Service** starts, it registers itself with the Service Registry (for example, Eureka).
 2. The **Order Service**, before calling Payment Service, asks the registry where it is.
 3. The registry replies with the current address of Payment Service.
 4. The **Order Service** sends its request.

If Payment Service moves to a new server, the registry automatically updates its address. No manual change is needed anywhere in the system.

There are two ways to discover services:

1. **Client-Side Discovery**
2. **Server-Side Discovery**

Tool for Service Registry and Discovery is:

- **Netflix Eureka**: very popular in Spring Boot microservices.

Reference: [What is service discovery really all about? - Microservices Basics Tutorial](#)

73. Microservice 72: What is Server-side discovery and Client-side discovery.?

1. Client-Side Discovery

In client-side discovery, the client is smart.

When Service A wants to call Service B, it does not call it directly. Instead, it first asks the Service

Registry (like Eureka or Consul) where Service B is running. The registry gives the current address, and Service A connects to it.

The client decides which instance of Service B to call if there are multiple running. It even handles load balancing on its own.

How It Works

Client -> Service Registry -> Get address of Service B -> Call Service B

Example

Let's say Order Service wants to talk to Product Service. It asks Eureka for the address of Product Service and then calls it.

Tools Used

- Eureka (Netflix OSS)
- Ribbon (Client-side load balancer)

Used

- Used in Spring Boot with **Eureka + Feign Client**
- Common in Netflix-style microservice systems

2. Server-Side Discovery

In server-side discovery, the client is simple.

It sends the request to a load balancer or API gateway, which then talks to the Service Registry, finds the correct instance of the service, and forwards the request.

The client does not worry about where the services are. The server or gateway takes care of everything.

How It Works

Client -> Load Balancer / API Gateway -> Service Registry -> Service B

Example

A mobile app sends a request to the **API Gateway**. The gateway checks the service registry to find **Product Service** and forwards the request there.

Tools Used

- Spring Cloud Gateway
- AWS Elastic Load Balancer (ALB)
- Nginx
- Kubernetes Service

Used

- Common in cloud systems (AWS, Kubernetes)
- Modern microservice architecture

Simply:

Client-side discovery is like every person checking Google Maps themselves to find a location.

Server-side discovery is like asking a receptionist who guides us to the right place.

74. Microservice 73: When to choose Server-side discovery and Client-side discovery?

In client-side discovery, each service directly talks to the service registry to find the location of another service. The client is responsible for choosing which instance to call.

Example Scenario

We build a small system with 6–8 microservices using Spring Boot and Eureka. We want fast service-to-service communication inside the system and don't want to add a gateway yet. In this case, client-side discovery is easy and fits well.

In server-side discovery, the client does not discover services. Instead, it sends the request to a load balancer or API Gateway, and that component finds the service location using the registry.

Example Scenario

We have 20+ services and our frontend and mobile apps connect through a single URL using API Gateway. The API Gateway handles routing, load balancing, rate limiting, logging, and authentication. In this case, server-side discovery fits perfectly.

75. **Microservice 74:** How do you implement Service registry in Spring?

We can use **Eureka**, built by Netflix, is one of the most popular tools for this in Spring Boot.

How Eureka Works

1. Eureka Server (Service Registry)

This is the main directory where all microservices register themselves.

2. Eureka Clients (Microservices)

Each service (like User Service, Product Service) registers itself with the Eureka Server. It also keeps checking in ("heartbeat") so Eureka knows it's still alive.

3. Service Discovery

When one service wants to call another (for example, Order Service wants to call Product Service), it asks Eureka for the current location of that service.

Code link: [Spring-Samples/ServiceRegistryWithEureka at main · CodingLyf-Fullstack/Spring-Samples](#)

Project Setup Overview

We'll create **three small Spring Boot projects**:

1. **Eureka Server**: the service registry
2. **Product Service**: a client that registers itself
3. **Order Service**: another client that discovers and calls Product Service

Step 1: Create Eureka Server

Dependencies (pom.xml)

Add this dependency:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaserverApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaserverApplication.class, args);
    }
}
```

application.yml

```
spring:
  application:
    name: eurekaserver
server:
  port: 8761

eureka:
  client:
    register-with-eureka: false # Server doesn't register itself
    fetch-registry: false # Server doesn't fetch registry
```

1. register-with-eureka: false

This setting tells the Eureka Server not to register itself as a service inside the registry. Only client services like product-service or order-service should register. The Eureka Server is just a directory, not a service that others should discover, so it does not need to register itself.

2. fetch-registry: false

This tells the Eureka Server not to fetch or download a list of services from another Eureka Server. Since this setup has only one Eureka Server, there is no other registry to communicate with. That means it should not try to get any service information from anywhere else.

Simply, fetch-registry: false means the Eureka server will **not try to get a list of services** from another Eureka server.

Now start the server and open: <http://localhost:8761>

We can see the Eureka dashboard; it's empty for now because no services are registered yet.

Step 2: Create Product Service**Dependencies (pom.xml)**

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableDiscoveryClient
public class ProductserviceApplication {

    public static void main(String[] args) {
```

```

        SpringApplication.run(ProductserviceApplication.class, args);
    }
}

```

application.yml

```

server:
  port: 8081

spring:
  application:
    name: productservice

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

```

This setting tells the microservice where the Eureka Server is running so it can register itself there. defaultZone is the address of the Eureka Server. Here, it is running on localhost at port 8761. So this product-service will connect to Eureka at <http://localhost:8761/eureka/> during startup.

Controller

```

@RestController
public class ProductController {

    @GetMapping("/products")
    public List<String> getProducts() {
        return List.of("Laptop", "Phone", "Tablet");
    }
}

```

Now, when we start this service, it automatically registers itself with the Eureka server. We can see it listed under Instances currently registered on the Eureka dashboard which is running here <http://localhost:8761>.

Step 3: Create Order Service

This service will discover the Product Service from Eureka and call it.

Dependencies (pom.xml)

```

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

```

Main Class

```

@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients

public class OrderserviceApplication {

```

```

    public static void main(String[] args) {
        SpringApplication.run(OrderserviceApplication.class, args);
    }
}

```

application.yml

```

server:
  port: 8082

spring:
  application:
    name: orderservice

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

```

Feign Client (Refer Question – Microservice 125)

```

@FeignClient(name = "productservice")
public interface ProductClient {
    @GetMapping("/products")
    List<String> getProducts();
}

```

Controller

```

@RestController
public class OrderController {

    private final ProductClient productClient;

    public OrderController(ProductClient productClient) {
        this.productClient = productClient;
    }

    @GetMapping("/orders")
    public List<String> getOrders() {
        List<String> products = productClient.getProducts();
        return List.of("Order1 - " + products.get(0), "Order2 - " + products.get(1));
    }
}

```

Eureka Dashboard View

Once all three services are running, open: <http://localhost:8761>

The screenshot shows the Spring Eureka web interface. The top navigation bar includes the Spring Eureka logo and links for HOME and LAST 1000 SINCE STARTUP. The main content area is divided into several sections:

- System Status:** A table showing environment details.

Environment	test	Current time	2025-12-11T09:58:20 +0530
Data center	default	Uptime	00:03
		Lease expiration enabled	false
		Renews threshold	5
		Renews (last min)	0
- DS Replicas:** A section with a link to localhost.
- Instances currently registered with Eureka:** A table listing registered services.

Application	AMIs	Availability Zones	Status
ORDERSERVICE	n/a (1)	(1)	UP (1) - DESKTOP-LOOS583.orderservice:8082
PRODUCTSERVICE	n/a (1)	(1)	UP (1) - DESKTOP-LOOS583.productservice:8081
- General Info:** A section for general information.

We Can see:

- product-service
- order-service

Both marked as “UP.”

How It Works

1. When **Product Service** starts, it registers itself with **Eureka Server**.
2. **Order Service** also registers itself.
3. When we hit <http://localhost:8082/orders>, the Order Service asks Eureka for “productservice.”
4. Eureka returns its current location (host and port).
5. The call is made automatically through **Feign**, no hardcoded IPs needed.

We can see what are the services running in Eureka server from this address:

localhost:8761/eureka/apps

76. **Microservice 75:** How Eureka server works internally?

Eureka is a Service Registry used in microservices to keep track of all running service instances. It knows which services are active, which are down, and where they are running.

To understand it better, let us see how it works inside.

a) How It Tracks Service Instances?

When a microservice like order-service starts, it does not know who else is running. So, it sends a message to Eureka Server and says: "I am order-service and I am running at this address."

- This first message is called service registration.
- Eureka Server saves this information in memory as a service registry list.
- If we run two or three instances of the same service, like:

order-service running on [port 8082](#)

order-service running on [port 8083](#)

```
order-service running on port 8084
```

Then all three instances register separately with Eureka, but under the same service name. Eureka tracks them as separate entries and can send traffic to any one of them.

b) How Eureka Knows If a Service Is Alive?

Just registering once is not enough. A service may crash later or we may stop one instance. To solve this, Eureka uses a **heartbeat system**.

Every service that registers with Eureka **sends a heartbeat signal every 30 seconds**.

This is a simple message that says: "I am still alive. Keep me in the registry."

If Eureka does not receive a heartbeat from a service for around **90 seconds**, it assumes that the service is dead and removes it from the list.

c) Who Sends the Heartbeat?

The **Eureka Client** sends the heartbeat automatically.

There is no manual work here. Spring Cloud Eureka has a small background task that keeps sending heartbeat messages to Eureka Server every 30 seconds using HTTP.

So, if we start three instances of a service, each one sends its own heartbeat:

4) What if service stops sending heart beat due to network failures?

Eureka is careful during network issues. If many heartbeats suddenly stop (due to network failure, not real service failure), Eureka does not remove services quickly. It activates something called **self-preservation mode**.

This avoids false removals and keeps the system stable during temporary issues.

For detailed explanation about service discovery, check this video link: [Java Microservices Concept Walkthrough - Service Discovery and Registration | Spring Cloud Eureka #5](#)

77. **Microservice 76:** If two or more instances of same service are running, how Eureka client decides which service to call?

In a microservice system, we often run more than one instance of the same service. This is done to handle more users, improve performance, and avoid failure if one instance crashes. For example:

All three instances register themselves with Eureka Server. Now the question is:

When another service, like order-service, calls product-service, which instance does Eureka choose?

```
product-service running on port 8081
```

```
product-service running on port 8082
```

```
product-service running on port 8083
```

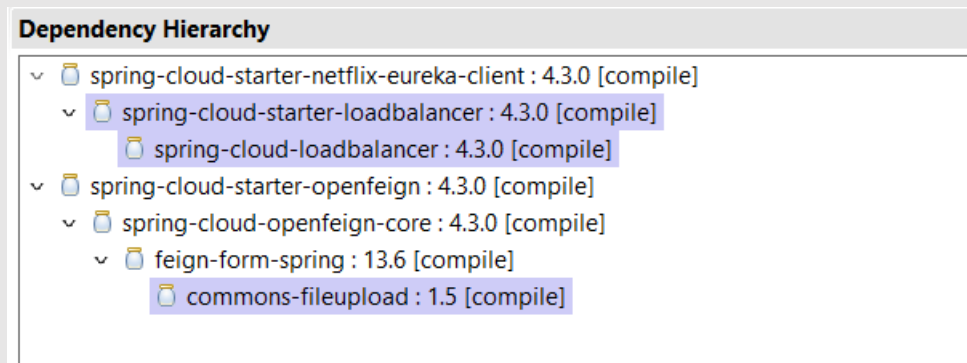
a) How It Works?

Eureka itself does not decide which instance to call. It only gives a list of available service instances. The Eureka Client in the calling service (for example, order-service) decides which one to call by using load balancing.

b) Who Does the Load Balancing?

In a Spring Boot Eureka setup, load balancing is handled on the client side. Earlier, this was done by Netflix Ribbon, and now it is done by Spring Cloud LoadBalancer, which replaced Ribbon.

spring-cloud-starter-netflix-eureka-client dependency internally contains *spring-cloud-loadbalancer*



c) How Instance Is Selected?

Spring Cloud LoadBalancer uses **Round Robin** by default. That means it sends requests one by one to each instance in order.

Point to remember: When we add Eureka-Client in a Spring Boot microservice, Spring automatically enables load balancing without any extra setup.

Earlier, Netflix Ribbon was used for client-side load balancing, but it is now deprecated. Spring replaced Ribbon with Spring Cloud LoadBalancer, which is lighter, faster, and maintained by the Spring team.

78. Microservice 77: What is the difference between API Gateway and Service Discovery?

At first, both API Gateway and Service Discovery may seem similar because they both deal with how services communicate.

But their purposes are different. API Gateway focuses on external access, and Service Discovery focuses on internal communication.

Let's break it down in simple, clear terms.

1. Purpose

API Gateway acts as the single-entry point for all external client requests. It decides which microservice should handle the request and forwards it there.

It's like the main reception desk of a company, everything from the outside world passes through it.

Service Discovery helps microservices find and communicate with each other *inside* the system.

It's like the internal directory of that company, where departments look up where others are located.

2. How They Work

- **API Gateway:**

A client (like a mobile app or website) never calls microservices directly. It always sends the

request to the API Gateway first.

The gateway then routes the request to the right microservice, for example, Order, Payment, or User.

- **Service Discovery:**

Inside the system, when one service (say, Order) needs to talk to another (say, Payment), it asks the **Service Registry** for the current address of that service.

It helps the services locate each other dynamically without hardcoding IPs or URLs.

3. Example in a Microservice Setup

Let's say we have:

- **Client (Frontend App)**
- **API Gateway**
- **Order Service**
- **Payment Service**
- **Notification Service**
- **Eureka (Service Registry)**

Here's what happens:

1. The **Order**, **Payment**, and **Notification** services register themselves with Eureka (Service Registry).
2. The **API Gateway** also connects to Eureka.
3. When a client sends a payment request, the Gateway looks up the **Payment Service** address in Eureka.
4. The request is then routed correctly to the live instance of Payment Service.

Without Service Discovery, the Gateway wouldn't know where to send the request.

79. **Microservice 78:** How API Gateway and Eureka server works together?

In a microservice system, both **Eureka** and **API Gateway** play important roles, and they often work side by side. They solve two different problems:

- Eureka helps services find each other
- API Gateway helps control how requests enter the system

Together, they make a clean, safe, and flexible architecture.

Why Do We Need Them Together?

Let us say we are building an **online movie platform (Book my show)**. We have these services:

- user-service
- movie-service
- review-service

- payment-service

Now imagine a mobile app wants to fetch a user's watchlist. Without a gateway, it would need to know the address of each service. This becomes messy when services run on different servers or different ports, and when we scale to more instances. Also, we want to secure all services at one place.

How Eureka Helps

Eureka acts like a **service directory**. Each microservice registers itself with Eureka when it starts. If a service has two or three instances, all of them register with Eureka. Eureka keeps track of which ones are UP and healthy.

Example registry inside Eureka:

Service Name	Instance Address
movie-service	localhost:8081
movie-service	localhost:8082
review-service	localhost:8083
user-service	localhost:8084

Now Eureka knows where each service is running and movie-service is running in two instances.

How API Gateway Helps

Instead of the mobile app talking to services directly, all requests go through the API Gateway. It is the single-entry point to our system.

The gateway checks the incoming request, decides which service to route to, and sends it there. It also adds extra features like logging, authentication, rate limiting, and load balancing.

How They Work Together, Step by Step

Let us say a request comes from the mobile app: `GET /movies/top10`

Here is what happens inside:

1. The request first reaches the **API Gateway**
2. The Gateway checks its route table and sees this belongs to movie-service
3. But the Gateway does not know where movie-service is running, so it asks **Eureka**
4. Eureka responds with a list of available movie-service instances:
 - localhost:8081
 - localhost:8082
5. Gateway picks one instance using load balancing (round robin)
6. Gateway forwards the request to that instance
7. The response goes back to the client through Gateway

Everything happens behind the scenes. The client only talks to the Gateway and does not know anything about service locations. Services can scale, restart, move, and the client never gets affected.

Code Sample link: [How to Build an API Gateway with Spring Cloud Gateway and Eureka — The Beginner's Guide | by Vivek Rajyaguru | Medium](#)

80. **Microservice 79: Is Service discovery mandatory to have in distributed systems?**

No, Service Discovery is not always mandatory, but in most true distributed or microservice systems, it becomes almost essential.

When It's NOT Mandatory

We can skip Service Discovery when:

1. **We have a small, static system**
 - For example, only 2 or 3 services running on fixed servers.
 - Each service has a known IP or DNS name that rarely changes.
2. **We use a load balancer with static backend configuration**
 - Tools like Nginx or AWS ALB can forward traffic to fixed backend servers.
 - This works fine when scaling isn't dynamic.
3. **We deploy everything in a single machine or simple VM setup**
 - For small internal tools or prototypes, a discovery service might be unnecessary.

In these simple setups, hardcoded URLs or environment-based configuration files are enough.

When It BECOMES Mandatory

Service Discovery becomes critical when:

1. **We scale dynamically**
 - In container environments like **Kubernetes** services can come and go frequently.
 - The system must automatically detect these changes.
 - In Kubernetes, the service registry is a built-in, automated system that maps **Service names** to their stable **ClusterIP** addresses, which in turn route traffic to dynamic Pod IPs
2. **We use many microservices**
 - When 20+ services talk to each other, managing addresses manually becomes impossible.
3. **We deploy in cloud environments**
 - In AWS, Azure, or GCP, services are distributed across zones and networks, Service Discovery keeps track of them automatically.
4. **We need fault tolerance**
 - If one service instance fails, the discovery mechanism removes it from the registry, so requests aren't sent to dead nodes.

5. Deployment patterns

81. Microservice 80: What is the deployments pattern we use Microservice?

When we work with microservices, one of the key parts of the system is how we deploy these services. Deployment patterns help us decide where and how each service should run in production.

Each service can have its own lifecycle. That means we can update one service without touching the others. To make this work smoothly, we use different deployment patterns.

Let's imagine an online shopping portal with services like Product, Price, Order, Review, and Inventory. Each of these can be deployed in different ways depending on the system's needs.

1. Multiple Service Instances per Host

In this pattern, we run many services on the same machine or server. For example, the Product Service and Price Service might run on one machine, while Order and Review run on another.

This is simple to manage when the system is small. But as the number of services grows, it becomes harder to isolate issues. If one service fails, it may affect others on the same machine.

Example:

Product and Price Service are deployed together on one server. If the server goes down, both services become unavailable.

2. Single Service Instance per Host

Here, each service runs on its own machine or virtual machine. This gives better isolation and control. It's easier to restart or scale one service without touching the others.

Example:

Product Service runs on one machine, Price Service runs on another, and Order Service runs on a separate machine. If Price Service needs more resources, we add more machines only for it.

3. Service Instance per Container

In modern systems, we use containers to deploy services. A container is a lightweight package that has everything the service needs to run. This gives flexibility and makes deployments fast and consistent.

Example:

Product Service runs inside one Docker container, Price Service in another, and so on. We can scale each service by adding more containers whenever needed.

4. Serverless Deployment

Here, we don't manage any servers. The service runs only when needed. It's event-driven. This is useful for small or less frequently used services.

Example:

A Notification Service sends order confirmation messages. It doesn't run all the time. It runs only when an order is placed, sends the message, and stops.

82. Microservice 81: What is End to End Deployment Steps we follow in Microservice?

Note: This is high level of information on how we can handle deployment in distributed systems. From next question we will go deep on each step like (containerization, Docket, Kubernetes)

Let's walk through how a real microservice project is deployed from start to finish. Think of this like a journey. We build the project, pack it, move it, deploy it, and finally keep it running smoothly.

Step 1: Build the Application

Everything starts with the application itself. Let's say we build a small online shopping portal with services like Product, Order, and Payment.

Each service is developed using Spring Boot. We write code, add dependencies, and build it using Maven or Gradle to create a JAR file. This JAR file is what will run inside our container.

Step 2: Create a Docker Image (Refer Question Microservice-83 for Docker)

Once the service is ready, we containerize (Refer Question Microservice-82 for Containerization) it. We create a **Dockerfile** that tells Docker how to build the container image for our service. This image contains:

- JDK
- Our Spring Boot JAR
- All required configurations

Then we run:

```
docker build -t product-service:1.0 .
```

This creates an image named product-service:1.0.

We repeat this for other services like order and payment. Sample docker file:

```
# Step 1: Use OpenJDK as the base image
FROM openjdk:17-jdk-slim

# Step 2: Create a directory inside the container
WORKDIR /app

# Step 3: Copy the Spring Boot JAR into the container
COPY target/product-service-1.0.jar app.jar

# Step 4: Expose port (optional, for documentation)
EXPOSE 8080

# Step 5: Run the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

To build the image: `docker build -t product-service:1.0 .`

To run the container: `docker run -p 8080:8080 product-service:1.0`

The image contains:

- JDK (from openjdk:17-jdk-slim)
- Your Spring Boot JAR (copied as app.jar)
- All configs packaged inside the jar

Step 3: Push the Image to a Container Registry

A container registry is like a storage place for these images.

Instead of keeping everything on our laptop, we push them to a place where our deployment platform can pull from.

We can use:

- Docker Hub
- Amazon ECR
- Google Container Registry
- Azure Container Registry

For example:

```
docker tag product-service:1.0 myrepo/product-service:1.0  
docker push myrepo/product-service:1.0
```

Now the image is safely stored in the registry.

Step 4: Prepare Kubernetes Deployment Files - Container Orchestration

When we have one container, Docker is enough. But when we have many containers (for product, order, payment, notification services), we need something to manage them.

That's where Kubernetes comes in.

Kubernetes can:

- Deploy multiple containers
- Restart failed containers
- Scale up or down automatically
- Load balance traffic

Here, we write YAML deployment files. Sample YAML File:

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: product-service  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: product-service  
  template:  
    metadata:  
      labels:  
        app: product-service  
    spec:  
      containers:  
        - name: product-service  
          image: product-service:1.0  
          ports:  
            - containerPort: 8080  
---  
apiVersion: v1  
kind: Service  
metadata:
```

```
name: product-service
spec:
  type: ClusterIP
  selector:
    app: product-service
  ports:
    - port: 8080
      targetPort: 8080
```

Kubernetes learns from this YAML:

- **Which image to run:** product-service:1.0
- **How many containers:** replicas: 3
- **How to expose it:** Service on port 8080
- **Self-healing:** Kubernetes restarts failed pods automatically
- **Scaling:** Change replicas for scale up/down

Step 5: Deploy to Kubernetes Cluster

Once YML files ready, we apply them to the Kubernetes cluster.

```
kubectl apply -f product-deployment.yaml
```

```
kubectl apply -f product-service.yaml
```

Kubernetes pulls the image from the registry and runs it in containers.
We repeat the same for the order and payment services.

Step 6: API Gateway and External Access

Once the services are running, we usually don't want clients to call each one directly.
We set up an API Gateway that acts like the front door.

The gateway receives incoming requests and forwards them to the correct service.
For example:

- /products goes to Product Service
- /orders goes to Order Service

We can use Spring Cloud Gateway, Nginx, or any API gateway.

Step 7: Configuration and Secrets

Every environment (dev, test, prod) has its own configuration.
We store them safely using:

- ConfigMaps (for normal configs)
- Secrets (for passwords, tokens)

Kubernetes mounts them into containers at runtime. This keeps everything secure and clean.

Step 8: Scaling and Load Balancing

If traffic increases, we scale up the service.

```
kubectl scale deployment product-deployment --replicas=5
```

Now 5 containers handle incoming traffic. Kubernetes load balances them automatically.

Step 9: Observability and Monitoring

We set up tools to watch what's happening in the system. Metrics, logs, and tracing help us keep everything healthy.

Tools commonly used:

- Prometheus and Grafana for metrics

This step is key to catch problems early.

Step 10: Automate with CI/CD

Doing all these steps manually every time is slow. We set up a CI/CD pipeline.

Typical flow:

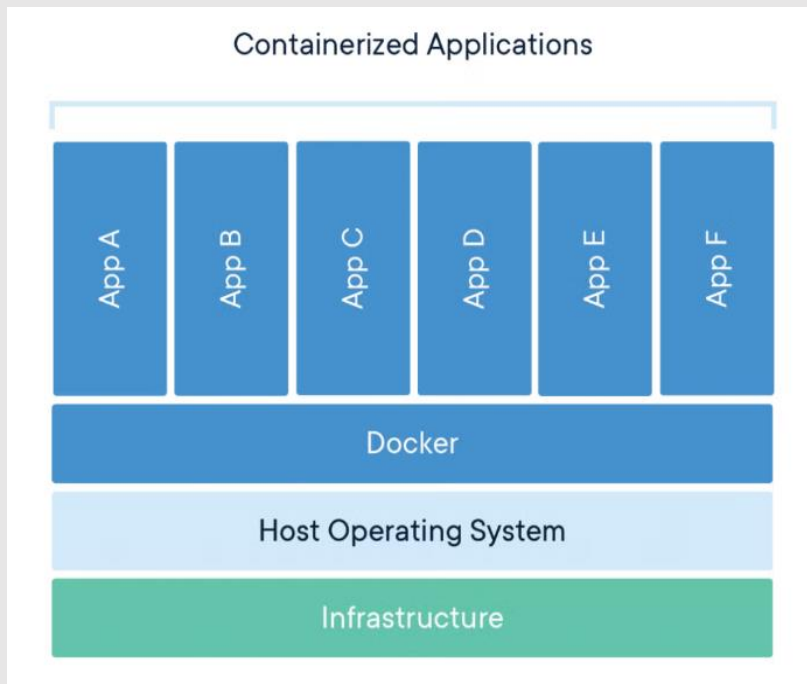
1. Developer pushes code to Git
2. Pipeline builds the Docker image
3. Pushes it to the registry
4. Deploys it to Kubernetes
5. Runs tests
6. Updates the running services

Tools like Jenkins, GitHub Action are often used.

83. **Microservice 82:** What is containerization and why it useful?

Let's imagine we built three services for a food delivery app: Items, Order, and Payment. Each one runs well on a local system. But when we move them to a testing server or production, things may break. One machine may have a different Java version. Another may not have the right library. This is where containerization comes in.

Containerization means packing everything needed to run an application into a single unit called a **container**. This includes the code, libraries, configuration, and runtime. Once we build the container, it will run the same way on any machine, whether it's a laptop, a server, or the cloud.



Simply put, containerization allows an application to be written once and run anywhere

Common Tools

The most used tool for containerization is **Docker**. It allows us to build, run, and manage containers with simple commands.

Other tools that work with containers are:

- Kubernetes (for managing many containers in production)

Example with Product Service

Let's say we have a simple Spring Boot Product Service.

Step 1: We first build the JAR file: `mvn clean package`

Step 2: Then we create a file called **Dockerfile** in the root folder:

```
# Base image with Java
FROM openjdk:17-jdk-slim

# Copy the jar file into the container
COPY target/product-service-0.0.1-SNAPSHOT.jar product-service.jar

# Run the application
ENTRYPOINT ["java", "-jar", "/product-service.jar"]
```

Step 3: Build the Docker image: `docker build -t product-service:1.0`

Step 4: Run the container: `docker run -p 8081:8081 product-service:1.0`

The Product Service is now running inside a container. If we move this image to any machine that has Docker installed, it will behave the same way.

Uses:**1. Runs the same everywhere**

A container has the application and everything it needs. So it runs the same on any machine.

Example

A Spring Boot app works the same on a laptop, a test server or a cloud server without setup issues.

2. Fast startup time

Containers start quickly because they are light.

Example

A product service container can start in a few seconds during deployment, which keeps downtime low.

3. Easy scaling

We can run more containers when load increases.

Example

During a big sale, an order service can scale from 2 containers to 10 containers to handle more traffic.

4. Easy to roll back

If a new version fails, we can switch back to the old image.

Example

Version 1.1 of payment service fails. We instantly move back to version 1.0 without trouble.

5. Better resource usage

Containers use memory and CPU more efficiently than full virtual machines.

Example

Instead of running five VMs, we run ten containers on the same server without performance issues.

6. Simple deployments

A Dockerfile builds a repeatable image, making deployments predictable.

Example

A team can deploy the same image to development, testing and production without changes.

7. Easy updates

We can update a service by replacing the old container with a new one.

Example

A bug fix in the catalog service is pushed as a new image, and Kubernetes replaces old containers automatically.

Reference Video : [Lets understand containers | 5 simple steps to create a docker container](#)

84. **Microservice 83:** How are Docker and Kubernetes related??

Docker and Kubernetes are often mentioned together, but they solve different parts of the same problem. Docker helps us create and run containers. Kubernetes helps us manage many containers once our system grows.

Docker is a container tool. It lets us pack the application, its libraries and its settings into one small unit. This unit runs the same on any machine. It is simple to build and great for small or medium services.

The challenge starts when we have many containers. A real system may have product, order, payment and notification services running at the same time. Each service may need five or ten containers during busy hours. Managing all of these by hand becomes difficult.

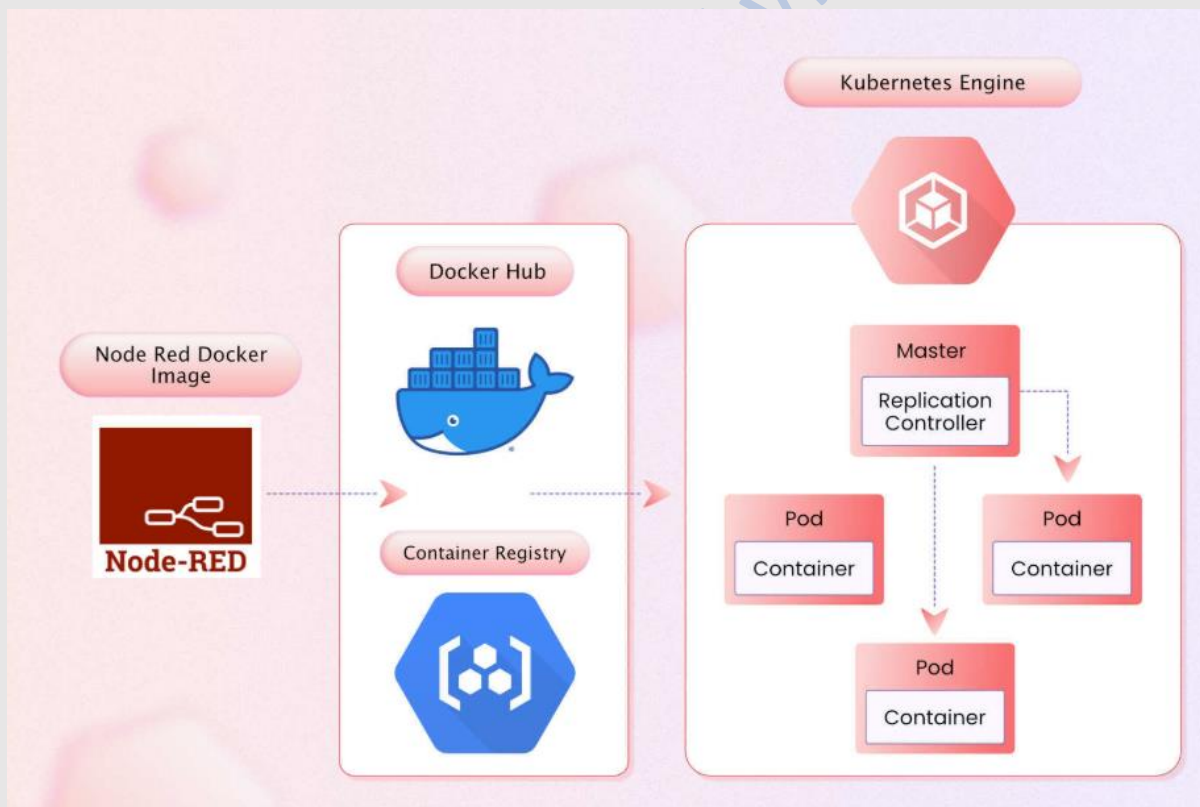
This is where Kubernetes helps. Kubernetes is a container manager. It decides where containers run, restarts them if they fail, adds more containers when traffic increases and removes them when traffic goes down. It can also balance requests between containers and handle smooth updates.

A simple example makes it clear.

Docker builds a container for the order service.

Kubernetes runs ten copies of that container during a festival sale, keeps them healthy and sends traffic to all of them. When the sale ends, Kubernetes reduces it back to two containers.

Docker creates containers. Kubernetes takes care of them. They work side by side and make the whole system stable and scalable.



Common Tools

The most used tool for containerization is **Docker**. It allows us to build, run, and manage containers with simple commands.

Other tools that work with containers are:

- Kubernetes (for managing many containers in production)

Example with Product Service

Let's say we have a simple Spring Boot Product Service.

Step 1: We first build the JAR file: `mvn clean package`

Step 2: Then we create a file called **Dockerfile** in the root folder:

```
# Base image with Java
FROM openjdk:17-jdk-slim

# Copy the jar file into the container
COPY target/product-service-0.0.1-SNAPSHOT.jar product-service.jar

# Run the application
ENTRYPOINT ["java", "-jar", "/product-service.jar"]
```

Step 3: Build the Docker image: `docker build -t product-service:1.0`

Step 4: Run the container: `docker run -p 8081:8081 product-service:1.0`

The Product Service is now running inside a container. If we move this image to any machine that has Docker installed, it will behave the same way.

85. **Microservice 84:** What is Docker, how it useful and how it works?

Before Docker came into the picture, running software on different machines was a trouble. Every machine had its own setup, versions, and dependency issues.

Let's imagine a Product Service built on Java 17. On the development machine, everything works fine. But when it moves to the testing server, Java might be of a different version or some library might be missing. The service breaks.

Developers spend hours fixing environment problems instead of building features.

To fix these problems, many teams started using **virtual machines**.

How Virtual Machines solved it

VMs gave each application its own full operating system, so the environment stayed the same across machines. This reduced dependency issues but used heavy resources.

But virtual machines are heavy. Each VM runs its own full operating system, takes time to start, and eats a lot of resources.

There was a clear need for something lighter, faster, and more flexible.

That's where containers came in.

How Containers solved it

Containers packed the application with only what it needed, including the exact Java 17 runtime and all required libraries. So the Product Service behaves the same on any machine, because the container carries its own fixed environment. No server needs Java installed, and version issues disappear completely.

Containers use the operating system of the host machine instead of creating a separate operating system for each application. This makes them much lighter, because they don't need to start a full OS every time. They also start faster and can be moved or copied easily between different machines without breaking.

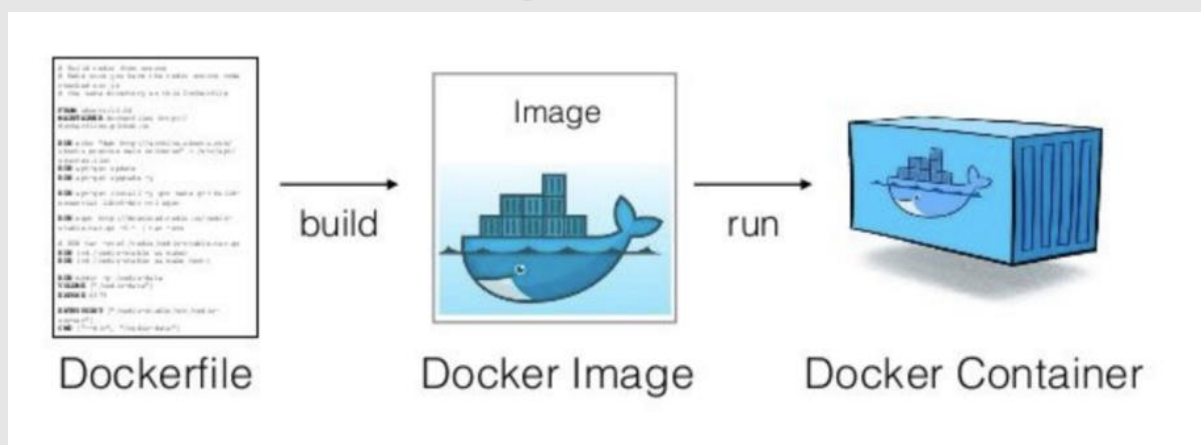
Docker made containers simple to build and run. It gave developers a way to package everything needed for an application into one single image that works everywhere.

Definition: Docker is a tool that makes containerization simple and practical. It helps us build, ship, and run applications inside containers without worrying about what kind of machine or environment we are using.

Think of Docker as a box that carries everything our application needs: code, libraries, and runtime. Once we build that box, we can run it anywhere and it will work exactly the same way.

How Docker Works

1. We create a **Dockerfile**. It is a simple text file with instructions on how to build the container image.
2. Docker reads the Dockerfile and builds an **image**.
3. An image is a blueprint. A **container** is a running copy of that image.
4. We run the container, and the application starts.



Example: Product Service

Let's say we built a Spring Boot Product Service that returns a list of products.

```
@RestController
public class ProductController {
```

```
@GetMapping("/products")
public List<String> getProducts() {
    return List.of("Laptop", "Phone", "Tablet");
}
```

Step-1: We build the jar file: `mvn clean package`. This creates a jar in the target folder. Now we create a Dockerfile to containerize it.

Step-2: Create a Dockerfile

```
# Use OpenJDK 17 as the base image
FROM openjdk:17-jdk-slim

# Copy the jar file from target folder to the container
COPY target/product-service-0.0.1-SNAPSHOT.jar product-service.jar

# Expose the port the application runs on
EXPOSE 8081

# Run the jar file when the container starts
ENTRYPOINT ["java", "-jar", "/product-service.jar"]
```

Step-3: Now we build the Docker image: `docker build -t product-service:1.0`

Step-4: Run the container from the image: `docker run -p 8081:8081 product-service:1.0`

What happens here:

- Docker uses the image as a blueprint
- Starts a new container
- Runs the Product Service on port 8081
- We can access it on `http://localhost:8081/products`

86. **Microservice 85:** Difference of Virtual machine, container and Docker?

A **Virtual Machine (VM)** is like running a full computer inside another computer.

A **Container** is like running just the application inside a shared computer.

Docker is the most popular tool that helps build and run containers.

Virtual Machine

A virtual machine comes with its own full operating system. It has its own kernel and runs on top of a hypervisor.

Think of it like having multiple separate computers inside one big computer.

Example: In an online shopping portal, suppose we want three services, Product, Order, and Payment. With VMs, each service would need its own operating system.

This means three OS installations, more memory use, and slower startup time.

Container

A container shares the same host operating system but runs its own isolated environment. It only carries what the application needs, not the whole OS. This makes it light, fast, and easy to move between machines.

Example: For the same Product, Order, and Payment services, we can run three containers on one OS. No extra OS needed. They start in seconds and use less memory.

Docker

Docker is a platform that helps build, package, and run containers easily. It acts as a manager for containers, making sure everything runs the same way on any machine.

Example: We create a Docker image for each service and run it anywhere, in our local system, testing server, or cloud, without changing the code.

87. Microservice 86: List out some docker commands?

Here is a list of important Docker commands, categorized for clarity:

Image Management:

- `docker pull <image_name>`: Downloads an image from a registry (e.g., Docker Hub).
- `docker build -t <image_name> <path_to_Dockerfile>`: Builds a Docker image from a Dockerfile.
- `docker images`: Lists all local Docker images.
- `docker rmi <image_name_or_id>`: Removes one or more images.
- `docker push <image_name>`: Pushes an image to a registry.

Container Management:

- `docker run <image_name>`: Creates and starts a new container from an image.
- `docker ps`: Lists running containers.
- `docker ps -a`: Lists all containers, including stopped ones.
- `docker start <container_name_or_id>`: Starts a stopped container.
- `docker stop <container_name_or_id>`: Stops a running container.
- `docker restart <container_name_or_id>`: Restarts a container.
- `docker rm <container_name_or_id>`: Removes one or more stopped containers.
- `docker exec -it <container_name_or_id> <command>`: Executes a command inside a running container.
- `docker logs <container_name_or_id>`: Displays the logs of a container.

88. **Microservice 87: What is Kubernetes and how it is useful in Microservices?**

Before Kubernetes came into the picture, managing applications across multiple servers was tough.

Let's imagine we built an online shopping portal with services like Product, Order, and Payment. We containerized each one using Docker. Everything worked great on one or two servers. But as traffic grew, we needed to run dozens, sometimes hundreds, of containers.

That's where the real trouble began.

Before Kubernetes

In the early days, when we ran multiple containers, we had to handle everything manually. We had to:

- Start and stop containers ourselves
- Restart them when they crashed
- Keep track of which server each container was running on
- Distribute traffic evenly among containers
- Update containers without taking the system down

Let's say our Product Service crashed at midnight. Someone had to log in, restart it, and make sure it connected back to the rest of the system. If a new version of the Order Service needed deployment, we had to stop the old one first and start the new one carefully, all manually.

Managing this manually was fine for a few containers but became impossible when dozens of services were involved.

How Kubernetes Changed Everything

Kubernetes was created to solve all these problems. It's an **open-source system** that helps us **deploy, manage, and scale containers automatically**.

Definition:

Kubernetes is an open-source container orchestration platform that automates many of the manual processes involved in deploying, managing, and scaling containerized applications.

Think of it as a smart manager for containers. We tell Kubernetes what our application should look like, how many containers, what images to use, what ports to open, and it takes care of everything.

If a container crashes, Kubernetes restarts it automatically.

If traffic increases, Kubernetes creates new containers to handle the load.

If a node (server) fails, Kubernetes moves containers to healthy machines without downtime.

We no longer worry about where each container runs, Kubernetes handles the placement, scaling, and recovery automatically.

Real World Example

Let's go back to our online shopping portal. We have three main services: Product, Order, and Payment.

- Each service runs inside Docker containers.
- We deploy them using Kubernetes.

When 10,000 users visit the site during a festival sale, Kubernetes notices the traffic spike. It automatically creates more containers of the Product and Order services to handle the load. When traffic drops later, it scales down to save resources.

If one container crashes, Kubernetes replaces it instantly.

If we release a new version of the Payment Service, Kubernetes updates it gradually without stopping the old one. Users never notice the switch.

89. **Microservice 88:** Explain the architecture of Kubernetes?

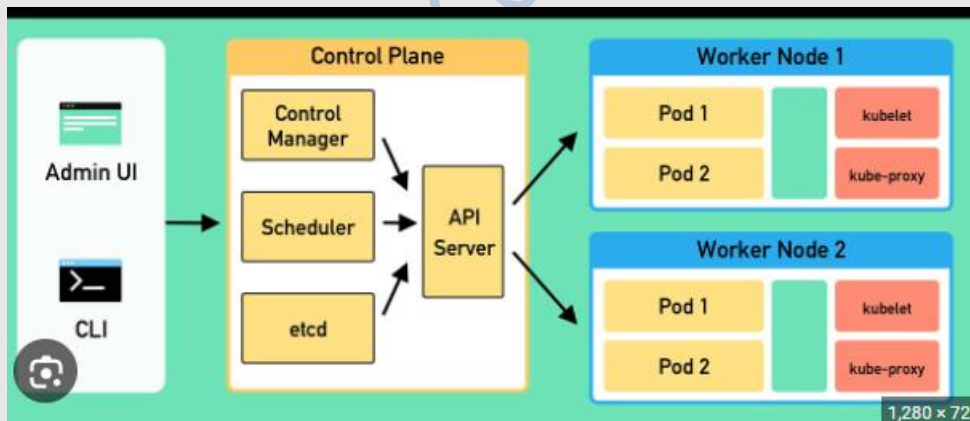
To understand how Kubernetes works, think of it like a team or crew in the airport.

Example: Every plane (a container) must take off, land, refuel, and follow clear instructions. There are people who plan the schedule, people who control flights, and workers who maintain the planes. Kubernetes works exactly like that, but for containers.

Video links: [Kubernetes crash course: In less than 15 minutes](#)
[Kubernetes Architecture in 7 minutes | K8s explained](#)

It has two main parts:

1. **Control Plane** – the brain that makes decisions.
2. **Worker Nodes** – the machines that actually run the containers.



Let's go step by step.

1. Control Plane - The Brain of Kubernetes

The Control Plane manages everything happening in the cluster. It decides where to run containers, monitors them, and reacts if something goes wrong.

It has a few key parts that work together:

a. API Server

The API Server is like the receptionist of the airport. It's the main communication point for everything. Whenever we give instructions (like "run five containers of Product Service"), the API Server receives them and passes them to the right people inside the control plane.

b. etcd

etcd is like the database that stores all important information about the cluster which containers are running, their current state, and the overall configuration.

If we think of Kubernetes as a control tower, etcd is where all flight plans and records are safely kept.

c. Scheduler

The Scheduler decides where to place new containers.

It looks at all worker nodes, checks which have enough resources, and assigns containers accordingly. For example, if Node A is busy but Node B is free, the Scheduler will tell Kubernetes to run new containers on Node B.

d. Controller Manager

The Controller Manager makes sure everything in the cluster stays as it should be.

If a container crashes, it creates a new one.

If we asked for five containers and only four are running, it starts another.

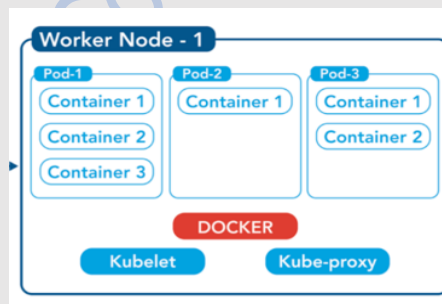
It constantly watches the system and fixes things automatically.

2. Worker Nodes - The Hands That Do the Work

Worker Nodes are the machines where our containers actually run.

Each node has its own small set of tools that help it communicate with the control plane.

Each node has three main components:



a. Kubelet

The Kubelet acts like a local supervisor.

It talks to the control plane and makes sure the containers assigned to that node are running properly.

If a container crashes, the Kubelet reports it and restarts it if needed.

b. Kube Proxy

The Kube Proxy manages communication between containers and the outside world.

It routes network traffic correctly so that services can talk to each other, even if they're running on different nodes.

c. Container Runtime

This is the software that actually runs the containers. Docker used to be the most common runtime. The runtime starts, stops, and manages the lifecycle of containers on the node.

3. Pods -The Smallest Building Block

In Kubernetes, containers don't run directly. They are grouped into small units called **Pods**. A Pod can have one or more containers that share the same network and storage. Pods are like small rooms inside a building; they hold containers that work closely together.

For example, in an online shopping app:

- The Product Service might have its own Pod.
- The Order Service runs in another Pod.
- The Payment Service in yet another.

4. Services - Connecting the Pieces

Services in Kubernetes act like stable doors to access Pods. Even if Pods are replaced or moved, the Service keeps the same name and IP. It ensures smooth communication between different parts of the application.

5. Ingress - The Entry Gate

Ingress is like the main gate for outside users to reach the application. It routes incoming traffic (like from a browser or app) to the correct Service inside the cluster.

90. **Microservice 89:** What are the types of deployments?

When we release a new version of our application, we want to do it safely, without breaking what's already running.

1. Rolling Update Deployment

This is the most common type used in real-world projects. Here, Kubernetes replaces old containers gradually, one by one, while keeping the application running.

Example:

Order Service is running five containers of version 1.0. Kubernetes creates one container of version 2.0, waits until it's ready, then removes one old container. It repeats this process until all are upgraded.

Pros:

- No downtime.
- Easy to rollback if something goes wrong.

Cons:

- Slightly slower than Recreate because of the gradual process.

When to use:

Perfect for production systems where uptime is critical, like online stores or payment system

2. Blue-Green Deployment

In this method, we run two environments side by side, one with the old version (Blue) and one with the new version (Green).

We send all user traffic to the Blue version while testing Green in the background.

Once the new version is tested and ready, Kubernetes switches traffic from Blue to Green instantly.

Example:

In an online shopping portal, version 1.0 (Blue) handles real users.

Version 2.0 (Green) runs quietly in the background.

When we are confident it's stable, Kubernetes routes all users to version 2.0 immediately.

Pros:

- Zero downtime.
- Instant rollback (just switch back to Blue).

Cons:

- Needs double the resources during deployment.

When to use:

Used in large-scale, high-traffic systems like e-commerce or banking apps.

3. Canary Deployment

Canary deployment means testing the new version with a small portion of users before giving it to everyone.

It's like releasing the update slowly to see if it works well.

Example:

Let's say our Payment Service has version 1.0 live.

We deploy version 2.0 but let only 10% of users use it.

If it works fine, we slowly increase it to 25%, then 50%, and finally 100%.

Pros:

- Safe and gradual rollout.
- Problems are detected early without affecting all users.

Cons:

- Takes more time and setup to manage traffic splits.

91. **Microservice 90:** What is the role of continuous integration (CI) and continuous deployment (CD) in Microservices??

Continuous Integration (CI) and Continuous Deployment (CD) play a very important role in microservices because each service is small, independent, and changes often.

To keep all services healthy, we need a smooth way to build, test, and release them without breaking anything.

The Role of CI in Microservices

CI makes sure every change we write is tested quickly and safely.

When a developer updates even one microservice, the CI pipeline automatically:

- pulls the new code
- runs unit tests

- runs integration tests
- builds a fresh service image

If anything fails, CI stops the process and tells us what broke.

Example:

If we change the Payment Service, CI immediately tests only the Payment Service code and its dependent tests.

This helps us catch bugs early before they reach production.

In microservices, CI is very important because each service changes often and must stay independent and stable.

The Role of CD in Microservices

CD takes the next step.

It deploys the successfully tested microservice automatically.

Once the CI tests pass, CD:

- creates a new version of the microservice
- deploys it to staging or production
- updates the service in Kubernetes or any target environment
- verifies that the service is running correctly

Since each microservice is small, deployments are quick and safe.

Example:

If we update only the Notification Service, CD deploys just that service without touching the others. This makes releases faster and reduces downtime.

CI/CD helps us:

- deploy small changes safely
- avoid breaking other services
- catch bugs early
- release updates faster
- keep the system stable

In simple words: **CI tests the change; CD delivers the change.**

92. Microservice 91: Can we use embedded server in microservice deployment for production?

Yes, we can use an embedded server in microservices for production deployment. In fact, most modern microservices use embedded servers by default. So that we need to setup separate sever to run the application.

What is an Embedded Server?

An embedded server means the server runs inside the application itself instead of being installed separately. In Spring Boot, when we build a microservice, it comes with **Tomcat** or **Jetty**

embedded inside the application. So, we do not need to deploy a WAR file on an external server. Instead, we package the application as a JAR and run it directly.

Example:

6.Observability pattern

93. Microservice 92: What observability design pattern?

When we build a system with many microservices, things can go wrong at any layer like network, database, or even inside a single container. Sometimes a request passes through five or six services before reaching its destination. If something fails midway, how do we know where it broke?

That's where the **Observability Design Pattern** helps.

Observability is not just about collecting logs or metrics. It's about understanding **what's happening inside a system** by looking from the outside.

It helps us answer three big questions:

1. Is the system healthy?
2. If something is slow, where is the problem?
3. Why did the problem happen?

Why We Need Observability

Let's imagine we built an online shopping portal.

When a customer places an order, the request moves through:

- API Gateway
- Order Service
- Payment Service
- Inventory Service
- Notification Service

If the order fails, how do we find the real issue?

Was it a database timeout? A slow API call? Or did the payment gateway fail?

Without observability, we would be guessing.

With observability, we can trace the entire path of that request and find exactly where it broke.

The Three Pillars of Observability

Observability mainly depends on three key elements: **Logs, Metrics, and Traces.**

1. Logs

Logs are like diary entries. They record what each service is doing and what went wrong.

Each log contains details such as timestamp, service name, and message.

Example:

In the Payment Service, a log might say:

Payment failed for Order #12345 due to timeout from bank API.

By reading logs, we understand what happened inside each service.

2. Metrics

Metrics are numbers that tell us how the system is performing.

They show CPU usage, memory, request count, and error rates.

Metrics help us see trends, for example, if response times are slowly increasing over time.

Example:

The Order Service might show:

- Average response time: 250ms
- Error rate: 1.2%
- Active containers: 4

These numbers help us decide when to scale up or fix performance issues.

3. Traces

Traces follow a single request as it travels through multiple services. Each part of the request is called a **span**, and all spans together form a **trace**.

Example:

For an order request, a trace might look like this:

1. API Gateway -> 5ms
2. Order Service -> 20ms
3. Payment Service -> 150ms
4. Inventory Service -> 10ms
5. Notification Service -> 15ms

If the payment step took 150ms, we instantly know where the delay happened.

How It Works in Practice

In a real system, we use tools to collect and visualize this data:

- **Prometheus** and **Grafana** for metrics.
- **ELK Stack (Elasticsearch, Logstash, Kibana)** for logs.
- **Spring Cloud Sleuth** or **Zipkin** for distributed tracing.

Reference link: [Distributed Tracing with Spring Cloud Sleuth and Zipkin | by Bubu Tripathy | Medium](#)

Distributed Tracing System: Spring Cloud Sleuth

[\(23\) The Importance of Observability Patterns in Microservices Architecture | LinkedIn](#)

Each service sends its logs, metrics, and traces to these tools.

From a single dashboard, we can view all system activity and pinpoint problems in seconds.

Real World Example

Let's say a customer reports that payments are failing during checkout.

Without observability, we'd check logs manually in each service, hoping to find the error.

With observability in place:

- The trace shows the request was delayed in the Payment Service.
- The logs there reveal the "Bank API timeout."
- Metrics confirm that timeouts increased after 3 PM.

Now we know the issue is not in our code but in an external dependency.

This saves hours of debugging.

Note: You can mention what tools you have used in your project for logging and metrics.

94. **Microservice 93:** What is monitoring in the microservices?

In a monolithic application, everything runs in one place, so debugging is easier. But in microservices, the application is split into many small services. Each service runs separately, has its own database and its own network calls. If something breaks, it is hard to find where the problem started.

Monitoring in microservices means watching what is happening inside our system so that we know if everything is working correctly or if something is going wrong.

Example:

Let us take a simple food delivery system:

- Order Service
- Payment Service
- Delivery Service

When a customer places an order, the request passes through all three services. One day, customers start complaining that orders are failing. We do not know why. Payment might be slow or maybe Delivery Service is down. Without monitoring, we must check each service manually, read logs, and guess the issue. This takes hours.

With monitoring, we can see that:

- Order Service: healthy
- Delivery Service: healthy
- Payment Service: error rate 40 percent, response time high

At this point, we know the problem is inside Payment Service. We open logs and see "Timeout while connecting to bank API". Now we can fix the issue quickly.

Common tools used:

- **Prometheus:** collects metrics
- **Grafana:** shows metrics using graphs
- **Zipkin** or **Jaeger:** trace request flow between services
- **ELK Stack:** reads logs from all services

95. **Microservice 94:** What is centralized logging in the microservices?

Centralized logging means collecting logs from all microservices in one place so we can search and troubleshoot problems easily.

In microservices, each service runs on different servers, so logs are scattered. If a request fails, finding the root cause becomes difficult without a single log view.

Centralized logging solves this by storing all logs together and allowing us to filter them by service, time, or request ID.

Centralized logging tools do not dump logs randomly. They organize them with filters and search options.

We can search logs by:

- Service name
- Time range
- Log level (error, warn, info)
- User ID
- Request ID
- Keyword

Example: If a payment failed for user 1023, we can search like: `service:payment AND userId:1023 AND level:error`

Common tools: ELK Stack (Elasticsearch, Logstash, Kibana), Splunk etc.

96. **Microservice 95:** What is distributed tracing in the microservices?

Distributed tracing is a way to track a single request as it travels through multiple microservices.

In a microservice system, one user request may pass through many services like Order, Payment and Inventory. If the request fails or becomes slow, it is difficult to find where it got stuck. Distributed tracing solves this by giving each request a unique trace ID and following it across services. With this ID, we can see the complete flow, time taken by each service and where the problem started.

Tools used for distributed tracing are Spring Sleuth, Zipkin, Jaeger, OpenTelemetry.

Spring Cloud Sleuth is a library used in microservices to add distributed tracing easily. It automatically generates Trace ID and Span ID for every request and attaches them to logs. This helps us track the complete journey of a request across multiple services.

Sleuth is often used together with Zipkin or Jaeger to visualize the trace. It helps find slow services and failure points quickly.

97. **Microservice 96:** What are the best practices we use for monitoring?

Monitoring in microservices works well only when we follow some best practices.

- First, every service should expose health and metrics endpoints so we can track uptime, errors and response time.
- We should use dashboards to watch important metrics like CPU, memory and latency.
- Alerting rules must be added so teams know when a service is going down before users complain.
- Each request should include a trace ID for tracking.
- Logs, metrics and traces must be connected together to find root causes faster.
- Finally, monitoring should be tested in real conditions, not only after deployment, but also during development and testing.

98. **Microservice 97:** What are tools and techniques we use for monitoring and logging in the microservices?

ELK Stack

The ELK Stack is used for centralized logging. It collects logs from all services, stores them, and allows us to search and analyse them in one place.

It includes **Elasticsearch** for storage, **Logstash** for processing logs and **Kibana** for visualizing them. It helps in debugging and monitoring production systems.

Zipkin

Zipkin is a distributed tracing tool. It shows how a request moves from one microservice to another. It helps find slow services and performance issues.

By using trace IDs, Zipkin shows the complete call flow and time taken by each service, making troubleshooting simple.

Jaeger

Jaeger is another distributed tracing tool similar to Zipkin but built for large systems. It helps track request flow and detect latency issues between services.

It also supports performance tuning and root cause analysis in microservices.

Sleuth

Spring Cloud Sleuth automatically adds tracing information like Trace ID and Span ID to logs.

This helps us track a single request across multiple microservices. Sleuth is often used together with Zipkin or Jaeger.

Prometheus

Prometheus is a monitoring tool that collects metrics from microservices like CPU usage, memory, request count and error rate.

It stores time-series metrics and helps us understand the health and performance of services.

Grafana

Grafana is used to create dashboards for monitoring systems. It connects with Prometheus and shows metrics in graphs and charts.

It helps us watch real system behaviour in real time and take action before failures happen.

Log Correlation ID

A Log Correlation ID is a unique ID assigned to every request. It connects logs from different services related to the same request.

This makes debugging easier because we can track the journey of one request across many microservices by searching a single ID.

Reference: [Microservices Observability - What You Need to Know | Wissen](#)

7. Cross-cutting pattern

99. **Microservice 98:** What are the cross-cutting pattern in Microservices?

When we build a microservice system, each service usually has its own job, like processing payments, managing orders, or handling user details. But apart from these main responsibilities, there are some common tasks that every service needs.

For example, every service needs to handle errors, manage configurations, discover other services, and sometimes log or secure data.

These are not business features but are essential for the system to work correctly.

These shared needs are called **cross-cutting concerns**, and the way we handle them across all services is known as the **Cross-Cutting Concern Design Pattern**.

3 Important things we need to have as part of cross cutting concerns.

1. Error handling
2. Configuration Management
3. Service Discovery

1. Error Handling

In any system, errors are bound to happen like network failures, invalid inputs, or database timeouts. If we don't handle these properly, one small issue can cause a chain reaction of failures.

To make the system reliable, we apply error handling as a cross-cutting concern so all services handle failures in a consistent and graceful way.

Here are the main techniques used:

a. Try-Catch and Fallbacks

Every service wraps risky operations in a try-catch block and defines fallback responses. For example, if the Product Service is down, the Order Service can still accept the order and mark it as “pending” until the product data is available again.

b. Circuit Breaker (Refer Question: Microservice-103)

If one service repeatedly fails, a circuit breaker temporarily stops sending requests to it. Once the failing service recovers, the circuit “closes” automatically. This prevents one weak service from bringing down the whole system. Tools like **Resilience4j** or **Hystrix** are often used here.

c. Retry Mechanism (Refer Question No: Microservice-106)

Some errors are temporary, like a momentary network glitch. In such cases, the service can retry a few times before failing completely. We set retry limits and intervals to keep it balanced.

d. Centralized Exception Handling

In frameworks like **Spring Boot**, we can use `@ControllerAdvice` to handle all exceptions in one place. This ensures every service returns uniform, clear error messages.

Example:

In an online store, if the Payment Service fails during checkout, the Order Service uses a circuit breaker to pause communication. It then logs the failure, shows a friendly error message, and retries later - without crashing the system.

2. Configuration Management

Each service has its own configuration like database details, API URLs, tokens, and more. If these are stored separately inside every service, updating them becomes painful. Imagine changing a database password in 20 different services one by one.

Configuration Management as a cross-cutting concern solves this problem. We keep all configurations in one centralized place, and services read from there automatically at startup.

Example:

Using **Spring Cloud Config Server** (Refer Question No: Microservice-116), we can store configuration files in a Git repository. If we update one file (like changing a URL or key), every service refreshes its configuration automatically without redeployment.

Benefits:

- One place to manage all settings.
- Easy updates without downtime.
- Environment-based configuration (Dev, Test, Production).

3. Service Discovery (Refer Question No: Microservice-71)

In a microservice setup, services are dynamic. They start, stop, scale up, or move to other machines. If one service needs to talk to another, how does it know where it's running?

That's where **Service Discovery** comes in.

It allows services to find each other automatically without hardcoding addresses.

There are two main parts:

- **Service Registry:** Keeps track of all running services and their locations.
- **Service Discovery:** Lets services look up others when needed.

Example:

The Order Service doesn't need to know the Payment Service's exact location.

It simply asks the **Service Discovery Server** (like **Eureka**, **Consul**, or **Zookeeper**) for "Payment Service." The server returns the correct address, and communication happens smoothly.

This avoids manual configuration and helps the system adapt automatically when services scale or move.

100. Microservice 99: What is Resilience or Resilient System?

In the real world, things don't always go as planned. A network might fail for a few seconds, a database might restart, or a third-party service might slow down. In a large system with many microservices, these small problems happen all the time.

A **resilient system** is one that continues to work even when parts of it fail. It doesn't collapse when something breaks. Instead, it absorbs the problem, handles it gracefully, and recovers on its own.

In very simple terms, **Resilience** is the ability of something to withstand or recover quickly from difficulties or challenges. In the context of architecture and design, resilience refers to the ability of systems to adapt and recover from failures or disruptions. It's like having a strong and flexible foundation that can handle unexpected problems and keep going.

Short note: Resilience is not about avoiding failure. It's about **surviving failure**.

A Simple Way to Understand It

Let's imagine we run an **online movie streaming platform**.

The platform has several microservices:

- User Service
- Subscription Service
- Payment Service
- Recommendation Service
- Video Streaming Service

Now imagine the **Recommendation Service** goes down for a few minutes.

In a poorly designed system, this failure might cause the whole platform to stop working.

In a **resilient system**, everything else continues to function.

Users can still log in, play movies, and manage subscriptions. The only thing that fails temporarily is the “Recommended Movies” section.

101. **Microservice 100: How do you achieve resilience?**

We use a few design patterns and techniques to make our systems resilient.

1. Circuit Breaker

Stops calling a service that keeps failing and gives it time to recover.

For example, if the Payment Service fails multiple times, the Order Service stops calling it temporarily.

2. Retry

Tries the same request again if it fails once due to a temporary issue, like a short network drop.

3. Timeout

Limits how long a service waits for another service’s response.

This prevents threads from getting stuck waiting forever.

4. Fallback

Provides a backup response when the main service fails.

For instance, if the Recommendation Service is down, we can show popular movies instead.

5. Bulkhead (Refer Question: Microservice-111)

Separates services into isolated sections so one failure doesn’t overflow into others , just like compartments in a ship.

6. Load Balancing

Distributes requests evenly across service instances so no single one gets overloaded.

7. Health Checks and Monitoring

Continuously watch services to detect failures early and restart them automatically.

Reference: [Resilient Design Principles - GeeksforGeeks](#)

102. **Microservice 101: What is Fault Tolerance?**

Every system faces problems like a server might crash, a network might go down, or a piece of code might fail.

But a good system doesn’t stop working just because one-part breaks. It finds a way to keep going.

That ability to keep running even when something goes wrong is called **fault tolerance**.

Example:

Let’s imagine we are driving a car. If one headlight stops working at night, the car doesn’t shut down. We can still drive using the other headlight until we reach home or a repair shop. That’s fault tolerance means one part fails, but the system continues to function.

Now, think about an online system the same way. If one service or server fails, the others should still work, so users don’t face a complete outage.

Fault tolerance is about designing systems that can handle failure gracefully without total breakdown.

Why It Matters

In a microservice architecture, we usually have many small services talking to each other, like Order, Payment, User, and Notification.

If one service stops responding, and the others can't handle it, the entire system may go down.

Fault tolerance helps us make sure that even if one or two parts fail, the rest of the system keeps running. It keeps users happy and the business running smoothly.

103. **Microservice 102: What is the difference between fault tolerance and Resilience?**

Fault tolerance is about **continuing to work** even when some part of the system fails.

The system is built in such a way that it doesn't stop or crash when one-piece breaks.

Resilience is about **recovering and adapting** when failures happen.

It's not just about surviving a failure, but also bouncing back quickly and continuing to function smoothly afterward.

Example from Daily Life

Let's say we are watching a movie online.

- **Fault tolerance** is when the streaming platform automatically switches to another server if one fails, and we don't even notice any pause.
- **Resilience** is when the stream pauses for a second but then adjusts the quality and continues smoothly instead of stopping completely.

Example from Software Systems

Imagine we have three microservices: **Order**, **Payment**, and **Notification**.

- In a **fault-tolerant system**, if one instance of the Payment Service fails, another identical instance immediately takes over, users never notice the failure.
- In a **resilient system**, if the Payment Service fails, the Circuit Breaker opens, retries are triggered later, and the system automatically recovers without manual help.

Reference: [Making microservices Fault Tolerant and Resilient](#) | by Ritesh Panigrahi | Medium

104. **Microservice 103: What is circuit-breaker pattern?**

In a microservice system, services talk to each other constantly. for example, the **Order Service** calls the **Payment Service**, which might call an external **Bank API**.

Now imagine what happens if the Bank API goes down or becomes very slow.

Every time the Payment Service tries to reach it, it waits... and waits... until it times out.

This delay doesn't just affect one request, it piles up. Soon, all threads are busy waiting, and the whole system slows down.

That's exactly the kind of chain reaction we want to avoid. And that's why we use the **Circuit Breaker Pattern**.

The Idea Behind Circuit Breaker

The Circuit Breaker works like an **electrical fuse** in our home.

When something goes wrong, it “trips” and stops the current to prevent further damage.

Similarly, when a microservice detects too many failed calls to another service, the circuit “opens” meaning no new requests are sent until things recover.

This helps protect the rest of the system from repeated failures and unnecessary waiting.

Definition: The Circuit Breaker pattern is a resilience design pattern in distributed systems that prevents cascading failures by monitoring service calls; if a service fails repeatedly, the breaker “trips,” immediately stopping further calls to it and allowing it to recover, thereby protecting the entire system from overload and improving user experience by failing fast.

How It Works

The Circuit Breaker has three states:

1. Closed

Everything is working normally. All requests go through as usual. The breaker keeps an eye on failures and response times.

2. Open

If failures cross a certain limit, for example, 5 failed requests in a row, the circuit “opens.”

No more calls are made to the failing service.

Instead, the system immediately returns a fallback response or a friendly error.

This saves time and prevents overload.

3. Half-Open

After a short delay, the breaker allows a few test requests to see if the failing service has recovered.

If they succeed, the breaker closes again.

If they fail, it stays open for a while longer.

Example: Online Shopping Portal

Let’s say our **Order Service** calls the **Payment Service** for every new order.

If the Payment Service becomes slow or crashes, the Order Service will keep trying and waiting for responses.

After a few failed attempts, the circuit breaker opens and stops sending requests.

Instead, it immediately returns a message like *“Payment service is temporarily unavailable, please try again later.”*

Meanwhile, the system stays responsive, other features like viewing products or adding to cart still work fine.

After some time, the breaker tries again (half-open state).

If the Payment Service responds successfully, everything goes back to normal.

This way, one failing service doesn’t drag down the entire system.

Real Tools Used

In Java microservices, we often use:

- **Resilience4j**
- **Netflix Hystrix** (older but popular)

These libraries make it easy to define thresholds , for example:

- “If 50% of requests fail within 10 seconds, open the circuit.”
- “Try again after 30 seconds.”

All this happens automatically behind the scenes.

Reference: [Implementing the Circuit Breaker Pattern in Spring Boot: A Comprehensive Guide | Medium](#)

105. **Microservice 104: How do you implement Circuit breaker in Spring?**

We can use **Resilience4j** or **Netflix** hystrix to implement. Resilience4j is the newer and lighter option, while Hystrix is older and no longer actively developed.

1. Implementing Circuit Breaker with Resilience4j

Resilience4j is designed to work perfectly with **Spring Boot**.

It gives us features like retries, rate limiting, and most importantly, circuit breaking, all with simple annotations.

Let's imagine we are building an **Order Service** that calls a **Payment Service**.

If the Payment Service is slow or unavailable, we don't want the Order Service to keep waiting.

We will use a circuit breaker so that after a few failed attempts, it stops calling the Payment Service for a short time and returns a fallback response instead.

Step 1: Add Dependency

In the Order Service, we add this to the pom.xml file:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

Step 2: Enable Circuit Breaker

We enable Resilience4j by adding this annotation in the main application class:

```
@SpringBootApplication
@EnableCircuitBreaker
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Step 3: Add Circuit Breaker to a Method

Now, in our service class, where we call the Payment Service:

```
@Service
public class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public String makePayment() {
        RestTemplate restTemplate = new RestTemplate();
        return restTemplate.getForObject("http://localhost:8082/pay", String.class);
    }

    public String paymentFallback(Exception ex) {
        return "Payment service is currently unavailable. Please try again later.";
    }
}
```

Here's what happens:

- When the call to the Payment Service fails multiple times, the circuit breaker “opens”.
- New requests are blocked for a while.
- The fallback method runs immediately, giving a quick response.
- After some time, the circuit “half-opens” and tests the Payment Service again.
- If it responds successfully, the circuit closes.

Step 4: Configure Settings

We can adjust how the circuit behaves in application.yml:

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        failure-rate-threshold: 50
        sliding-window-size: 10
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
```

This means:

- If 50% of the last 10 calls fail, open the circuit.
- Keep it open for 10 seconds before trying again.
- Allow 3 trial calls in the half-open state.

Reference link: [Micro Service Patterns: Circuit Breaker with Spring Boot | by Salitha Chathuranga | Medium](#)

Code: [SalithaUCSC/spring-boot-circuit-breaker: Spring Boot Circuit Breaker](#)

The above link one of the best implementations I found. Download the code from GitHub, Run and follow the steps to understand how it works.

mvn clean install before running the project.

2. Implementing Circuit Breaker with Netflix Hystrix (Older Option)

Before Resilience4j, Hystrix was the most popular circuit breaker library. It works similarly but uses a different annotation:

```
@Service
public class PaymentClient {

    @HystrixCommand(fallbackMethod = "paymentFallback")
    public String makePayment() {
        RestTemplate restTemplate = new RestTemplate();
        return restTemplate.getForObject("http://localhost:8082/pay", String.class);
    }

    public String paymentFallback() {
        return "Payment service is down. Please try again later.";
    }
}
```

To use Hystrix, we add this dependency:

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

And enable it in the main class:

```
@EnableCircuitBreaker
@SpringBootApplication
public class OrderServiceApplication { }
```

Hystrix is still found in some legacy projects but has been replaced by Resilience4j because it's faster, lighter, and works well with Java 17 and Spring Boot 3.

106. **Microservice 105: When to use Circuit breaker in Spring?**

In a microservice world, every service depends on others. One small failure can spread quickly if we don't manage it well. That's where the **Circuit Breaker pattern** becomes useful. It helps us stop calling a service that is already failing and gives it time to recover.

Let's understand when and why we actually need to use it.

1. When One Service Depends on Another

Suppose we have an **Order Service** that calls a **Payment Service** to complete every order. If the Payment Service becomes slow or goes down, the Order Service will keep waiting for a response. Each request waits until it times out, using up memory and threads.

Over time, this delay can affect all users and may even crash the Order Service.

By using a Circuit Breaker, we can detect repeated failures and temporarily stop sending requests to the Payment Service.

Instead, we can return a quick fallback response like:

"Payment service is currently unavailable, please try again later."

This protects the Order Service from unnecessary waiting and keeps the rest of the system healthy.

2. When External APIs or Third-Party Services Are Involved

Sometimes our microservices depend on **external APIs**, for example, a bank API for payments, a shipping API for deliveries, or a currency conversion API for prices.

If these APIs fail or respond too slowly, our system shouldn't suffer. A Circuit Breaker helps us handle such unstable or unpredictable services gracefully.

Example:

In a travel booking app, the **Flight Service** calls an external airline API. If that API goes down, we open the circuit and stop calling it for a while.

During this time, we show users a message like *"Flight details are being updated, please check again shortly."*

The system stays responsive even though one external service failed.

3. When Performance Starts to Degrade

Even if a service is technically "up," it might be too slow to handle the load.

A Circuit Breaker can watch for performance issues. If calls start taking longer than expected or exceed a timeout limit, it opens the circuit.

That way, instead of waiting on a slow service, our system quickly returns a fallback response. It's better to fail fast than to slow down everything.

4. When We Have Limited Resources

In real systems, we have limited threads and memory.

If all threads are waiting for a response from a failing service, no new requests can be processed. The system becomes unresponsive even though it hasn't crashed.

Circuit Breakers prevent this by stopping calls to failing services early.

This ensures that healthy services and new users continue to work smoothly.

5. When We Need to Improve System Resilience

In distributed systems, failure is normal. Networks go down, APIs crash, databases restart. A resilient system accepts that things will fail and plans for recovery instead of collapse.

The Circuit Breaker pattern helps us build such resilience. It detects repeated failures, stops making risky calls, and retries automatically after a short break.

This simple behaviour can prevent one small issue from turning into a system-wide outage.

107. **Microservice 106: What is retry mechanism and how you achieve it?**

Sometimes, we call an external service or a database, and it fails, maybe because of a network glitch, a timeout. In such cases, instead of immediately throwing an error, we might want to try again a few times before giving up.

Retry Mechanism is a simple technique used when a request or operation fails due to a temporary issue, like a network glitch or timeout. Instead of giving up immediately, the system automatically tries the same request again after a short delay, improving reliability and avoiding unnecessary failures.

We achieve it using Spring-Retry. It gives us a neat way to retry failed operations automatically without writing boilerplate loops.

How it Works

When we put `@Retryable` on a method:

1. Spring wraps that method in a proxy.
2. If the method throws an exception that we have marked for retry, Spring will call the method again.
3. It repeats this until either:
 - The method succeeds
 - The maximum retry attempts are exhausted
4. If retries are exhausted, we can handle the failure using `@Recover`.

Example:

Step 1: Add Dependency

```
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Step 2: Enable Retry

```
@SpringBootApplication
@EnableRetry
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Step 3: Use `@Retryable`

```
@Service
public class PaymentService {

    private int counter = 1;

    @Retryable(
        value = RuntimeException.class, // Which exception to retry
        maxAttempts = 3,                // How many times
        backoff = @Backoff(delay = 2000) // Wait 2s between retries
    )
```

```

    )
    public void makePayment() {
        System.out.println("Trying payment... attempt " + counter);
        counter++;
        throw new RuntimeException("Payment failed!");
    }

    @Recover
    public void recover(RuntimeException e) {
        System.out.println("All retries exhausted. Payment failed permanently: " +
e.getMessage());
    }
}

```

Key Points to Remember

- @Retryable works only on Spring-managed beans.
- It needs Spring AOP (that is why we added spring-boot-starter-aop).
- Use @Recover to define a fallback method after retries fail.
- We can control retries with:
 - maxAttempts means how many times
 - backoff means delay, multiplier (exponential backoff)
 - value to which exceptions trigger retry

Reference: [Implementing Method-Level Retry in Spring Boot | Medium](#)

108. **Microservice 107: What is Rate limiting and how do you implement in Microservices?**

In a microservice system, hundreds or even thousands of requests can reach a service every second. Some may come from genuine users, and some may come from automated scripts or even by mistake. If too many requests hit a service at once, it can become slow or even crash.

Rate Limiting helps us prevent that. It is a simple idea, we control **how many requests** a user, client, or system can make within a certain period of time.

Real-World Example

Let's say we run an online movie ticket platform. We have a Booking Service that handles seat reservations.

During a big movie release, thousands of users open the app and start booking tickets at the same time.

If every request goes to the Booking Service instantly, the database will get flooded, causes slow responses or even system crashes.

With Rate Limiting, we can control how many booking requests each user can make per second or minute. For example:

- Each user can send 5 booking requests per minute.
- If they exceed that, we show: *"You're trying too fast. Please wait a moment."*

This keeps the system safe and ensures fair access for all users.

How Rate Limiting Works

Rate limiting can be done at different levels:

1. **At API Gateway:** most common approach.
The gateway sits in front of all microservices and decides how many requests each client can make.
2. **At Service Level:** each microservice can implement its own limit for specific endpoints.
3. **At Network Level:** tools like Nginx or cloud load balancers can limit incoming requests globally.

109. **Microservice 108:** How can you implement Rate Limiter in Spring?

There are a few popular ways to add rate limiting in microservices:

1. Spring Cloud RateLimiter (Using API Gateway)

Most API Gateways like **Spring Cloud Gateway**, **Kong**, or **AWS API Gateway** support rate limiting out of the box.

Example in **Spring Cloud Gateway** (YAML configuration):

```
spring:
  cloud:
    gateway:
      routes:
        - id: booking-service
          uri: http://localhost:8081
          predicates:
            - Path=/bookings/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5 # requests per second
                redis-rate-limiter.burstCapacity: 10 # max burst capacity
                key-resolver: "#{@userKeyResolver}"
```

Here:

- Each user can make 5 requests per second.
- Burst capacity allows a short spike of up to 10.
- Redis keeps track of counts for each user across instances.

When to Use Spring Cloud RateLimiter

- When we already use Spring Cloud Gateway in front of microservices.
- Best for centralized rate limiting , all limits are enforced at the entry point.

2. Bucket4j

Bucket4j is a pure Java library for token-bucket-based rate limiting. It doesn't depend on Spring or any specific framework, so we can use it anywhere: inside microservices, filters, gateways, or REST controllers.

```
import io.github.bucket4j.*;
```

```

Bucket bucket = Bucket4j.builder()
    .addLimit(Bandwidth.classic(10, Refill.greedy(10, Duration.ofMinutes(1))))
    .build();

if (bucket.tryConsume(1)) {
    // process request
} else {
    // return "Too Many Requests"
}

```

When to Use Bucket4j

- When we want in-memory or distributed rate limiting logic inside our service.
- Works great for Spring Boot REST controllers or filters.

Reference: [Understanding Bucket4J use cases. Bucket4j is a Java rate-limiting... | by Arvind Kumar | Medium](#)

3. Resilience4j RateLimiter:

Resilience4j is a fault tolerance library, best known for circuit breakers and retries. It also provides a RateLimiter module, which works slightly differently from Bucket4j.

Instead of tracking “tokens,” Resilience4j RateLimiter uses permits per time window, meaning it controls how many calls can be made within a time frame.

```

import io.github.resilience4j.ratelimiter.*;
import java.time.Duration;

RateLimiterConfig config = RateLimiterConfig.custom()
    .limitForPeriod(5)
    .limitRefreshPeriod(Duration.ofSeconds(10))
    .timeoutDuration(Duration.ofSeconds(1))
    .build();

RateLimiter rateLimiter = RateLimiter.of("serviceA", config);

Runnable limitedCall = RateLimiter.decorateRunnable(rateLimiter, () -> {
    System.out.println("Processing request...");
});

limitedCall.run();

```

When to Use Resilience4j RateLimiter

- When we already use Resilience4j for Circuit Breaker, Retry, or Bulkhead, and want to keep consistency.
- When we want fine-grained control at method level using annotations.

Video link for Sample: [API Rate Limiter - Resilience4J + Spring Boot Implementation Example](#)

110. Microservice 109: What are the Rate limiting algorithms?**Reference video links:**

At a high level: [\(1\) Rate Limiter and its types #systemdesign #softwareengineer - YouTube](#)

More in depth with good graphics: [Five Rate Limiting Algorithms ~ Key Concepts in System Design](#)

Let's look at the main algorithms one by one and understand how each one works, along with the tools that use them in real projects.

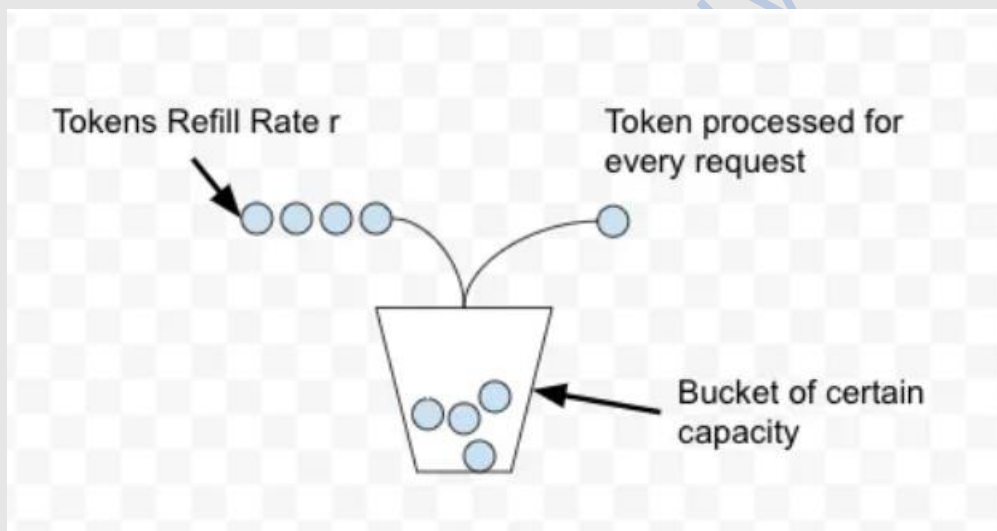
1. Token Bucket Algorithm

This is one of the most popular and practical algorithms used for rate limiting.

- Imagine a bucket that slowly fills up with tokens.
- Each request needs one token to pass through.
- If the bucket has tokens, the request is allowed.
- If it's empty, the request has to wait until more tokens are added.

Tokens are refilled at a steady rate, for example, 10 tokens every second.

This means we can allow short bursts of traffic without crashing the system, but still maintain an overall limit.

**Example:**

If we allow 10 requests per second, and a user sends 15 requests, the first 10 will pass, and the rest will be blocked or delayed until more tokens are available.

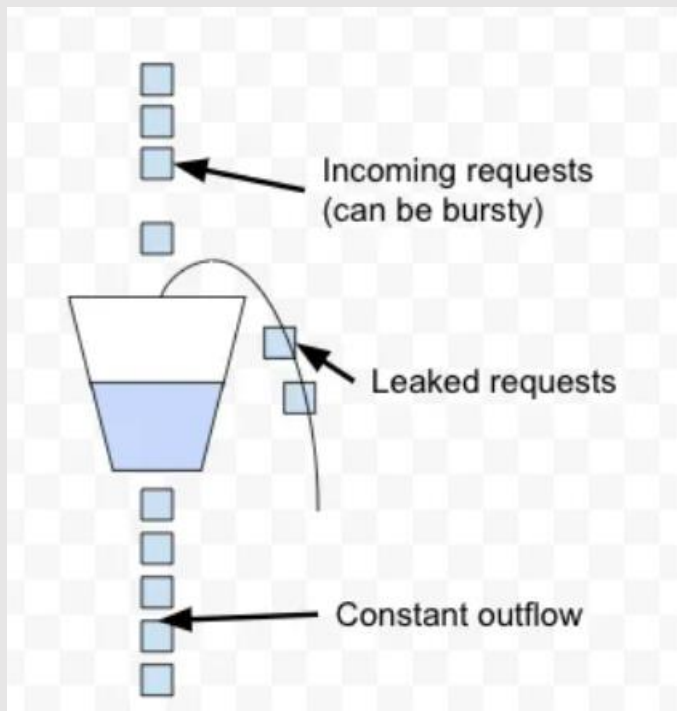
2. Leaky Bucket Algorithm

This is one of the oldest and simplest methods.

Imagine a bucket with a small hole at the bottom.

Requests enter the bucket like drops of water, but they leave at a fixed rate through the hole.

If too many requests come at once, the bucket overflows, and extra requests are dropped.



3. Fixed Window Counter

Time is divided into fixed windows, like every 1 minute or every 10 seconds.

We count how many requests arrive in that time.

If the count crosses the limit, new requests are blocked until the next window starts.

Good for: simple use cases, internal systems, or low traffic.

4. Sliding Window

This one is smarter than the fixed window. Instead of counting requests in strict one-minute boxes, it looks at the last few seconds continuously.

It checks how many requests arrived in the last “window” (say, last 10 seconds) and makes sure the total stays under the limit.

111. **Microservice 110:** Explain about Token bucket algorithm

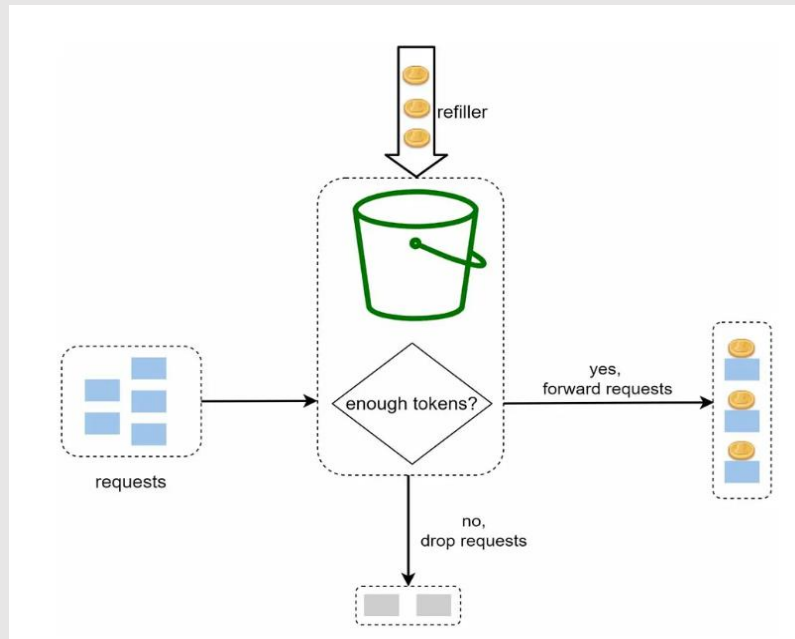
The bucket has tokens with a pre-defined capacity. When a request comes in it takes a token from the bucket and is processed further. If there is no token to be picked, the request is dropped and the user will have to retry.

The payment gateway, Stripe ([read here](#)) uses a token bucket algorithm to throttle their API requests.

From Stripe documentation:

We use the **token bucket algorithm to do rate limiting**. This algorithm has a centralized bucket host where you take tokens on each request, and slowly drip more tokens into the bucket. If the bucket

is empty, reject the request. In our case, **every Stripe user has a bucket, and every time they make a request, we remove a token from that bucket.** We implement our rate limiters using Redis.



Example use-case:

- We have our rate-limiting rules set to 3 requests per user per minute.
- User1 makes the request at 0th time interval, currently the bucket is full with 3 tokens so request will be processed. Tokens will be updated in the bucket to 2 now.
- User1 makes a 2nd request at 10th sec, bucket has 2 tokens so again the request will be processed further. Tokens will be updated in the bucket to 1 now.
- User1 makes a 3rd request at say 30th sec, bucket has 1 token so again the request will be processed further. Now the bucket is empty till the whole 1 minute completes.
- User1 makes a 4th request at say 55th sec, the bucket has 0 tokens so the request is throttled and the user is returned with a 429 status code - too many requests and asked to retry it later in some time. The HTTP 429 Too Many Requests response status code **indicates the user has sent too many requests in a given amount of time.**
- At the completion of that 1 minute, the token count is refreshed at a fixed rate and the bucket is again full with 3 tokens and requests can be processed again for that particular user.

In simple words:

In the Token Bucket algorithm, we process a token from the bucket for every request. New tokens are added to the bucket with rate " r ". The bucket can hold a maximum of b tokens. If a request comes and the bucket is full it is discarded.

The token bucket algorithm takes two parameters:

- Bucket size: the maximum number of tokens allowed in the bucket

- Refill rate: number of tokens put into the bucket every second

Sample Code:

```
public class TokenBucket {

    private static final double MAX_BUCKET_SIZE = 3.0; // Max tokens the bucket can hold
    private static final double REFILL_RATE = 1.0;      // Tokens added per second

    private double currentBucketSize;
    private long lastRefillTimestamp; // in nanoseconds

    public TokenBucket() {
        this.currentBucketSize = MAX_BUCKET_SIZE;
        this.lastRefillTimestamp = System.nanoTime();
    }

    // Method to check if a request can be allowed
    public synchronized boolean allowRequest(double tokens) {
        refill(); // refill tokens before checking

        if (currentBucketSize >= tokens) {
            currentBucketSize -= tokens;
            return true; // request allowed
        }

        return false; // not enough tokens, request dropped
    }

    // Refill tokens based on elapsed time
    private void refill() {
        long now = System.currentTimeMillis();
        long timePassed = now - lastRefillTimestamp;

        long tokensToAdd = (timePassed * refillRate) / 1000; // refillRate is tokens per
second
        currentBucketSize = Math.min(MAX_BUCKET_SIZE, currentBucketSize + tokensToAdd);
        lastRefillTimestamp = now;
    }

    public static void main(String[] args) {
        TokenBucket bucket = new TokenBucket();

        System.out.println("Request processed: " + bucket.allowRequest(1)); // true
        System.out.println("Request processed: " + bucket.allowRequest(1)); // true
        System.out.println("Request processed: " + bucket.allowRequest(1)); // true
        System.out.println("Request processed: " + bucket.allowRequest(1)); // false
        (bucket empty)
    }
}
```

Explanation:

1. The bucket starts with **3 tokens**, and the current time is stored.
2. Every request calls `allowRequest()`, which first triggers a `refill()`.

3. `refill()` checks how much time passed since the last refill.
4. Based on that time, new tokens are added at **1 token per second**, but never above 3.
5. After refilling, the method checks whether the bucket has enough tokens for the request.
6. If enough tokens exist, they are deducted and the request is allowed.
7. If not enough tokens exist, the request is rejected and must wait for tokens to refill.

112. Microservice 111: What is the Bulkhead pattern, and how does it prevent system-wide failures?

The **Bulkhead pattern** is a simple but powerful idea used in software systems to prevent one failure from spreading and breaking everything. The concept comes from ships. In a ship, the lower part is divided into separate sections called bulkheads. If one section gets damaged and fills with water, the other sections are still safe, and the ship does not sink. The damage won't spread.

In the same way, we apply the bulkhead idea in microservices and backend systems. We divide the system into separate parts so that if one part becomes slow, overloaded or fails, the rest of the system continues to work.

Why do we need the Bulkhead pattern?

In real systems, we often depend on many external services like payment, authentication, notifications, search and so on. If all of them share the same resources such as thread pools, memory or connection pools, then one slow service can block everything. The entire system slows down and becomes unhealthy even though only one part had a problem. This is called a cascading failure.

The Bulkhead pattern stops this from happening.

How it works

We separate resources by feature or by dependency. Each part of the system gets its own safe space. For example, we can give each service its own thread pool or limit the number of calls a service can consume. So, if one part fails, its failure stays in that part only. The rest of the system remains healthy.

Real world example

Imagine we are building an online shopping application. When a customer visits the home page, three services are called

- Product Service to show product listings
- Recommendation Service to show suggested items
- Review Service to show product ratings

Now let us say the Review Service becomes very slow due to heavy traffic. If all three services share the same thread pool, the slow Review Service will occupy many threads and stop Product and Recommendation services from responding. The whole website will feel slow or even timeout.

If we use the Bulkhead pattern

- Product Service gets ten threads
- Recommendation Service gets five threads
- Review Service gets three threads

Now even if Review Service is slow, it only affects its three threads. Product pages still load fast and recommendations still work. The customer experience stays smooth.

Reference: [Bulkhead Pattern with Spring Boot | Vinsguru](#)

113. **Microservice 112: How to implement bulkhead pattern using Spring?**

Take example of E-commerce application. The Order Service needs to call three other services

- Payment Service
- Inventory Service
- Notification Service

Sometimes, the Payment Service becomes slow because of a bank issue. If we are not careful, this delay will block the Order Service and it will stop accepting new orders. This is bad for business.

With the **Bulkhead pattern**, we can protect our Order Service by giving each dependency its own separate space. If the Payment Service is slow, it will only affect its own space and not the whole system.

Tools we use

- **Spring Boot** for building the service
- **Resilience4j** for applying the bulkhead pattern
- **Thread pool isolation** to protect resources

Step 1: Add dependency

We need Resilience4j in our application. In Maven, we add:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

OR

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

Step 2: Protect Payment Service calls

We add a **bulkhead** around the Payment Service call. This will limit how many threads are used for that call so it cannot block total system resources.

```
@Service
public class OrderService {

    @Bulkhead(name = "paymentServiceBulkhead", type = Bulkhead.Type.THREADPOOL)
    public String processPayment(String orderId) {
        return paymentClient.makePayment(orderId);
    }
}
```

The method now has a safety boundary. If the Payment Service becomes slow, it will not block all threads.

Step 3: Configure bulkhead limits

In application.yml, we configure limits for each service. Here we control how many simultaneous calls we allow.

```
resilience4j.bulkhead:
  instances:
    paymentServiceBulkhead:
      maxConcurrentCalls: 5
      maxWaitDuration: 2s
```

This means:

- Only 5 payment calls are allowed at the same time
- Others must wait a maximum of 2 seconds

If more calls come in, they will be rejected quickly instead of blocking everything. The Order Service stays responsive.

Step 4: Optional fallback for better user experience

If the Payment Service is overloaded, we can add a fallback.

```
@Bulkhead(name = "paymentServiceBulkhead", fallbackMethod = "paymentFallback")
public String processPayment(String orderId) {
    return paymentClient.makePayment(orderId);
}

public String paymentFallback(String orderId, Throwable t) {
    return "Payment service is busy, please try again in a moment";
}
```

This allows us to return a friendly response instead of an error.

Reference: [Bulkhead pattern in Microservices: Spring Boot example | by S Sivaraman | Medium](#)

114. **Microservice 113:** What is difference between Rate-limiting and bulkhead pattern?

What is Rate Limiting?

Rate limiting controls how many requests are allowed to enter a system in a given period of time. It is like a security guard at the entrance of a building who allows only a fixed number of people inside every minute. If too many people try to enter at once, new people are stopped at the gate.

Why do we use it?

To protect a service from too many incoming requests. It prevents abuse, sudden traffic spikes and denial of service situations.

Real world example

Imagine a ticket booking website during a cricket match sale.

One user keeps clicking the **Book Now** button very fast or refreshes the page again and again. This sends many requests in a short time. If this continues, it can overload the system.

With rate limiting for a single user, only a fixed number of requests from that user are allowed per second. When the user crosses that limit, further requests are blocked.

The user then sees a polite message like “Too many requests, please try again in a few seconds.”

Rate limiting protects the system by **controlling the entrance**.

What is the Bulkhead Pattern?

The Bulkhead pattern controls **how many resources one part of the system can consume**. It isolates parts of the system so a problem in one part does not break the others.

Why do we use it?

To stop a failure from spreading across services. It prevents one slow or failing dependency from blocking the whole system.

Real world example

In an online store, the Order Service calls three other services

- Payment Service
- Inventory Service
- Recommendation Service

If the Payment Service becomes slow, and there is no bulkhead protection, requests waiting for Payment can consume all available threads in the Order Service. This can block calls to Inventory and Recommendation as well, even though they are healthy.

With bulkhead protection, we give the Payment Service its own fixed number of threads or connections. If Payment becomes slow, only its allocated threads are affected. Calls to Inventory and Recommendation continue to work normally.

115. Microservice 114: How do you handle failures in Microservices?

When one service depends on another, things can go wrong. The network may fail, the service may be overloaded, or a bug may cause bad responses. To keep the system healthy, we use a few simple techniques.

1. Retry with Limits

Sometimes a service fails only for a moment. We try the request again after a small delay. But we retry only a few times so we don't overload the service.

Example: If the Payment Service times out once, the Order Service retries twice before giving up.

2. Timeout

We never wait forever for another service. If a service takes too long, we stop the request and handle it gracefully.

Example: If the Inventory Service doesn't reply in 2 seconds, we show a message like "Unable to check stock now."

3. Circuit Breaker

If a service keeps failing, we temporarily stop calling it.
This protects the system from making the situation worse.
Once the service recovers, the circuit closes and calls resume.

Example: If User Service fails 10 times in a row, Order Service pauses all calls for a short time.

4. Fallback Response

When the main service is down, we return a safe backup response.

Example: If Review Service is not responding, the Product Page can still load without reviews instead of showing an error.

5. Bulkhead Pattern

We separate services so one failing service doesn't pull down others.
It's like having separate rooms in a ship so a leak in one room doesn't sink the ship.

Example: High load on Chat Service should not affect Payment Service.

116. **Microservice 115:** Explain about security in microservices?

Note: In-depth security related questions along with scenarios and implementations (**30+ questions on Spring security**) are covered in Java & Spring knowledge boost

Link: <https://topmate.io/codinglyf/1746709>

Microservices communicate over networks, which increases security risk. Each service exposes APIs that can be attacked. Managing authentication across services, securing internal communication, handling service-to-service communication and avoiding token misuse become difficult as the number of services grows.

OAuth2 in Microservices

OAuth2 allows secure authorization between services. Instead of sharing passwords, a user gives permission using an access token. Microservices use this token to verify requests. For example, a user can log in through Google and access different services without sharing their login details.

JWT Authentication

JWT (JSON Web Token) is used for authentication in microservices. After login, a token is generated and sent to the client. The client includes this token in every request. Each service can validate the token and identify the user without checking a central database every time.

Role-Based Access Control

Role-Based Access Control (RBAC) ensures that users can only access what they are allowed to. Users are assigned roles like admin, user or manager. Each role has limits on what actions can be done. This avoids unauthorized access to sensitive parts of the system.

API Gateway Security

The API Gateway is the entry point for all clients, so it must validate tokens, limit requests, block invalid users and prevent attacks. It acts like a security guard that checks identity and permissions before allowing access to internal microservices.

Token Validation

In microservices, every request must be trusted. Token validation makes sure that the JWT or OAuth token is real, not expired and not changed. Each service verifies the token before serving the request. This protects the system from fake or stolen tokens.

SSL/TLS Usage

SSL/TLS secures communication between microservices by encrypting data. Without encryption, data can be read or modified during transfer. Using HTTPS ensures that passwords, tokens and sensitive data remain safe while moving between users and services.

117. **Microservice 116:** How do you manage the configurations in distributed systems?

Normally, Spring Boot reads its configuration from `application.properties` or `application.yml` files at startup. So, if we change them, we need to restart the app

To solve this, Spring provides ways to apply globally and refresh configurations without restarting.

To do this we need to follow these things:

1. Store configurations centrally in a separate Git repo.
2. Set up a Spring Cloud Config Server
3. Connect your main applications to Config Server
4. Enable actuator > refresh API

1. Store configurations centrally in a separate Git repo.

Create a Git repository (say `config-repo`). Inside it, place environment-specific property files:

```
application-dev.properties
application-test.properties
application-prod.properties
```

2. Set up a Spring Cloud Config Server

Create a new Spring Boot app with `@EnableConfigServer` and add

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

Point it to the Git repo (config-repo) in application.properties:

```
spring.cloud.config.server.git.uri=https://github.com/org/config-repo
```

```
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServer {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServer.class, args);
    }
}
```

@EnableConfigServer. This annotation tells Spring Boot to enable the Config Server functionality for the application.

3. Connect your main applications (all the services) to Config Server

- Add `spring-cloud-starter-config` dependency
- To connect the application to config server, we need to give below property pointing to the config server URL.

```
spring.cloud.config.uri=http://config-server:8888
spring.profiles.active=prod
```

4. Enable actuator > refresh API:

- Add `spring-boot-starter-actuator` dependency.
- Annotate beans that should reload on config change with `@RefreshScope`. Example:

```
@RestController
@RefreshScope
public class ConfigController {
    @Value("${featureX.enabled}")
    private boolean featureEnabled;

    @GetMapping("/feature-status")
    public String featureStatus() {
        return "Feature X is " + (featureEnabled ? "ENABLED" : "DISABLED");
    }
}
```

```
}  
}
```

The **@RefreshScope** annotation in Spring Cloud is used to enable dynamic refreshing of configuration properties within a Spring Boot application without requiring a full application restart.

How it works?

- Suppose you want to disable the feature in prod.
- Edit application-prod.properties in config-repo branch: `featureX.enabled=false`
- Commit and push changes to Git.
- Go to main running app.
- Call the actuator refresh endpoint on your service (main app):
- POST `http://main-app:8080/actuator/refresh`
- Spring reloads properties from Config Server into your beans without reboot.

Reference: [Complete Guide to Spring Cloud Config | Medium](#)

8. Cache

118. **Microservice 117: What is Local-Cache and Distributed-Cache?**

Caching is one of the simplest ways to speed up a system. Instead of fetching the same data again and again from a slow source like a database, we keep a copy of it in a faster place.

Now the question is, where should this fast copy live? Inside the service itself or outside as a shared system?

That is where local cache and distributed cache come in.

Local Cache

A local cache lives inside the application instance. Each service keeps its own copy of the cached data in its memory.

For example, we run three instances of the same microservice. Each instance has its own small storage inside its memory.

Benefits

- Very fast, because the data is in the service memory
- Simple to set up with libraries like **Caffeine** or **Ehcache**

Limitations

- Each instance holds its own copy, so there is a chance that data may not match across instances
- Memory usage increases as instances grow
- Not suitable for shared or large data

Distributed Cache

A distributed cache lives outside the service. All service instances connect to the same cache system. **Redis** and **Hazelcast** are popular choices.

Here the cache works more like a shared storage. Every instance reads and writes to the same place.

Benefits

- Data stays consistent across all service instances
- Can store large data without affecting service memory
- Easy to scale when traffic grows

Limitations

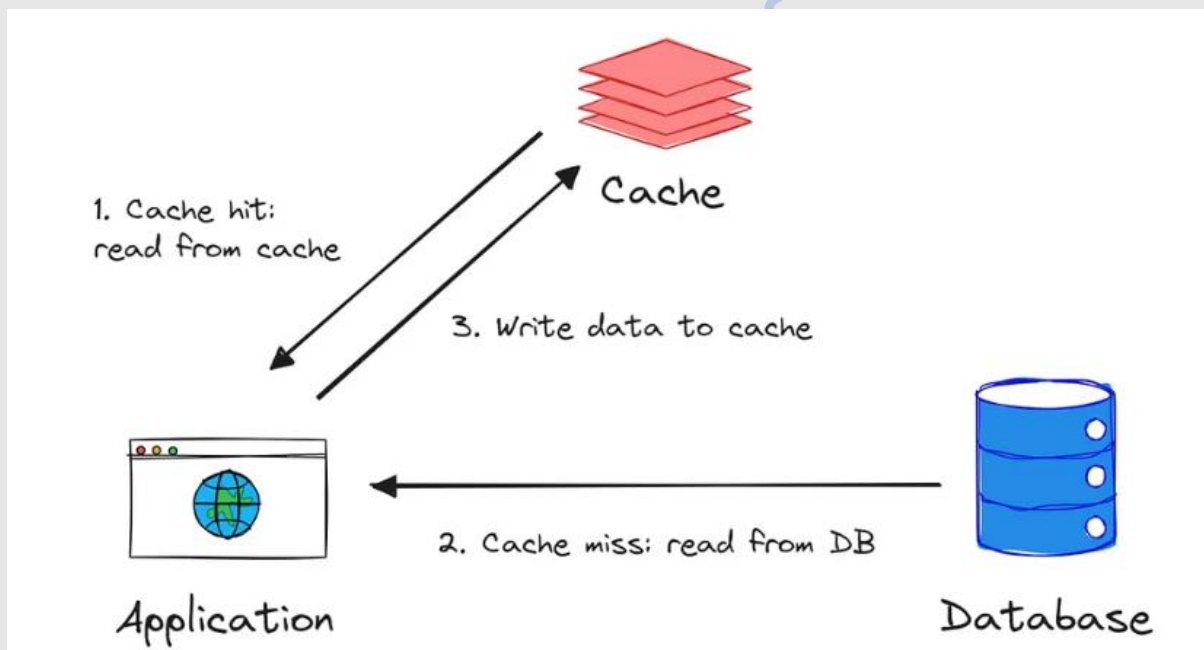
- Slightly slower than local cache because it needs a network call
- If the cache server goes down, the system may slow down

Example

When we store customer session data or user login tokens, every service instance must see the same value. A distributed cache like Redis fits this case well.

119. Microservice 118: What is In-memory caching and give any example?

In-memory caching is a way of storing data in the system's RAM instead of fetching it from a database or external source every time it's needed.



Since RAM is incredibly fast compared to disk storage, you can access cached data almost instantly. This approach is perfect for information that doesn't change very often, like API responses, user profiles, or rendered HTML pages.

Example: Redis

Redis is an open-source, in-memory data store that developers use to cache and manage data in real time.

Unlike traditional databases, Redis stores everything in memory, which makes data retrieval incredibly fast. But Redis isn't just a simple key-value store. It offers a wide range

Follow up:**So, what is *not* in-memory?**

Any cache or storage that keeps data on disk instead of RAM.

1. Traditional databases: MySQL, Postgres, Oracle These store data on disk. Reads and writes are slower.
2. File-based storage: Data stored directly as files
3. Disk-based caches: Some systems keep cache on disk when memory is not enough.

Example:

Ehcache has a mode where it stores extra data on disk. This avoids memory overflow but is slower than RAM.

In-memory cache:

Redis, Memcached, Caffeine, Hazelcast (in-memory mode)

Not in-memory (Disk cache):

MySQL, Postgres, disk-based Ehcache, file storage, object storage

120. Microservice 119: How do you achieve caching in distributed systems?

In a distributed system, we usually deal with a lot of requests, large databases, and many services talking to each other. Every service might need to fetch the same kind of data repeatedly, for example, product details, user profiles, or recent transactions.

Fetching the same data again and again from the database slows everything down. That's where caching helps.

Caching means storing frequently used data in memory so that the next time it's needed, we can get it quickly without going all the way to the database.

Why Caching Is Important

Without caching, every request goes straight to the database. As requests increase, the database becomes overloaded, responses slow down, and eventually, the system struggles to keep up.

With caching, we store recently or frequently used data in a faster storage (like memory). When the same request comes again, we serve it directly from cache

How We Implement It**1. Choose a caching tool**

The most popular tools are:

- **Redis and Hazelcast:** These are distributed caches. They run outside the application and can be shared by all service instances. If one instance writes something to Redis or Hazelcast, every other instance can read it
- **Ehcache and Caffeine:** These are in-memory caches inside the application JVM. They store data locally in the same process where the code is running. This makes them extremely fast, but the data stays only inside that instance

2. Store and Retrieve Data

When data is fetched from the database:

- First check if it's available in the cache.
- If found, return it immediately.
- If not, fetch it from the database, store it in the cache, and then return it.

3. Set Expiration (TTL)

Cached data should expire after some time to stay updated.

For example, we might cache course details for 10 minutes.

After that, the next request fetches fresh data from the database and updates the cache.

121. **Microservice 120:** What are pros and cons of caching?

Pros:

1. Speed: Caching makes your system faster by reducing data fetching time.
2. Reduced Server Load: Caching reduces the load on your database or primary servers.
3. User Experience: Quick response times lead to happier users!

Cons:

1. Stale Data: Cache data can become outdated, leading to data inconsistency.
2. System Complexity: Implementing caching adds an extra layer of complexity to system design.
3. Cache Invalidation: Determining when to refresh or clear cache can be challenging.

122. **Microservice 121:** What is Cache Invalidation?

Caching improves speed, but it comes with one tricky issue. When data changes in the main database, the cache must also reflect that change.

If the cache shows old data and the database shows new data, the system becomes unreliable.

This challenge is called the **cache invalidation problem**.

Problem 1: Stale Data

Stale data happens when the cache still holds old values even after the database has been updated.

Example

In an internet billing system, a customer changes their monthly plan from 50 Mbps to 100 Mbps.

The database gets updated, but the cache still holds the old plan value.

Now billing calculations become wrong because the service reads from cache instead of database.

Stale data is the most common cache issue.

Problem 2: Multiple Updates at the Same Time

When many requests update the same data, the cache can get confused about which value is the latest.

Example

A customer tries to change their plan at the same time customer care also updates the plan. If one update hits the database and another hits the cache, the values may not match.

Race conditions like this can break the flow in high-traffic systems.

Problem 3: Cache and Database Sync Timing

There is always a small delay between writing to the database and updating the cache. If the service reads the cache in this exact moment, it may read old data.

This is known as the “cache sync gap”.

Example

A payment is marked as completed in the database. Before the cache is updated, another service reads the old value from cache and thinks payment is still pending.

Problem 4: Cache Deletion Failure

Sometimes the cache fails to delete or update because of network issues or service crashes. The database is correct, but the cache still shows old data.

This is rare but dangerous because the system will silently behave wrong.

Problem 5: Distributed Cache Issues

When many service instances share a distributed cache like Redis, all instances must see the updated value at the same time.

If one instance updates and another instance read old data, problems start. This becomes harder when the system runs many replicas.

Follow-Up**How We Solve Cache Invalidation**

There are three common ways:

1. Cache Aside Pattern

Read from cache. If not present, fetch from database and fill cache. On update, first update the database, then remove the cache entry.

This avoids stale data in most cases.

2. Write Through

Every write goes to cache first and then to database. Good for strong consistency.

3. Time-to-Live (TTL)

Cache entries expire automatically after a fixed time. Even if something goes wrong, the cache becomes fresh again soon.

123. Microservice 122: Explain Cashe Aside, Write Through and Write-Behind?

Caching looks simple from the outside, but the real challenge begins when the data in the database changes.

The cache must stay updated or the whole system starts showing old values. To handle this safely, we rely on three patterns that shape how the cache and database work together.

Cache Aside

Cache Aside is the most common pattern. The service decides when to read from cache and when to go to the database.

When a request comes in, the service looks for the data in the cache. If the data is found, the service returns it immediately. If the data is not found, the service loads it from the database and then stores a copy in the cache. This way, the next request finds the data in memory and becomes faster.

For updates, the service first writes to the database. After that, it removes the old data from the cache. This forces the next read to pull fresh data from the database.

Example is a product catalog in an e-commerce site.

- Product names and prices do not change very often.
- When a user opens a page, the service checks the cache.
- If the product is missing, the service loads it from the database, places it into cache and continues.
- If the price is updated by an admin, the database gets the new price and the old cache entry is cleared.

Write Through

Write Through works differently. Every write goes into the cache first and the cache writes the same data into the database. Reads continue as usual, often coming directly from the cache.

Imagine updating a customer's profile.

- When the change comes in, the service writes the new value into the cache.
- The cache then writes the updated data into the database.
- Both stay in sync without gaps.

The only drawback is that writes can become slower because the data must pass through two layers.

Write Behind

Write Behind looks similar to Write Through, but with one important difference. The cache does not write the data to the database immediately. It collects changes for a short time and writes them in batches. This reduces database load and improves performance during heavy traffic.

Think of updating customer usage records every few seconds.

- If we update the database on every small change, it becomes slow.
- Write Behind allows the cache to buffer these updates and write them in a controlled way.

Write Behind is used only in systems where small delays are acceptable

124. Microservice 123: Where would you place caching and which is best (DB level, Service Level or API Gateway)?

Caching is one of the easiest ways to make a system fast. Instead of doing the same work again and again, we store the result once and reuse it.

The question is where to place this cache so it helps the most.

1. API Gateway
2. Service layer
3. Database layer

Each one has a purpose. Let's walk through them with simple examples.

1. Caching at the API Gateway

The gateway sits in front of all services. This is the first door every request hits.

When this helps

This works well when the data is the same for everyone and does not change often.

Example

- A list of top products rarely changes.
- If thousands of users request it every minute, the gateway can store the response once and give it back instantly.

2. Caching at the Service Layer

This sits inside the backend code. We control exactly which methods or operations should be cached.

When this helps

This is the best place when the service performs heavy work. Maybe it calls another service, maybe it runs a complex calculation, or maybe it hits the database many times.

Example

- A service loads product details and combines them with ratings.
- This work does not need to be repeated for every request.
- We store the result in Redis or Caffeine inside the service.

3. Caching at the Database Layer

This sits inside the database engine. The database keeps the hot queries in memory. Databases like MySQL, Postgres, MongoDB all have their own query caching or buffer caching. This helps return results quickly without touching disk.

When this helps

This is useful when read traffic is heavy and the same queries keep repeating.

Example

A query like “select product by id” runs thousands of times. The database keeps this result in its buffer. Later calls return much faster because the data is already in memory.

9. Spring Cloud

125. **Microservice 124:** What is Spring cloud and how it is useful in Microservices?

When we move from building a simple application to building a full microservice system, everything becomes harder. A microservice application is not just code. It needs **service discovery, load balancing, API gateway, centralized configuration, fault tolerance, logging, monitoring, and security**. Doing all this manually is painful and slows down development.

This is where **Spring Cloud** helps us.

Spring Cloud is a set of tools built on top of Spring Boot that helps us solve common problems in microservices. It does not force any rules. Instead, it gives solutions for real challenges like service communication, routing, failure handling, configuration, and deployments.

Without Spring Cloud, we would have to write many things by hand. With it, we focus more on business logic and less on infrastructure problems.

Why We Need Spring Cloud: Instagram Example

Let us imagine **Instagram with microservices**. We would have services like:

Service	Responsibility
user-service	manage profiles and followers
post-service	store posts
comment-service	store comments
like-service	handle likes
notification-service	send notifications
feed-service	show home timeline

These services must talk to each other and stay reliable even when traffic grows. Now imagine Instagram has millions of users online at the same time. That means the system needs to handle:

- service discovery (find service locations)
- load balancing (share traffic across instances)
- communication (REST/gRPC)
- API security and routing
- failure handling
- monitoring and logging
- centralized configuration

- scaling easily

Doing all this from scratch is nearly impossible.

How Spring Cloud Helps

Problem in Microservices	Spring Cloud Solution
Services must find each other	Eureka (Service Discovery)
Single entry point needed	Spring Cloud Gateway
Load balancing required	Spring Cloud LoadBalancer
Configuration changes should be centralized	Config Server
Service failures must be handled	Resilience4j
Communication between services	Feign Client
Tracing and logging	Sleuth + Zipkin
Distributed messaging	Spring Cloud Stream + Kafka/RabbitMQ

Example Flow: Feed Page in Instagram

When we open Instagram and refresh the feed:

1. The request goes to **API Gateway**.
2. API Gateway sends it to **feed-service**.
3. feed-service wants details of users and posts, so it calls:
 - **user-service** to get user info
 - **post-service** to get posts
4. These calls are routed through **Eureka service discovery**.
5. If post-service is slow, **Resilience4j** applies retry or fallback.
6. If load is high, **Spring Cloud LoadBalancer** balances traffic.
7. Configuration for all services is taken from **Config Server**.
8. Logs and tracing go to **Zipkin** for debugging.

Reference: [What Is Spring Cloud? A Beginner's Guide to Microservices Made Easy | by harsh_singh | Medium](#)

126. **Microservice 125: What do you know about feign-client?**

In a microservice system, services often need to talk to each other. One service may need data from another service to complete a task. Normally, we write HTTP calls manually using RestTemplate or WebClient, which takes more code and becomes hard to maintain as the project grows.

Example: In an ecommerce app. Order-service need to call the payment-service.

Feign Client solves this problem by giving a simple and clean way to call other services. Instead of writing HTTP code, headers, and request handling again and again, we only define an interface and Feign creates the HTTP call in the background. It makes service-to-service calls look like normal method calls in Java. This keeps the code readable and easy to follow.

Feign Client is a way to make communication easy between microservices in Spring Boot.

@FeignClient belongs to Spring Cloud, which provides tools for building applications that work well in cloud and microservice setups. Feign helps us call other services using simple Java interfaces instead of writing low-level HTTP code.

Feign also works well with service discovery tools like Eureka and load balancing tools like Spring Cloud LoadBalancer. It is widely used in Spring Boot microservices and is considered a standard way for internal service communication.

```
//Step 1: Enable Feign in main application
@EnableFeignClients
@SpringBootApplication
public class App { }

//Step 2: Create Feign Client to call another service
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable String id);
}

//Step 3: Use Feign Client in service class
@Service
public class OrderService {
    @Autowired
    private PaymentClient paymentClient;

    public Payment fetchPayment(String id) {
        return paymentClient.getPayment(id);
    }
}
```

Step 1: Enable Feign

Feign is activated using @EnableFeignClients in the main Spring Boot class. This tells Spring to scan and load Feign clients in the project.

Step 2: Define a Feign Client

The @FeignClient interface declares a connection to another microservice. Instead of writing HTTP code, we define a method that matches the API of another service.

Step 3: Call another service using Feign

The Feign client is injected into a service class and used like a normal Java method. When the method is called, Feign sends an actual HTTP request in the background to another microservice.

Reference: [Mastering @FeignClient in Spring | Medium](#)

Reference with Eureka server and load balancer: [How To Implement Feign Client In Spring Boot Microservices? - JavaTechOnline](#)

Sample code from the above link: [Spring-Samples/FeignClient at main · CodingLyf-Fullstack/Spring-Samples](#)

In the sample:

- Producer is BOOK-SERVICE
- Consumer is STUDENT-SERVICE
- Consumer contains BookRestConsumer.java which calls producer using Feign.
@FeignClient(name="BOOK-SERVICE").
- Means we are calling BOOK-SERVICE from STUDENT-SERVICE using FeignClient

127. **Microservice 126:** How Feign-client work internally?

Feign Client is a way for microservices to talk to each other in Spring Boot without writing long HTTP code. It gives a clean method-call style for calling another service. Inside, it still uses HTTP, but it hides the complexity and gives a simple interface.

How Feign Client Works

When a method in Feign Client is called, it does not run directly. Instead, Spring creates a dynamic proxy behind that interface. The proxy builds the HTTP request and sends it to the target service. After getting the response, it converts it into a Java object and returns it to the caller.

Steps inside Feign:

1. We define an interface with @FeignClient.
2. Spring creates a proxy for that interface during startup.
3. When we call the method, the proxy sends an HTTP request to another service and returns the response.

128. **Microservice 127:** How does Feign Client know which service URL to call in a microservice system?

Feign Client does not store or hardcode service URLs inside it. Instead, it depends on the service discovery mechanism in a microservice system. When a Feign Client is created with a service name, that name is registered in a service registry like Eureka or Consul. During the call, Feign asks the service registry to find the current network location of the target service.

If multiple instances of the same service are running, Feign works with load balancers to choose one healthy instance. After the right instance is found, Feign builds the full URL and sends the HTTP request. This process happens automatically in the background.

129. Microservice 128: How Feign client works if Service registry is not used in the project?

Feign client can still be used even when there is no service registry like Eureka in a project. In this case, Feign does not discover services automatically. Instead, the target service URL must be given manually in the configuration. Feign will always call that fixed URL for every request.

This approach works well in small systems or local setups where services run on known ports. But it does not support automatic scaling or load balancing. If the service location changes, we must update the configuration manually. So, this method is simple but not flexible compared to using a service registry.

```
@FeignClient(  
    name = "payment-service",  
    url = "http://localhost:8082"    // Fixed URL since no Eureka  
)  
public interface PaymentClient {  
  
    @GetMapping("/payments/{id}")  
    Payment getPayment(@PathVariable String id);  
}
```

```
@Service  
public class OrderService {  
  
    @Autowired  
    private PaymentClient paymentClient;  
  
    public Payment fetchPayment(String id) {  
        return paymentClient.getPayment(id); // Feign calls fixed URL  
    }  
}
```

130. Microservice 129: How feign-client is different from Rest -Template and Web Client?

In a microservice system, one service often needs to call another service. In Spring Boot, this can be done using RestTemplate, WebClient, or Feign Client. All three solve the same problem, but they work in different ways.

RestTemplate is a simple way to make HTTP calls, but it needs more code for every request. We must build the URL, set headers, and handle responses manually. It is synchronous and blocks the thread until a response comes back.

```
@Service  
public class UserService {  
    @Autowired  
    private RestTemplate restTemplate;  
  
    public User getUser(String id) {  
        return restTemplate.getForObject("http://user-service/users/" + id, User.class);  
    }  
}
```

WebClient is a newer and non-blocking way to make HTTP calls. It supports asynchronous programming and can handle high traffic better. But the code becomes complex when handling multiple service calls or when combining responses.

```
@Service
public class UserService {
    private final WebClient webClient = WebClient.create();

    public Mono<User> getUser(String id) {
        return webClient.get()
            .uri("http://user-service/users/" + id)
            .retrieve()
            .bodyToMono(User.class);
    }
}
```

Feign Client is the simplest among the three. It hides the HTTP call and allows us to call another service just by writing an interface. It keeps the code clean and readable and fits well in microservices.

```
@FeignClient(name = "user-service")
public interface UserClient {
    @GetMapping("/users/{id}")
    User getUser(@PathVariable String id);
}

@Service
public class UserService {
    @Autowired
    private UserClient userClient;

    public User getUser(String id) {
        return userClient.getUser(id);
    }
}
```

Reference: [Feign Client vs Rest Client. A Comprehensive Guide | by Lahiru Rajapakshe | Medium](#)

131. **Microservice 130: When to use feign-client, Rest -Template and Web Client?**

RestTemplate: RestTemplate is a simple way to call REST APIs. It works in a blocking manner, which means it waits for the response before moving to the next step. It is easy to understand and good for small projects, but it needs more code to handle headers, errors, and response mapping.

Use RestTemplate when:

- The project is small
- The system has fewer API calls
- Performance is not a priority
- Simplicity is more important than flexibility

WebClient: WebClient is the modern way to call APIs in Spring Boot. It is asynchronous and non-blocking, which makes it faster and more efficient. It can handle multiple parallel requests easily.

Use WebClient when:

- The system needs high performance
- There are many APIS calls
- The service must handle more traffic

Feign Client: Feign Client is built for microservices. It keeps code clean by hiding the HTTP call logic. Instead of writing full request code, we define an interface and Feign takes care of the rest. It works well with load balancing and service discovery. It is the best choice for service-to-service communication.

Use Feign Client when:

- The system follows microservice architecture
- Services call each other frequently

132. **Microservice 131:** What are the advantages of Feign Client?

1. Declarative approach

Feign client follows a declarative style. We only write a Java interface and mark it with `@FeignClient`. We do not write HTTP code, connection logic or response handling. Spring takes care of calling the API and converting the response. This keeps the code clean and easy to understand.

Example idea

An order service calls product service using a simple method call instead of HTTP code.

2. Built-in load balancing

In microservices, many instances of the same service can run at the same time. Feign works with service discovery tools like Eureka. When a request is made, Feign automatically sends it to one of the available instances. This spreads the load and avoids sending all traffic to one server.

3. Integrated security

Feign works smoothly with Spring Security. Security tokens and headers can be passed automatically between services. This ensures that only allowed services can talk to each other. Communication stays protected without adding extra code in every service call.

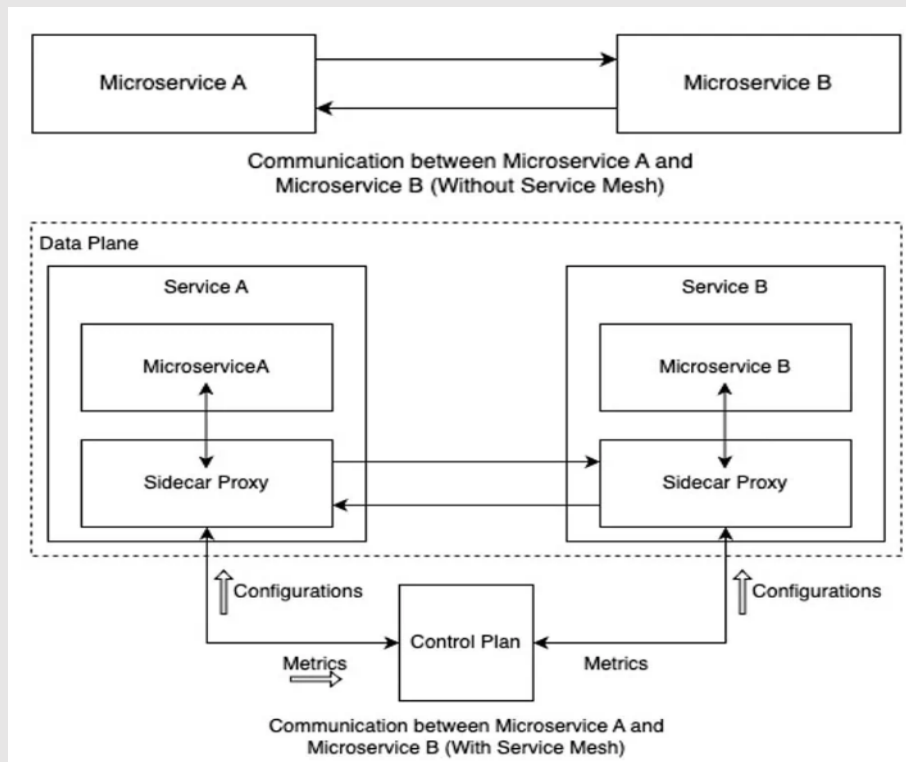
4. Fallback mechanism

Service failures can happen at any time. Feign allows us to define fallback logic. If a service is down or slow, a fallback method runs instead. This returns a safe response and keeps the system working instead of failing completely.

133. **Microservice 132:** What is service mesh and how it is useful in microservices??

A **service mesh** is a dedicated layer in a microservices system that manages how services communicate with each other. In microservices, many small services talk to each other over the network. Each service must handle security, retries, monitoring, timeouts and traffic control. Managing all this inside every service becomes difficult and leads to repeated code.

A service mesh solves this problem by moving these responsibilities out of the services and into a separate network layer. Each service keeps only business logic, while the mesh takes care of communication.



How it works

A service mesh uses **sidecar proxies**. These small proxy programs run next to each service. When one service talks to another, the traffic goes through these proxies. The proxies manage security, control network failures, collect logs and send metrics. This keeps the services clean and focused.

Example

Imagine a food delivery system with services like order service, payment service, delivery service and notification service. Each service communicates with others. If every service handles retries, timeouts and security by itself, the system becomes complex. Code is repeated everywhere.

With a service mesh like **Istio**, we add proxies beside each service. Now features like retry, logging and monitoring are handled by the mesh. The services only focus on what they are built to do.

Where it is used

Service mesh is used in large microservice architectures where there are many moving parts and communication must be managed centrally. It improves visibility, security and control without changing service code.

Reference: [Service Mesh for Microservices. Microservices architecture has been... | by Kasun Indrasiri | Microservices in Practice | Medium](#)

134. Microservice 133: How service mesh is different from API Gateway?

Both API Gateway and Service Mesh handle communication, security and monitoring in microservices, so they look similar. But they solve different problems and work in different parts of the system.

Let us understand the real difference in a simple and clear way.

Where they are used

Feature	API Gateway	Service Mesh
Works at	System entry point	Inside the system between services
Controls	Client to service traffic	Service to service traffic
Used for	External communication	Internal communication

Simply

- An API Gateway is like the main gate of a building. It handles everything coming from outside before entering the system.
- A Service Mesh is like the internal security system in the building. It manages internal traffic between rooms (services).

Example

Let us say we run an e-commerce app with these services:

- User Service
- Cart Service
- Order Service
- Payment Service

API Gateway

All users from the outside world enter through the gateway.

- It validates tokens
- Controls access
- Routes /orders to Order Service
- Protects system from high traffic

Service Mesh

Once inside, services talk to each other.

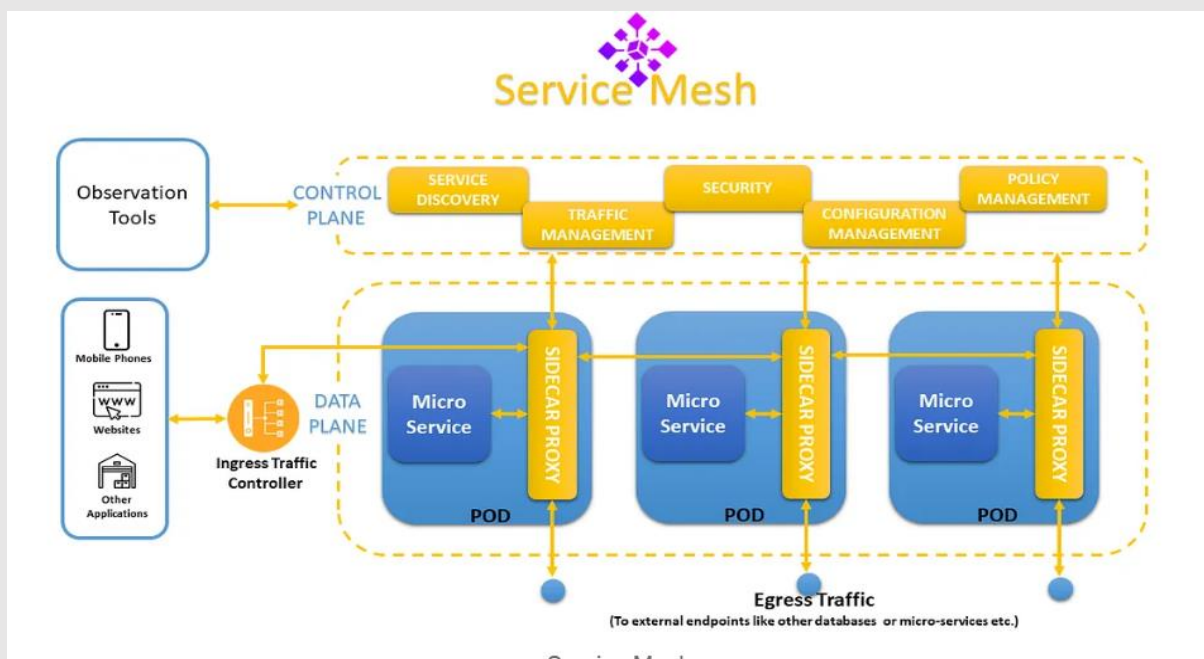
- Order Service calls Payment Service
- Payment Service calls Notification Service
- Mesh handles retries, timeouts and security between services

135. Microservice 134: Explain service mesh architecture?

In a service mesh, each microservice does not talk directly to another service. Instead, every service has a small helper program running next to it. This helper is called a **sidecar proxy**. All network communication goes through these proxies.

- When **Service A** wants to talk to **Service B**, it sends the request to its own proxy.
- The proxy forwards the request to the proxy of **Service B**.
- The proxy of **Service B** delivers the request to **Service B**.
- The same path is followed for the response.

This way, services do not need to worry about networking and security.

**Control Plane and Data Plane**

A service mesh has two main parts:

1. Data Plane

This is made up of all sidecar proxies. They handle the actual traffic between services. They perform encryption, retries, rate limits and collect metrics.

2. Control Plane

This manages and configures all the sidecar proxies. It tells them how to route traffic, when to retry and what security rules to follow.

Example

Imagine a food delivery app with services like Order Service, Payment Service and Delivery Service. Normally, Order Service talks directly to Payment Service. But if Payment Service is slow, Order Service must retry, log errors and handle failures.

With a service mesh, Order Service does not handle any of that. It sends the request to its sidecar proxy. The proxy handles retry if Payment Service is slow, logs the request and secures communication. Payment Service also has a proxy to receive the request safely.

Reference: [Understanding the Role of Service Mesh in Microservices Architecture | by ShivajiKant | Medium](#)

Microservices Scenarios

136. Scenario 1: Monolith to Microservice?

Question: You have given a big single application and your job is to convert into microservices, how would approach?

Solution:

We never break the system in one shot. That is risky and can create problems in production. Instead, we follow a step-by-step plan.

Step 1: Understand the Business Domains

First, we study the business and group related features together.

For example, let us say we have an e-commerce application. The monolith may have these modules:

- Product Management
- User Management
- Order Management
- Payment
- Inventory
- Delivery Tracking

All of these are mixed together in one codebase. This is a problem when the code grows bigger.

Step 2: Identify Services

We divide the application based on business areas. Each area becomes a separate microservice.

For our example, we can split like this:

Microservice	Responsibility
Product Service	Manage products, categories
User Service	Handle users, login, profile
Order Service	Create orders, order history

Microservice	Responsibility
Payment Service	Process payments
Inventory Service	Track stock
Delivery Service	Shipping and tracking

Each microservice will have its **own database** so that it stays independent and safe from other services.

Step 3: Communication

Now we must decide how these services talk to each other. They cannot share code directly anymore. They must speak through APIs.

- For simple calls, we can use **REST APIs** between services.
- For asynchronous events like order placed or payment success, we can use **message queues** like Kafka or RabbitMQ.

Example:

When the Order Service creates a new order, it sends a message to the Inventory Service to reduce stock.

Step 4: Create a Strangler Plan

We do not delete the monolith suddenly. We slowly replace parts of it. This safe method is called the **Strangler Pattern**. We take one module at a time, move it out as a microservice, and connect it back to the old system.

For example:

- First, move Product module out of the big app.
- Then connect it using API.
- Test it.
- Then move User module.
- Continue like this step by step.

Step 5: Build Deployment and Monitoring

Microservices bring complexity, so we must plan deployment and monitoring before going live.

- Container tools like Docker help run each service separately.
- Kubernetes can manage deployment and scaling.
- Monitoring tools like Prometheus or Grafana help check if services are healthy.
- Logs must be centralised to track errors.

Step 6: Testing and Migration

Before we go live, every service must be tested.

We write **unit tests**, **integration tests**, and **API tests**.

Also, we must migrate old data carefully into new service databases without losing anything.

137. Scenario 2: Monolith to Microservice: Billing system

Question: You have single Internet billing system application and you need to divide into invoice, Payment & Pricing Microservices

Solution:

The best way to bring back order is to divide the system based on the natural flow of internet billing. This flow always follows three steps:

1. Decide the price
2. Generate the bill
3. Collect the payment

These steps map perfectly to three microservices: **Invoice**, **Pricing**, and **Payment**.

1. Invoice Service

Internet billing is mostly monthly.

The invoice service is responsible for creating and tracking each month's bill.

What it handles

- Monthly invoice generation
- Billing cycles
- Due dates
- PDF or email bill
- Invoice status such as generated, pending, late, or paid

Example

It is the first of the month.

The invoice service picks all active internet connections.

For each connection, it calls the pricing service to get the current charge and generates the monthly invoice.

If the user upgraded speed mid-month, it calculates a partial charge as well.

2. Pricing Service

Internet plans usually come with different speeds, data limits, add-ons, and promos.

The pricing service handles all of this. It becomes the single place where billing asks for the cost of a plan.

What it handles

- Base rate of internet plans
- Speed-based pricing
- Promotions and discounts
- Taxes
- Final billable amount

Example

A user upgrades from a 50 Mbps plan to a 200 Mbps plan.
The invoice service calls pricing with the user's plan details.
Pricing calculates the new monthly rate, applies tax, and returns the amount.
This keeps the pricing logic in one clean place.

3. Payment Service

The payment service deals with the actual money movement. Users may pay through online banking, UPI, card, or auto-debit. This service manages all of it.

What it handles

- Payment requests
- Payment success and failure
- Refunds
- Transaction logs
- Informing the invoice service that payment is complete

Example

A user pays the bill through UPI.
The payment gateway sends a success callback.
The payment service updates the transaction record and informs the invoice service that the bill is paid.

4. Connecting the Services

Even though the system is now split, the services still need to work together.

How they talk

- The invoice service calls pricing to calculate the amount
- The payment service notifies invoice after a successful payment using message queues like kafka or RabbitMQ

138. Scenario 3: Authentication Token Refresh - Streaming App

Scenario: In a video streaming app like Netflix, every service call needs a valid authentication token. Tokens expire often.

Question: Will you add refresh token logic in every service; how would you implement it?

Solution:

Instead of writing token refresh logic in every service, a **sidecar service** handles:

- Checking token expiry
- Refreshing tokens from Auth Server
- Attaching tokens to outgoing API calls

Main service stays clean and focuses only on streaming logic.

Or We can use API Gateway to handle refresh token logic.

139. Scenario 4: Shared data between multiple services of Database

Scenario: How would you handle Shared Data in Microservices?

Solution:

In a microservices architecture, handling shared data between multiple services is tricky because each service should be loosely coupled and own its data. Here's how you can handle it:

1. Database per Microservice (Best Practice)

Each microservice should have its own database/schema. Prevents tight coupling and improves scalability.

Example: UserService manages users table, OrderService manages orders table.

2. Data Sharing via APIs (Service-to-Service Communication)

Services communicate using REST/gRPC/GraphQL. Instead of directly accessing another service's database, they call APIs.

Example: OrderService calls UserService API to fetch customer details.

3. Event-Driven Architecture (Asynchronous Sharing)

Use a message broker (Kafka, RabbitMQ, AWS SQS, etc.) to publish and consume events.

Example: When PaymentService processes a transaction, it publishes an event, then OrderService and NotificationService consume it.

4. Shared Data through Caching (Read-Heavy Use Cases)

For frequently accessed data, use distributed caches (Redis, Hazelcast). Reduces dependency on synchronous calls.

Example: User profile info cached for quick access by multiple services.

5. API Gateway for Aggregation

Instead of services directly calling each other, use an API Gateway to aggregate responses.

Example: Customer Dashboard request -> API Gateway -> fetches from UserService, OrderService, PaymentService -> merges response.

140. Scenario 5: Selection of Database

Scenario: You are working on a microservices application and you have to use either database per service or shared database for all the services.

Question. Which one you prefer and tell the reason?

Solution:

In a microservice system, it is better to follow the **database per service** pattern. Each service should have its own database so that it stays independent and safe from failures in other services. This also allows each service to choose the right database for its own need.

For example, a Product Service may use MongoDB for fast reads, while an Order Service may use MySQL for transactions. If we use a shared database, services become tightly connected, and a change made by one team can break others

The only challenge with separate databases is data consistency, but this can be solved using patterns like Saga and event-driven communication.

141. **Scenario 6: Book a Flight - Travel App**

In a travel booking app, we book a flight through multiple services:

1. **Flight Booking Service** reserves the flight ticket
2. **Payment Service** charges the customer
3. **Notification Service** sends confirmation

Now imagine this situation:

- Money is successfully deducted by Payment Service
- But Flight Booking Service fails while reserving the seat

Question: How would you solve this issue in real time?

Solution: This problem happens because each service runs separately. The payment succeeds, but the flight booking fails. So, money is lost and no ticket is confirmed.

We solve this using the Saga pattern. It is a simple step-by-step process that also supports undo steps when something fails.

When the user tries to book a flight, we start a booking workflow.

First, the Flight Booking Service tries to reserve a seat. If this is successful, then the Payment Service collects money.

After that, the Notification Service sends the booking message.

If any step fails, we run a compensation step.

For example, if the payment is done but later flight booking fails due to no seats, the Saga will undo the payment by calling the Payment Service to refund the amount. So, each step has a reverse step. This way, the system stays consistent even when one service fails. No data will be lost and the experience stays safe.

142. **Scenario 7: Bank Account Transfer Between Two Different Databases**

Imagine a banking system where we transfer money from Account A to Account B. But the catch is:

- Account A is in Database 1 (maybe in one branch system)

- Account B is in Database 2 (maybe another branch system)

Both databases are separate. Still, the money transfer must be 100% consistent. Money should not disappear or duplicate.

Problem

We perform a transfer of ₹10,000 from A to B.

Steps:

1. Debit 10,000 from Account A (DB1)
2. Credit 10,000 to Account B (DB2)

What if DB1 is updated but DB2 fails to update? Money will disappear. This is a **serious financial risk**.

Solution: Use 2 Phase Commit (2PC)

We use a Transaction Coordinator to manage this safely in two phases. 2PC is a protocol that ensures all participating systems in a distributed transaction either commit or roll back together, guaranteeing strong data consistency.

Phase	What Happens
Prepare Phase	Coordinator asks both databases if they are ready to commit
Commit Phase	If both say yes, then coordinator commits in both DBs, else rolls back

Additional Question: Why can't we use Saga pattern here?

Saga works well when **eventual consistency is acceptable**.

For example in e-commerce:

- If payment succeeds but inventory update fails, we can cancel order later
- A short delay is okay
- Business can survive temporary inconsistency

But bank money transfers are different they expect Strong consistency

Saga is good for long business workflows that can tolerate delay and compensation. But money transfer needs strong consistency, not eventual consistency. So 2PC is preferred over Saga for bank transfers.

143. Scenario 8: Multiple instances of a Service

Scenario: Multiple instances of a service is running and one instance got down while processing the DB transaction. How would you ensure it won't break the functionality?

Solution:

When many instances of a service are running, one of them can fail in the middle of a database operation. A good design should handle this without breaking the flow. The safest way is to rely on two things: database transactions and idempotent logic.

Use Distributed Transaction Patterns: A transaction makes sure the database commits everything together or cancels everything. So, if an instance goes down halfway, the database rolls back the incomplete work automatically. Nothing stays in a half-done state. Use Distributed Transaction Patterns like Two-Phase commit or Saga

The second part is **idempotency**. This means the same task can be retried safely without creating duplicates. For example, creating an order with the same request id should always produce one final order, even if the system retries it after a crash.

Another example, every transaction can carry a unique transaction ID; if retried, the service checks if it already processed that ID before executing again.

Retry using message queue: To make retries possible, we can store each task in a queue or a job table. When one instance fails, another instance sees the pending task and completes it cleanly.

This way, even if one instance stops during a transaction, the system recovers smoothly and continues without data corruption or repeated work.

144. Scenario 9: Communication between two services - 1

Imagine an e-commerce app, where it has two services order-service and payment-service. After user added items to add, from order-service we need to call payment-service for payment

Question: How would you achieve this communication between those two services?

Solution: We need to use Synchronous communication in this case or we can use webhooks for this..

When we buy something online and click Pay Now, the Order Service calls the Payment Service and waits for a response before proceeding. If payment is successful, the order gets confirmed. If payment fails, the order is cancelled. The system waits for the reply.

145. Scenario 10: Communication between two services - 2

Imagine an e-commerce app, now after the order is placed you need to send confirmation mail or message to user. For that we have a new service called notification-service.

Question: How would you achieve this communication between those order-service and notification-service?

Solution: We need to use Asynchronous communication in this case.

After placing the order, the system does not need to wait for email or SMS to be sent. The Order Service just sends a message to a message queue like Kafka or RabbitMQ. The Notification Service picks it up later and sends the email.

146. Scenario 11: Invoke API from Frontend

We have a **blog application** with three REST APIs inside the same backend project:

API	Purpose
/api/user/{id}	Get user profile
/api/posts/{userId}	Get all posts by user
/api/comments/{postId}	Get comments for a post

Now the frontend wants to load a User Profile Page that shows:

- User details
- User posts
- Comments for each post

With REST, the frontend must make multiple calls like this:

```
GET /api/user/1
GET /api/posts/1
GET /api/comments/11
GET /api/comments/12
```

Question: How would you implement this in better way?

Solution: Use GraphQL

Instead of calling multiple REST APIs, we add GraphQL on top of the existing REST APIs.

Frontend now makes one GraphQL call:

```
query {
  user(id: "1") {
    name
    email
    posts {
      title
      comments {
        text
      }
    }
  }
}
```

GraphQL internally calls the existing controllers and combines the data.

147. **Scenario 12: Real-Time Data Requirements**

Scenario: Your application requires real-time updates, such as displaying new messages in a chat application. How would you implement this using REST and GraphQL?

Solution:

REST typically uses polling or long-polling to check for updates, which can be inefficient and resource-intensive.

GraphQL supports real-time data through **subscriptions**, allowing clients to receive updates over a WebSocket connection when data changes on the server.

148. Scenario 13: API Gateway

Scenario: If the frontend needs data from 5 different microservices, how will you reduce the number of API addresses from the frontend?

Solution: We can solve this using API Aggregator at the API Gateway. Instead of five API addresses from the frontend, the frontend will make only one request to the gateway. The gateway will call the required services internally, merge the data, and return a single combined response.

This keeps the frontend clean and improves performance.

Frontend -> API Gateway -> Calls 5 Services -> Returns one combined response

```
@RestController
public class DashboardController {

    @Autowired UserClient userClient;
    @Autowired OrderClient orderClient;
    @Autowired WalletClient walletClient;
    @Autowired CartClient cartClient;
    @Autowired OfferClient offerClient;

    @GetMapping("/dashboard/{userId}")
    public Dashboard getDashboard(@PathVariable String userId) {

        Dashboard dashboard = new Dashboard();
        dashboard.setUser(userClient.getUser(userId));
        dashboard.setOrders(orderClient.getOrders(userId));
        dashboard.setWallet(walletClient.getWallet(userId));
        dashboard.setCart(cartClient.getCart(userId));
        dashboard.setOffers(offerClient.getOffers(userId));

        return dashboard;
    }
}
```

Follow-up Question: Would you use API Gateway aggregation or GraphQL?

Use GraphQL when the frontend wants to choose what data to receive and reduce extra fields.

Use API Gateway aggregation when the backend must combine data and apply business logic before sending the response.

149. Scenario 14: Rate Limiting

Scenario: How will you implement a Rate limiting, different rate limits for free users and premium users?

Solution: Use API Gateway Filter + Bucket4j

Rate limiting is used to control how many requests a user can make in a given time in a real product, we do not want all users to have the same request limit. Free users can have limited access, but premium users should get better speed and more allowed requests. This is done at the API Gateway level before the request reaches any microservice.

To build this, we check the user type based on a token or user role. Then we apply different request limits. For example, free users may get 10 requests per minute, but premium users can get 100 requests per minute. If the limit is crossed, the gateway blocks extra requests for that period.

Sample code:

```
@Component
public class RateLimitFilter extends
AbstractGatewayFilterFactory<RateLimitFilter.Config> {

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            String userType = exchange.getRequest().getHeaders().getFirst("X-User-
Type");

            Bucket bucket = buckets.computeIfAbsent(userType, k ->
createBucket(userType));

            if (bucket.tryConsume(1)) {
                return chain.filter(exchange);
            }
            exchange.getResponse().setStatusCode(HttpStatus.TOO_MANY_REQUESTS);
            return exchange.getResponse().setComplete();
        };
    }

    private Bucket createBucket(String userType) {
        int limit = userType.equals("PREMIUM") ? 100 : 10;
        return Bucket4j.builder()
            .addLimit(Bandwidth.simple(limit, Duration.ofMinutes(1)))
            .build();
    }

    public static class Config {}
}
```

- The class extends AbstractGatewayFilterFactory, so it runs as a filter inside API Gateway for every request.
- It reads the X-User-Type header to check whether the user is FREE or PREMIUM.
- It creates a separate Bucket4j rate limit bucket for each user type using createBucket().
- If the bucket allows a request (tryConsume(1)), the request is forwarded.
- If the request limit is crossed, it returns HTTP 429 (Too Many Requests) and blocks the call.

Reference links: [Getting Started with Spring Cloud Gateway — Part Two | by Jayaramanan Kumar | Medium](#)

150. Scenario 15: API Aggregation or Composition

A user API waits on different services. Response time is slow. What patterns reduce client-side latency?

Scenario: A digital banking application wants to show a personalized dashboard when a customer logs in. The dashboard must show:

- Account balance from Account Service
- Recent transactions from Transaction Service
- Reward points from Reward Service

The key point: All this data must be returned together in a single API call for performance reasons, and some data depends on combining results from multiple services.

Solution: API Composition using API Gateway. The gateway aggregates data from multiple backend services parallelly and returns a single response, reducing client calls and improving performance.

Below is a simple and clean example of API Composition using Spring Boot API Gateway. The API Gateway calls 3 backend services and builds one combined dashboard response.

Step1: pom.xml dependencies (Spring Cloud Gateway + Feign)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Step 2: Enable Feign Client

```
@SpringBootApplication
@EnableFeignClients
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
```

Step 3: Feign clients to Microservices

```
@FeignClient(name = "account-service")
public interface AccountClient {
    @GetMapping("/accounts/{customerId}")
    Account getAccount(@PathVariable String customerId);
}

@FeignClient(name = "transaction-service")
public interface TransactionClient {
    @GetMapping("/transactions/{customerId}")
    List<Transaction> getTransactions(@PathVariable String customerId);
}
```

```

}

@FeignClient(name = "reward-service")
public interface RewardClient {
    @GetMapping("/rewards/{customerId}")
    Reward getRewards(@PathVariable String customerId);
}

```

Step 4: Response DTO

```

@Data
@AllArgsConstructor
public class DashboardSummary {
    private String customerId;
    private double balance;
    private List<Transaction> recentTransactions;
    private int rewardPoints;
}

```

Step 5: API composition

```

@RestController
@RequiredArgsConstructor
public class DashboardController {

    private final AccountClient accountClient;
    private final TransactionClient transactionClient;
    private final RewardClient rewardClient;

    @GetMapping("/dashboard/{customerId}")
    public DashboardSummary getDashboard(@PathVariable String customerId) {

        Account account = accountClient.getAccount(customerId);
        List<Transaction> transactions = transactionClient.getTransactions(customerId);
        Reward reward = rewardClient.getRewards(customerId);

        return new DashboardSummary(
            customerId,
            account.getBalance(),
            transactions,
            reward.getPoints()
        );
    }
}

```

Request: GET <http://localhost:8080/dashboard/101>

Response

```

{
  "customerId": "101",
  "balance": 54000.0,
  "recentTransactions": [
    {"amount": -500, "type": "DEBIT"},
    {"amount": 2000, "type": "CREDIT"}
  ],
  "rewardPoints": 120
}

```

```
}
```

Why other options fail:

Approach	Why it fails
Frontend multiple REST calls	Too many calls, slow, bad for mobile
One backend REST API	Becomes a fat monolith
GraphQL	Frontend-driven, but cannot handle backend business logic combinations needed here

151. Scenario 16: Traffic routing

Scenario: How do we route traffic to multiple instances of the same microservice using API Gateway?

Solution: Use API Gateway Filter + Load balancing

In a microservice system, we often run more than one instance of the same service. This is done to handle more traffic and keep the application available even if one instance fails. But once we have multiple instances, we need a way to route incoming requests across them. This is done through load balancing at the API Gateway level.

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE # Load balancing
          predicates:
            - Path=/users/**
```

In this setup, if three instances of USER-SERVICE are running, the API Gateway will automatically balance the traffic between them using round-robin load balancing. No extra code is needed.

Follow-up: Will you use service discovery here?

Yes, service discovery is required here. It helps the API Gateway find all running instances of a microservice. Without service discovery, the gateway cannot distribute traffic dynamically.

152. Scenario 17: Secure the microservices

Scenario: How do we route traffic to multiple instances of the same microservice using API Gateway?

Solution: Use API Gateway + Spring Security

In a microservice system, services should not be exposed directly to the outside world. Only the API Gateway should be visible to the public. All other internal services must stay protected and allow calls only from trusted sources. The gateway works like a security guard at the main gate, checking every request before sending it inside.

To secure internal communication, we use authentication and authorization at the API Gateway level. Every request must include a valid token, such as a JWT token. The gateway verifies this token before forwarding the call to any microservice. After that, services trust only the gateway and reject any direct request from outside. This makes the system safe from bypass attacks and keeps internal APIs protected from illegal access.

Follow-up: Will you expose microservices directly to the public?

No, microservices should not be exposed directly to the public. They must stay behind the API Gateway for security and control. Only the gateway should be publicly accessible, and it should handle authentication, authorization, and request filtering before forwarding requests to internal services. This prevents direct attacks on microservices.

153. Scenario 18: Routing

Scenario: Based on user role in JWT, requests should go to different service clusters. Example: Free users to basic cluster, paid users to faster cluster. Can we do this at API Gateway?

Solution:

Yes, we can do this using API Gateway.

Every request that reaches the gateway carries a JWT token for authentication. Inside this token, we have user details like id, email, and role. The API Gateway can read this token before sending the request to any service.

Based on the role in the token:

- If role is FREE, send the request to the basic service cluster
- If role is PAID, send the request to the premium cluster

This way, the gateway controls how traffic flows without changing anything in the backend code.

154. Scenario 19: Handling errors

Scenario: Imagine an e-commerce system where three microservices are involved when a customer places an order:

- **Order Service (A)**
- **Payment Service (B)**
- **Inventory Service (C)**

The flow is simple:

Order > Payment > Inventory

Now let's see what happens if Inventory Service (C) throws an error.

1. Order Service (A) calls Payment Service (B)

A customer tries to buy a product. Order Service starts the flow by asking Payment Service to process the payment.

2. Payment Service (B) calls Inventory Service (C)

Once the payment is done, Payment Service asks Inventory Service to reduce the stock for that product.

Real-world situation:

Let's say Inventory Service finds that the product is already out of stock. It throws an error like: **"Stock not available"**

3. Inventory Service (C) throws exception

This error should not be hidden. It should travel back up the chain so the system behaves correctly.

4. How Payment Service (B) handles the exception

Payment Service receives error from Inventory Service.

It should:

- catch the error
- add a meaningful message
- send back a clear response to Order Service

Example:

Instead of sending a raw 404 error, Payment Service returns: **"Stock not available in Inventory Service"**. This makes debugging and logging easier.

5. Payment Service passes the error to Order Service (A)

Order Service receives the final clear error message.

It can now:

- cancel the order
- refund the payment if necessary
- show a simple message to the customer like **"Item is out of stock"**

The user sees a clean message even though multiple services were involved.

155. Scenario 20: Event Driven System

Scenario: A microservice needs to perform a long running task like generating the report, How would you implement it?

Solution

1. Asynchronous Processing with Message Queues

Use a message broker like Kafka, RabbitMQ, or AWS SQS. The main service sends the task request to a queue, and a background worker service processes it asynchronously. The client immediately receives an acknowledgment instead of waiting for completion.

2. Event-Driven Architecture

Publish an event (e.g., "GenerateReportRequested") and let another service subscribe to that event.

This ensures loose coupling and allows independent scaling.

4. Polling or Webhooks for Result Delivery

The client can poll a status endpoint to check progress or use webhooks to notify clients when the task completes. Webhooks are preferred.

156. **Scenario 21: Synchronous API Processing**

Scenario: Imagine a system like Netflix, every time a user does something, like play a video, pause, search, browse a genre, or give a thumbs up. How it can be handled over millions of users?

Problem: Millions of users generate events every second. Netflix cannot use a traditional request-response database model for this.

Solution

- In a real-time event streaming system like Netflix, Kafka is used to process millions of user interaction events per second, such as play, pause, and search.
- These events are published to Kafka topics and consumed by multiple microservices for real-time recommendations, analytics, and playback monitoring.
- Kafka is preferred over RabbitMQ because it supports high throughput, event retention, replay, and scalable consumer groups suitable for event-driven architecture

Kafka Brokers

- These are the servers that store and manage events happening on Netflix.
- Example: You press Play on a movie. That action is stored safely by the brokers.

Kafka Topics

- Topics are simple buckets that group similar types of events.
- Example:
Play events are stored in a topic called play_events
Pause events are stored in pause_events
Like events are stored in like_events

Producers

- A producer sends events into Kafka. In Netflix, the app behaves like a producer.
- Example: When you tap Pause, the Netflix app sends that event to Kafka.

Consumers

- These are services that read events from Kafka and take action.
- Example:
The watch history service adds the show to Continue Watching
The recommendation service updates Top Picks for You

Partitions

- Each topic is divided into smaller parts called partitions which helps Kafka process many events at the same time.
- Example: Even if millions of people press Play at the same moment, Kafka handles everything without delay.

Durability and Replay

- Kafka keeps events stored and safe. If any service goes down, it can replay the missed events later.
- Example: Your watch history is never lost even if a service crashes. It can replay your actions and recover.

Summary

Kafka helps Netflix track and stream user actions in real time. It keeps data safe, scales easily and allows different services to react instantly.

157. Scenario 22: Real-time Event Streaming

Scenario: Imagine a system like Netflix, every time a user does something, like play a video, pause, search, browse a genre, or give a thumbs up. How it can be handled over millions of users?

Problem: Millions of users generate events every second. Netflix cannot use a traditional request-response database model for this.

Solution

- In a real-time event streaming system like Netflix, Kafka is used to process millions of user interaction events per second, such as play, pause, and search.
- These events are published to Kafka topics and consumed by multiple microservices for real-time recommendations, analytics, and playback monitoring.
- Kafka is preferred over RabbitMQ because it supports high throughput, event retention, replay, and scalable consumer groups suitable for event-driven architecture

Kafka Brokers

- These are the servers that store and manage events happening on Netflix.
- Example: You press Play on a movie. That action is stored safely by the brokers.

Kafka Topics

- Topics are simple buckets that group similar types of events.
- Example:
Play events are stored in a topic called `play_events`
Pause events are stored in `pause_events`
Like events are stored in `like_events`

Producers

- A producer sends events into Kafka. In Netflix, the app behaves like a producer.
- Example: When you tap Pause, the Netflix app sends that event to Kafka.

Consumers

- These are services that read events from Kafka and take action.
- Example:
The watch history service adds the show to Continue Watching
The recommendation service updates Top Picks for You

Partitions

- Each topic is divided into smaller parts called partitions which helps Kafka process many events at the same time.
- Example: Even if millions of people press Play at the same moment, Kafka handles everything without delay.

Durability and Replay

- Kafka keeps events stored and safe. If any service goes down, it can replay the missed events later.
- Example: Your watch history is never lost even if a service crashes. It can replay your actions and recover.

Summary

Kafka helps Netflix track and stream user actions in real time. It keeps data safe, scales easily and allows different services to react instantly.

158. Scenario 23: Real-time Analytics vs Task Queue

Scenario: You need to track user clicks, page views, and login activity from a web app and run real-time analytics on it. Which one would you choose, Kafka or RabbitMQ, and why?

Solution

- Kafka is better for event streaming, high throughput, replay ability, long-term retention, publish-subscribe model.
- RabbitMQ is not ideal because logs must be retained for analytics and reprocessing later.
- Kafka supports consumer groups well and scales better for analytics workloads.

159. Scenario 24: Order Processing in E-Commerce

Scenario: You are designing order processing in an e-commerce platform. When a user places an order, the system should trigger email notification. Which one would you choose, Kafka or RabbitMQ, and why?

Solution

RabbitMQ fits better for transactional flows that need acknowledgement, priority queues, and retries.

Kafka is event streaming oriented; not ideal when message ordering and acknowledgment per message is critical.

So RabbitMQ is better here for order pipelines if real-time analytics is not needed.

160. Scenario 25: Microservices Communication

Scenario: You have 25+ microservices that need to communicate using domain events like USER_CREATED, ORDER_PLACED, PAYMENT_SUCCESS. Would you recommend Kafka or RabbitMQ??

Solution

- Kafka is better for event-driven microservices.
- Services subscribe to domain events decoupled.
- Kafka handles replay, scaling, audit logs.
- RabbitMQ is more point-to-point

161. Scenario 26: Payment Retry System

Scenario: In a payment processing system, failed payments must be retried 5 times, then moved to a Dead Letter Queue. Messages must be processed exactly once. Kafka or RabbitMQ?

Solution

- RabbitMQ is better due to Dead Letter Queue and retry policies out of the box.
- Kafka can do it, but needs custom implementation.
- Better choice: RabbitMQ for message orchestration + Kafka only if event history is required.

What is Dead Letter Queue?

A dead-letter queue (DLQ) is a special type of message queue that stores messages that could not be processed or delivered to their intended recipient. It acts as a "catch-all" for messages that fail due to errors, like incorrect formatting, network issues, or system failures, and is used to isolate these problematic messages for later analysis and handling without stopping the main message flow

162. Scenario 27: Real-Time Fraud Detection

Scenario: A banking app wants to process transaction streams in real-time to detect fraud patterns using ML models. System must support millions of messages per second. Kafka or RabbitMQ? and why?

Solution

- Kafka is the right choice.
- Supports high throughput + partitioning + streaming.
- RabbitMQ can't handle massive streaming workloads.

163. Scenario 28: Deploying 50 Microservices - Docker vs Kubernetes

Scenario: Your team has 50 microservices, each deployed separately using Docker on virtual machines. Managing deployments, scaling and service discovery is becoming hard. What will you use and why?

Solution

- In this situation, Kubernetes becomes the natural choice. Docker alone only solves the problem of packaging and running containers. But we also need automation for deployment, scaling, service discovery, and high availability.
- Kubernetes gives us exactly that. It can restart crashed containers automatically, distribute traffic across services, and scale up during heavy load.
- With Kubernetes Deployments, we can push new versions without downtime.
- With Kubernetes Services, we get built-in service discovery. So, for large microservice deployments, Kubernetes is preferred over plain Docker.

164. Scenario 29: Zero Downtime Deployment

Scenario: You need to deploy a new version of an order service without downtime. Which deployment strategy will you choose and why? Blue-Green or Rolling Updates?

Ex: Deploy a new version of a payments API

Solution

- Blue-Green runs two environments side by side, old and new.
- Traffic instantly switches only after testing, so rollback is easy.
- Rolling update is safer for simple deployments but slower to recover from production bugs.
- For high-risk services like payments, **Blue-Green deployment is safer.**

If fast rollback is important, **Blue-Green deployment** is a safe choice. In this strategy, two versions of the application run at the same time. The current version (Blue) serves traffic, while the new version (Green) waits. After testing, traffic is switched to Green instantly. If something goes wrong, you simply switch back to Blue. The downside is that it uses extra infrastructure temporarily.

165. Scenario 30: Managing Dev, QA, Stage, and Production Environments

Scenario: You need to manage deployments for Dev, QA, Stage, and Prod for 25 microservices. How would you design the deployment strategy?

Solution

The best approach is to use CI/CD pipelines and Kubernetes namespaces.

Each environment gets its own namespace inside the Kubernetes cluster.

Configuration values like database URLs or API keys are stored separately using ConfigMaps and Secrets.

Tools like Helm make it easy to manage environment-specific change

166. Scenario 31: Handling Traffic Spikes

Scenario: Your microservice system gets 10X traffic during festive sales. How would you handle deployment to scale quickly??

Solution

In Kubernetes, scaling is handled automatically using **Horizontal Pod Autoscaling (HPA)**. When CPU or memory usage increases, HPA adds more pods. If load reduces, it scales back down to save cost.

For very high traffic, we can also enable **Cluster Autoscaling**, so that even new nodes are added automatically. This makes sure the system stays healthy under stress.

Combined with **rolling deployments**, new versions can be released even during high traffic without any service interruption.

Reference links for more info:

[Kubernetes: The Magic of Automated Horizontal Scaling | by Akshay Kumar | Medium](#)

[Kubernetes Horizontal vs Vertical scaling - GeeksforGeeks](#)

167. Scenario 32: Ensuring Backward Compatibility when a breaking changes in a Microservice:

Scenario: Your microservice new deployment introduced some issues, how do you ensure backward compatibility

Solution

API Versioning: Simple way to handle this is to introduce changes slowly and keep both versions running for a while. For example, if we add a new field or change how an API responds, we keep the old response format active until every other service is ready to move to the new one. This is often done by versioning the API. One endpoint serves the older format, and another endpoint serves the new format.

Graceful fallbacks are another good technique. If a client still calls an endpoint, we return a safe default or convert the request to the new format inside the service. This avoids breaking older clients even when they haven't upgraded yet.

Clear communication also matters. When a change is coming, we share release notes, examples, and migration steps with all consumers. This gives teams enough time to adjust.

168. **Scenario 33: Managing Dev, QA, Stage, and Production Environments**

Scenario: You need to manage deployments for Dev, QA, Stage, and Prod for 25 microservices. How would you design the deployment strategy?

Solution

The best approach is to use CI/CD pipelines and Kubernetes namespaces.

Each environment gets its own namespace inside the Kubernetes cluster.

Configuration values like database URLs or API keys are stored separately using ConfigMaps and Secrets.

Tools like Helm make it easy to manage environment-specific change

169. **Scenario 34: Choosing deployment strategy**

Scenario: Your team is releasing a new version of the Recommendation Service in production. The code change is risky because it includes a new ML model. The team wants zero downtime but quick rollback if something goes wrong. Which deployment strategy will you choose: Blue-Green, Rolling, or Canary?

Explanation

- **Rolling Deployment** replaces old servers/pods gradually with new ones but treats all users the same. It is safe for normal releases but not ideal for risky changes.
- **Blue-Green Deployment** allows quick rollback but still shifts 100 percent of traffic to the new version at once. Risk is high if a hidden bug exists.
- **Canary Deployment** sends the new version to a small percentage of users first. If metrics look good, traffic is increased step by step. If something fails, rollback is instant with minimal impact.
- For risky releases, **Canary Deployment is the best choice** because it allows controlled rollout and protects user experience.

170. **Scenario 35: Service Failure**

Scenario: A microservice called Order Service depends on Payment Service. During high traffic, Payment Service becomes slow and starts timing out. As a result, Order Service also hangs and users can't place new orders. How will you prevent one failed service from affecting others?

Explanation

1. **This is a classic cascading failure scenario.** When a downstream service becomes slow, it blocks upstream services. To prevent this, we use a **Circuit Breaker** pattern. If Payment Service is failing repeatedly, the circuit breaker stops making new calls to it for a short time. During this time, Order Service can respond faster with a fallback like "Payment temporarily unavailable, try again."

Libraries like **Resilience4j** or **Hystrix** implement this easily.

2. Retry Mechanism with Exponential Backoff

Retries failed requests after a delay. The delay increases with each failure (exponential backoff). Useful for handling temporary issues like network glitches.

3. Fallback Mechanism

Provides a default response or alternative flow when a service fails. Keeps the user experience smooth by returning cached data or a friendly message.

4. Bulkhead pattern

The bulkhead pattern is a resilience design pattern in microservices that isolates components and resources to prevent a failure in one from cascading and taking down the entire system

For ex: let's assume that there are 2 Order Service and Payment Service. Order Service depends on Payment Service. For some reason, Payment Service is very slow. So, when we get multiple concurrent requests to Order Service, Order Service performance will also get affected. It could block Order Service's threads. Due to that Order Service might not be able to serve other requests which do NOT depend on Payment Service. So, the idea here is to isolate resources / allocate some threads in Order Service for Payment. So that we do not consume all the threads of Order Service and prevent Order service from hanging for all the requests!

Reference for bulkhead pattern: [Resilient Microservice Design – Bulkhead Pattern](#)

171. Scenario 36: Retry without causing overload

Scenario: Your Notification Service sends emails and SMS using a third-party provider. Sometimes the provider fails temporarily due to network issues. You try retries, but retrying too quickly causes overload. How will you retry safely?

Explanation

Blind retries can make things worse during an outage. Instead, use **Retry with Exponential Backoff**. That means first retry after 1 second, then 2 seconds, then 4 seconds, and so on

172. Scenario 37: Fallback on failure

Scenario: A travel Booking app collects hotel reviews from an external Review Service. When that service goes down or becomes slow, the entire booking page fails. The business team says even if reviews are not available, the page must load. How will you handle this failure??

Solution

Option 1: Fallback with Cached Response

If the external service fails:

- Try reading cached reviews from Redis or in-memory cache.
- If even cache is empty, show a friendly message:
"Reviews are temporarily unavailable. Please try again later."

This avoids a broken page and keeps the user experience smooth.

Option 2: Use Resilience4j Fallback (Java Example)

```
public String getHotelReviews(String hotelId) {  
    return Try.of(() -> reviewClient.getReviews(hotelId))  
        .recover(ex -> getCachedReviews(hotelId))  
        .get();  
}  
  
private String getCachedReviews(String hotelId) {  
    return redisClient.get("reviews:" + hotelId); // fallback cache  
}
```

This code:

- First tries the real Review Service.
- If it fails due to timeout or exception, it automatically calls fallback logic.

173. Scenario 38: Limit the resources

Scenario: Your app has three features: search, Wishlist, and user profile. During a sale event, the search feature got overloaded, and because everything shared the same thread pool, the whole system slowed down. How will you isolate failures?

Explanation:

Use the **Bulkhead Pattern**. Just like compartments in a ship prevent it from sinking, bulkheads isolate failures. Assign separate resource limits to each feature. So, if search goes down, Wishlist and profile still work.

174. Scenario 39: Dynamic IPs in microservices

Scenario: In your microservices system, services are running in Docker containers and Kubernetes pods. Their IP addresses keep changing every time they restart or scale. Hardcoding IPs in configuration files has become a maintenance nightmare. How do we make services discover each other dynamically?

Explanation:

This is exactly why a **Service Registry** is used. Services must not depend on fixed IP addresses. Instead, each service registers itself with a central directory called a service registry. Other services ask this registry to discover where to send requests.

For example, if we use **Eureka**, the Order Service registers itself as ORDER-SERVICE. The Payment Service does not need its IP. It simply asks Eureka for ORDER-SERVICE and Eureka returns the active instances. This solves the problem of dynamic service locations and avoids manual configuration changes.

175. Scenario 40: Load Balancing without an external Load Balancer

Question:

We have multiple instances of the Product Service running. The challenge is, how do other services know which Product Service instance to call? Do we always need an external load balancer?

Explanation:

No, we do not need an external load balancer if we use a service registry. When services register themselves with Eureka or Consul, the registry keeps track of all instances. A client can then use client-side load balancing.

For example, Spring Cloud LoadBalancer with Eureka can pick a Product Service instance automatically before sending a request. This improves performance and removes the dependency on traditional load balancers like NGINX for internal service communication.

176. Scenario 41: Handling service failures using service registry**Question:**

What happens if one instance of a service crash? How do other services avoid calling a dead instance?

Explanation:

A service registry performs health checks and maintains a list of only healthy service instances.

If one instance fails, it is automatically removed from the registry. Other services will not call it because they do not see it anymore.

For example, in Eureka, services send heartbeat signals to the registry. If no heartbeat is received for a fixed time, Eureka marks the instance as unavailable. This makes the system fault tolerant.

177. Scenario 42: Handling usage spikes

Scenario: Your e-commerce website receives normal traffic most of the year, but during festivals and flash sales, traffic increases by 10 times. Single server deployment is failing even after increasing CPU and RAM. How do we handle this situation?

Explanation:

- Horizontal scaling is a better solution than vertical scaling in this case. Instead of using one powerful server, we add more servers and distribute traffic across them using a load balancer like Nginx or AWS Elastic Load Balancer (ELB).
- The load balancer acts like a traffic manager and sends user requests to available servers. This improves performance and keeps the system stable even when traffic spikes.
- This approach is cheaper, more fault-tolerant and easier to scale.

178. Scenario 43: Load balancing inside microservices

Scenario: A blog platform runs on 5 servers behind a load balancer. But your monitoring dashboard shows something strange. One server is overloaded, and others are free. Why is this happening and how will you fix it?

Explanation:

- This usually happens when the wrong load balancing algorithm is used. If **Round Robin** is used blindly, traffic is spread equally without checking server load.
- Instead, using **Least Connections** or **Least Response Time** algorithm will distribute requests based on current load.

- Example: If Server A has 2 active users and Server B has 10 active users, the load balancer should send new users to Server A first. Choosing the right load balancing strategy avoids uneven load and improves reliability.

179. **Scenario 44: Some servers are overloaded while others are idle**

Scenario: In a microservices system, the Order Service calls the Payment Service which has 3 running instances. Should we use an external load balancer here?

Explanation:

- For internal service-to-service communication, we use client-side load balancing instead of a full network load balancer.
- Tools like Spring Cloud LoadBalancer, Ribbon (legacy), gRPC Load Balancing, or Consul pick a Payment Service instance automatically from a service registry.
- This reduces network overhead and improves performance. Use API Gateway + Service Registry + Client-side Load Balancer for clean microservice traffic management.

180. **Scenario 45: Rate-limiting**

Scenario: A banking application exposes a Login API. Hackers are trying to brute-force passwords by making thousands of login attempts per minute, causing system slowdown. How will you protect this service?

Explanation: This problem is not about performance; it is about abuse.

- We need to limit requests per user or per IP using rate limiting.
- For example, allow only 5 login requests per minute per user/IP. If someone exceeds this, block them temporarily. This protects the system from malicious attacks.
- In real systems, this can be done with API Gateway rate limiting (NGINX, Kong, AWS API Gateway) or with Resilience4j RateLimiter inside microservices.

181. **Scenario 46: Debugging Failures in a Microservice Chain**

Scenario: Your system has 12 microservices. A user reports that their order is not getting confirmed. The request flows through API Gateway > Order Service > Payment Service > Inventory Service. Logs show nothing helpful. How will you trace the exact point of failure??

Explanation:

In distributed systems, logs from one service are not enough. We need distributed tracing. Using tools like Zipkin with OpenTelemetry, we assign a trace ID to each request at the gateway. This trace ID is passed across services so we can follow the complete journey of the request. When a failure happens, we see which service took how long and where the request stopped.

182. Scenario 47: Handling High Volume Logs

Scenario: Your application generates millions of logs every day. Developers need logs for debugging but storing them on each server is filling disk space and making the system slow. How do you handle centralized logging?

Explanation:

Storing logs locally is risky and hard to search. The real solution is **centralized logging**. We send all logs to a common storage using the ELK stack (Elasticsearch, Logstash, Kibana)

We can search logs by service, trace ID or user ID. For example, we can search: `orderId = 5678 AND level = ERROR`. This makes debugging faster and keeps the system clean.

183. Scenario 48: Detecting Problems Before Users Report Them

Scenario: In production, issues are found only after users complain. The team wants a way to detect performance issues or service failures early. How would you do monitoring?

Explanation:

This is not just logging. We need metrics and alerting.

Tools like **Prometheus + Grafana** can collect metrics like CPU usage, memory, latency and request failure counts.

Then we define alerts. For example: "Trigger alert if error rate for Payment Service is more than 5 percent for 2 minutes." We can use alerting systems like **PagerDuty** to notify the Engineering teams or support teams.

184. Scenario 49: Debugging Slow or failing Requests

Scenario: Users report that some API calls are slow or failing but only at certain times. Logs are normal. Servers are healthy. How do you find the root cause of latency

Explanation:

For debugging slow API requests in microservices, we use **Distributed Tracing**. In Spring-based systems, **Spring Cloud Sleuth** adds tracing information like `traceId` and `spanId` to every request and log. Then we send this tracing data to a tool like **Zipkin** to see a timeline view of where time is being spent.

185. Scenario 50: Partial Failure

Scenario: Imagine we are building an online food ordering system. When a customer places an order, three services work together:

1. Order Service creates the order
2. Payment Service collects the money
3. Restaurant Service confirms the food availability

Now imagine this flow

- Order Service successfully creates the order

- Payment Service successfully takes the payment
- Restaurant Service fails due to timeout

Money is taken but food is not confirmed. The system is stuck in the middle. That is a partial failure.

Solution:

To handle failures safely, we use four key ideas: **Retry**, **Circuit Breaker**, **Compensation**, and **Eventual Consistency**.

1. Retry - Try again safely

Sometimes services fail for temporary reasons like network delays or slow responses. In such cases, retrying solves the problem.

But retry must be safe

- Retry only for temporary errors, not for bad requests
- Use retry with backoff to avoid overloading the service
- Set retry limits

Example

If the Restaurant Service fails once due to timeout, Order Service retries 2 more times before giving up.

2. Circuit Breaker - Stop failure from spreading

If a service keeps failing, calling it again and again is useless and dangerous. It causes delays and blocks threads. The Circuit Breaker pattern stops calling the failed service for a short time. This keeps the system fast and stable.

Example

If Restaurant Service is down, Circuit Breaker opens and Order Service immediately responds with "Restaurant is currently busy. Please try another restaurant."

This prevents long waiting time.

3. Saga pattern

When a failure happens after some steps succeed, we undo the completed steps using a compensation action.

Example

- Order is created
- Payment is taken
- Restaurant fails

To fix this, we run a compensation step: refund the payment and cancel the order.

This is called a Saga pattern in microservices.

4. Eventual Consistency

In the real world, everything is consistent immediately. In microservices, that is not always possible. Instead, we allow temporary inconsistency but fix it in the background.

Example

Payment Service sends a message: "Payment successful"

If Restaurant fails later, a compensating event is sent

Restaurant Service reads it and cancels the pending action
System becomes consistent after a short delay
This is called eventual consistency.

186. Scenario 51: Data consistency -1

Scenario: How do you ensure data consistency across services with separate databases??

Solution:

In a microservice system, each service has its own database. This gives flexibility and independence, but it also creates a challenge. How do we keep data consistent when one business operation touches many services? This becomes a serious problem in systems like banking, ecommerce, and booking platforms.

Let us take a simple example.

When a user places an online order, three services work together.

- The Order Service creates the order.
- The Payment Service collects money.
- The Inventory Service reduces stock.

Each service has its own database. What happens if payment succeeds but stock update fails? The system becomes inconsistent.

To solve this, we cannot use traditional database transactions because distributed systems do not support a single transaction across many services. Instead, we follow patterns built for microservices.

The Saga pattern breaks a big transaction into small local transactions. Each service completes its own step and publishes an event. If any step fails later, we run a compensating action. For example, refund payment and cancel the order.

We can also use Kafka to send events between services and keep them in sync. With these patterns, we reach eventual consistency, which is the practical form of consistency in microservices.

187. Scenario 52: Data consistency -2

Scenario: How do you ensure data consistency across services with separate databases??

Solution:

In a microservice system, each service has its own database. This gives flexibility and independence, but it also creates a challenge. How do we keep data consistent when one business operation touches many services? This becomes a serious problem in systems like banking, ecommerce, and booking platforms.

Let us take a simple example.

When a user places an online order, three services work together.

- The Order Service creates the order.

- The Payment Service collects money.
- The Inventory Service reduces stock.

Each service has its own database. What happens if payment succeeds but stock update fails? The system becomes inconsistent.

To solve this, we cannot use traditional database transactions because distributed systems do not support a single transaction across many services. Instead, we follow patterns built for microservices.

The Saga pattern breaks a big transaction into small local transactions. Each service completes its own step and publishes an event. If any step fails later, we run a compensating action. For example, refund payment and cancel the order.

We can also use Kafka to send events between services and keep them in sync. With these patterns, we reach eventual consistency, which is the practical form of consistency in microservices.

188. **Scenario 53: Sending notifications / Broadcast an event**

Scenario: Your registration service needs to send email or sms after successful creation of the user. How do you design it?

Solution:

When a new user registers, the system often needs to send a welcome email or an SMS. The main idea is simple: the registration flow should stay fast, and the notification work should happen in the background.

Here is a clean way to design it.

1. User registers

The registration service creates the user in the database.

Once the record is saved, the service adds a small event into a queue. This event only contains basic details like user id, email, or phone number.

2. Using a message queue

A queue helps us handle extra work without slowing down the main request.

The registration service pushes a message like "UserCreated" into the queue and finishes its job.

This keeps the registration API fast.

3. Notification service picks it up

A separate service listens to the queue.

Whenever a new event arrives, this service reads it and sends the email or SMS using a provider like AWS SES, SendGrid, or Twilio.

4. Handling failures

If sending fails, the event goes into a retry queue so the system can try again later.

This avoids losing notifications during temporary issues.

189. Scenario 54: Scaling the microservice

Scenario: Your microservice should scale according the traffic, how would you do it?

Solution:

Scaling a single microservice independently is one of the biggest advantages of microservices architecture. Here's how I approach it:

1. Containerization (Docker/Kubernetes)

Package the microservice into a container. Use Kubernetes (K8s) or Docker Swarm to run multiple instances (pods) of that service. This makes horizontal scaling easy by just increasing the replica count.

2. Load Balancing

Place a load balancer (NGINX, AWS ALB) in front of the microservice. Incoming requests are evenly distributed across multiple service instances. Ensures high availability and avoids overload on a single instance.

3. Auto-Scaling Policies

Define Horizontal Pod Autoscaling (HPA) in Kubernetes or AWS Auto Scaling Groups. Scale based on CPU, memory, or request latency thresholds.

Example: If CPU > 70% for 5 minutes then add 2 more instances.

4. Database & Cache Optimization

Use read replicas or sharding for databases to handle load. Implement caching (Redis, CDN) to reduce database hits. This ensures the microservice scales effectively without bottlenecks

190. Scenario 55: Polling

Scenario: In a ride booking system, different microservices work together.

Let's take two of them:

- **Driver Service:** holds driver status, location, and availability
- **Ride Service:** assigns rides to drivers

The Ride Service needs driver information to match riders quickly.

Now the question is: **Should Ride Service keep polling Driver Service again and again?**

No. That would overload both services and slow everything down.

There are better approaches.

1. Request Only What Is Needed (Direct API Call)

Ride Service can call a small API in Driver Service asking only the required data.

Example:

When a customer requests a ride, Ride Service sends a simple request like:

Driver Service returns only what is needed: `GET /driver/status?driverId=312`

```
{
  "driverId": 312,
  "status": "available",
  "location": "Madhapur"
}
```

```
}
```

No need to poll every few seconds. Ride Service asks only when required.

2. Event-Based Updates using Kafka (No Polling Needed, better approach)

A better approach is to let Driver Service **push** updates whenever something changes.

Whenever a driver changes status:

- comes online
- goes offline
- finishes a ride
- moves to a new location

Driver Service publishes an event like: DriverStatusUpdated

Ride Service subscribes to these events and updates its local cache.

This means Ride Service always has the latest driver information without asking again and again.

191. Scenario 56: Scheduler + Multiple Instances

Scenario: If I have a scheduler or cron job in my application, and it is deployed on multiple instances, will the scheduler job run simultaneously on all instances, how to prevent it?

Solution:

When we run our application on many instances, every instance loads the same code.

This means the scheduler or cron job inside the application will run **on every instance at the same time**.

If the job is supposed to run only once, this becomes a problem.

For example, consider a system that generates report logs every night.

If we have five instances, all five will run the cleanup at midnight, which is not what we want.

To avoid this, we make sure **only one instance runs the job**, even though the application is deployed everywhere.

1. Use a Distributed Lock (Most Common)

Before running the job, the instance tries to acquire a lock in a shared place like Redis, Mongo, MySQL, or Zookeeper.

Only the instance that gets the lock runs the job.

Others skip it.

Real example:

- A Report generating job runs every night.
- All instances try to acquire a Redis lock.
- Only one gets it and runs the job.
- The lock expires after the job finishes.
- This is the cleanest solution and used in many companies.

192. Scenario 57: API is failing

Scenario: Your external endpoint you are trying to call using Feign client is failing, how you handle this scenario?

Solution:

When an external endpoint called through a Feign client fails, we should handle it in a controlled way so that the failure does not break the whole application. The goal is to keep the system stable and give a safe response.

First, we avoid calling the failing service again and again. We do this by using a circuit breaker. If the service keeps failing, the circuit opens and stops further calls for some time. This protects our threads and resources.

Second, we provide a fallback response. A fallback is a simple backup logic that runs when the external call fails. Instead of throwing an error, we return default data or a safe message. This keeps the user flow working.

Third, we set timeouts. We do not wait forever for a response. If the service does not reply within a fixed time, we stop waiting and move to fallback logic.

```
@FeignClient(name = "product-service", fallback = ProductClientFallback.class)
interface ProductClient {
    Product getProductById(String id);
}

class ProductClientFallback implements ProductClient {
    public Product getProductById(String id) {
        return new Product(id, "Product unavailable");
    }
}
```

In this example, if the product service is down or slow, the fallback class runs and returns a safe response.

193. Scenario 58: Security

Scenario: In your application, multiple microservices need security like token validation, how you design it efficiently?

Solution: In a system with many microservices, doing token validation inside every service creates repetition and slows things down. A better design is to place token validation at a single-entry point. We use an **API gateway** that checks the token once for every request. If the token is valid, the request is allowed to go inside the system.

Inside the system, services trust the request because it already passed security. Services only focus on business logic, not security rules. For example, when a request comes from a mobile app, the gateway validates the token and forwards user details to services. This reduces duplicate work, improves performance, and keeps security logic in one clear place

194. Scenario 59: Read heavy Service

Scenario: One service (e.g., Catalog) gets most of the reads. How would you redesign to improve performance

Solution:

Approach 1: When one service like Catalog gets most of the reads, the first problem is repeated database hits for the same data. The redesign starts by separating read and write paths. We keep writes simple and safe, but reads are optimized.

For example, when a product is added or updated, we save it in the main database and also update a cache. For read requests, we first read from the cache instead of the database. If data is not found, we load it once and store it. We can also create a read-only copy of data shaped exactly for screens. This reduces load, speeds responses, and keeps the system calm under traffic.

Approach 2: We can use CQRS pattern, CQRS means we handle writing data and reading data in two different ways.

In a service like Catalog, most requests only ask for data. Very few requests change data. So we design the write side only to save correct data and follow business rules. We design the read side only to return data quickly.

For example, when a product is added or updated, we save it in the main database. At the same time, we copy that data into a fast store made only for reading. When screens need product details, they read from this fast store, not from the main database. This keeps reads fast, lowers database load, and helps the system handle many users smoothly.